

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and communications technology

Software Design Document

Version 1.3

AIMS
Subject: ITSS-IT4549E

Group 23
Bùi Trung Hiếu - 20235932
Nguyễn Trần Bình - 20235899
Nguyễn Hữu Tuấn Tú - 20236005
Nguyễn Tuấn Khanh - 20235952
Trần Trung Nghĩa - 20235983

Hanoi, <June, 2026>

Table of Contents

1. Introduction.....	4
1.1 Objective.....	4
1.2 Scope.....	4
1.3 Glossary.....	5
1.4 References.....	7
2. Overall Description.....	9
2.1 General Overview.....	9
2.2 Assumptions/Constraints/Risks.....	9
2.2.1 Assumptions.....	9
2.2.2 Constraints.....	9
2.2.3 Risks.....	9
3. System Architecture and Architecture Design.....	10
3.1 Architectural Patterns.....	10
3.1.1. Client-Server Architecture.....	10
3.1.2. Layered Architecture.....	10
3.1.3. MVC Pattern.....	10
3.1.4. Service-Oriented Design.....	10
3.1.5. Repository Pattern.....	11
3.1.6. Factory Pattern.....	11
3.2 Interaction Diagrams.....	11
3.2.1 CUD Product.....	11
3.2.2 Pay order by PayPal.....	19
3.2.3 Pay order by VietQR.....	22
3.2.4 Place order.....	26
3.2.5 View Product Detail.....	29
3.3 Analysis Class Diagrams.....	32
3.3.1 CUD Product.....	32
3.3.2 Pay order by VietQR.....	35
3.3.2 Pay order by PayPal.....	37
3.3.3 Place order.....	38
3.3.4 View Product Detail.....	39
3.4 Unified Analysis Class Diagram.....	40
3.5 Security Software Architecture.....	40
3.5.1 Authentication.....	40
3.5.2. Authorization.....	41

3.5.3. Encryption.....	41
3.5.4. Logging and Auditing.....	41
4. Detailed Design.....	43
4.1 User Interface Design.....	43
4.1.1 Screen Configuration Standardization.....	43
4.1.2 Screen Transition Diagrams.....	46
4.1.3 Screen Specifications.....	47
4.2 Data Modeling.....	91
4.2.1 Conceptual Data Modeling.....	91
4.2.2 Database Design.....	94
4.3 Non-Database Management System Files.....	101
4.4 Class Design.....	101
4.4.1 General Class Diagram.....	101
4.4.2 Class Diagrams.....	102
4.4.3 Class Design.....	119
5.Design Considerations.....	236
5.1 Goals and Guidelines.....	236
5.1.1 Design goals.....	236
5.1.2 Guidelines.....	237
5.2 Architectural Strategies.....	238
5.2.1 Monorepo and Technology Stack.....	238
5.2.2 Layered + Hexagonal (Ports & Adapters) Architecture.....	238
5.2.3 RESTful API and DTO-based Contracts.....	238
5.2.4 Frontend – Angular Single-Page Application.....	238
5.2.5 Persistence and Data Management.....	238
5.2.6 Security and Authentication.....	239
5.2.7 Payment Gateway Integration (pluggable).....	239
5.2.8 Notifications and Communication.....	239
5.2.9 Error Detection and Recovery.....	239
5.2.10 Concurrency and Synchronization.....	240
5.2.11 File and Asset Management.....	240
5.2.12 Configuration and Environment Management.....	240
5.2.13 Testing and Quality Assurance.....	240
5.2.14 Extensibility and Future Plans (OCP).....	240
5.3 Design and Program Evaluation.....	241
5.3.1 Cohesion & SRP.....	241
5.3.2 Coupling & Other SOLID.....	259
5.4 Design Patterns.....	269

5.4.1 Payment.....	269
5.4.2 View Product Detail.....	270
5.4.3 CUD Product.....	271
5.4.4 Place Order.....	272

1. Introduction

1.1 Objective

This Software Design Document (SDD) is developed to describe in detail the architecture and technical design decisions of the AIMS (An Internet Media Store) system.

Specifically, the document aims at the following objectives:

- Present the overall architecture of the system, including how the components interact with each other.
- Describe the detailed design of each functional module, the database, and the user interface.
- Serve as a common reference document to help the development team members align on the technical direction throughout the project.
- Fulfill the evaluation purpose of the ISD.20252 course as required by the instructor.

The intended audience for this document includes:

- Development team: Members directly involved in designing and programming the system.
- Instructor and Teaching Assistant: The individuals who evaluate the design quality and the implementation progress of the project.

1.2 Scope

AIMS (An Internet Media Store) is an online e-commerce application developed to facilitate the buying and selling of media products, including books, newspapers, CDs, DVDs, and related items.

The system includes the following main features:

- Product Browsing and Searching: Users can search, filter, and view detailed information about media products based on various criteria.
- Shopping Cart and Order Management: Allows users to add products to their cart, confirm delivery information, and place orders online.
- Online Payment: Integrates two payment gateways: PayPal (for international credit cards) and VietQR (for domestic bank transfers), along with an automatic cancellation mechanism for abandoned transactions.

- Order and Refund Management: Managers can approve or reject orders and process refund requests; the system automatically sends email notifications to customers.
- Authentication and Authorization: The system distinguishes between two main roles, Customer and Manager, utilizing a JWT-based authentication mechanism.

The following features are out of the current scope:

- AI/ML-based product recommendation systems.
- Integration with external sales channels (marketplaces).

Objectives and Expected Benefits:

- Provide a convenient and fast online shopping experience for end-users.
- Support managers in controlling the entire order lifecycle, from order placement to completion or refund.
- Build a scalable platform to easily integrate additional features in the future.

1.3 Glossary

Term	Explanation
AIMS	An Internet Media Store – The name of the system being developed in this project.
SDD	Software Design Document – A document describing the architecture and technical design of the system.
SRS	Software Requirements Specification – The higher-level document that this SDD references.
API	Application Programming Interface – A set of endpoints that allow the client and server to communicate with each

	other.
REST / RESTful	Representational State Transfer – An API architectural style that uses the HTTP protocol with GET, POST, PUT, and DELETE methods.
JWT	JSON Web Token – A token encoding standard used to authenticate users after logging in.
ORM	Object-Relational Mapping – A tool that maps objects in the code to tables in a relational database. The project uses Prisma as its ORM.
Monorepo	A source code organization model where both the frontend and backend reside in a single repository. The project uses Nx Workspace.
Angular	A frontend framework developed by Google, utilizing TypeScript, used to build the user interface of AIMS.
NestJS	A backend framework for Node.js, utilizing TypeScript, used to build the RESTful API of AIMS.
Prisma	A modern ORM for Node.js/TypeScript, used to interact with the PostgreSQL database in the project.
PostgreSQL	An open-source relational database management system, used as the primary database for AIMS.
VietQR	A domestic payment QR code standard in Vietnam, integrated into AIMS to support bank transfers.
PayPal	An international payment gateway, integrated into AIMS to support credit card payments.

Clouldinary	A cloud-based image and video storage and management service, used to store product images in AIMS.
Invoice	An electronic invoice automatically generated after an order is successfully paid.
Refund	The process of returning money to a customer when a paid order is canceled.
Manager	The administrator role in the system, holding permissions to manage products, approve or reject orders, and process refunds.
VAT	Value Added Tax – A tax calculated and displayed on the system's invoices.
ERD	Entity-Relationship Diagram – A diagram used to describe the database structure.
Nx	A next-generation monorepo build and management tool, supporting both Angular and NestJS within the same workspace.

1.4 References

- N. T. T. Trang, "AIMS – Problem Statement (Version 3.1.1)," Tài liệu môn học ISD.20252, Khoa Khoa học máy tính, Đại học Bách khoa Hà Nội, 2025.
- NestJS Documentation. (2024). NestJS – A progressive Node.js framework. Retrieved from <https://docs.nestjs.com>
- Angular Documentation. (2024). Angular – The web development framework for building the future. Retrieved from <https://angular.dev>
- Prisma Documentation. (2024). Prisma ORM – Next-generation Node.js and TypeScript ORM. Retrieved from <https://www.prisma.io/docs>
- VietQR. (2024). VietQR – Tiêu chuẩn mã QR thanh toán Việt Nam. Retrieved from <https://www.vietqr.io>

- PayPal Developer Documentation. (2024). PayPal REST APIs. Retrieved from <https://developer.paypal.com/docs/api/overview>

2. Overall Description

This section describes the guidelines and strategies for designing and implementing the AIMS system.

2.1 General Overview

AIMS (An Internet Media Store) is a web-based e-commerce application designed with a Client-Server architecture. The project is organized using an Nx Monorepo. The frontend is an Angular Single Page Application and the backend is a NestJS REST API using Prisma ORM and a PostgreSQL database. The system must operate 24/7 and recover within a maximum of one hour after an incident. It is designed to serve up to 1,000 concurrent customers, maintaining a response time of under 2 seconds under normal conditions.

2.2 Assumptions/Constraints/Risks

2.2.1 Assumptions

The system assumes the stable operation of external APIs for PayPal Sandbox and VietQR payment processing. It also assumes that email delivery is reliably managed via the integrated Resend libraries, and that product images are efficiently hosted externally via the Cloudinary integration.

2.2.2 Constraints

Development is constrained strictly to the Nx, Angular, NestJS, and PostgreSQL stack. Business constraints dictate that product prices must always remain between 30% and 150% of their original value. Managers can only delete products with zero stock, and are limited to deleting 10 items per action and a maximum of 20 products per day; otherwise, items must be deactivated. Security constraints require all user passwords to be securely hashed using bcrypt, and administrators are strictly prohibited from viewing actual passwords. Finally, VietQR refunds cannot be automated and must be handled manually.

2.2.3 Risks

A key risk is stock mismatch during concurrent purchases, which must be managed using database transactions. Performance degradation under the 1,000-user load limit is another risk, requiring careful database query optimization. External payment API downtime poses a risk to the checkout process, requiring clear application-level error handling. Lastly, potential data loss is a risk that must be addressed through proper PostgreSQL backup procedures to ensure the system meets the mandatory one-hour recovery constraint.

3. System Architecture and Architecture Design

3.1 Architectural Patterns

The system applies a layered architecture combined with client-server communication and service-based separation of responsibilities. These architectural patterns are chosen to improve maintainability, scalability, testability, and separation of concerns.

3.1.1. Client-Server Architecture

The application follows a client-server architecture, where the frontend client provides the user interface and sends requests to the backend server through APIs. The backend server handles business logic, data validation, payment processing, order management, and database access.

This pattern was chosen because it clearly separates the presentation layer from the business and data layers. It also allows the frontend and backend to be developed, tested, deployed, and scaled independently.

3.1.2. Layered Architecture

The backend is organized into layers, including controller, service, repository, and entity/model layers. Controllers receive requests and return responses. Services contain business rules and application logic. Repositories handle data access. Entities or models represent core business objects such as users, products, orders, payments, and transactions.

This pattern was chosen to make the system easier to understand, maintain, and extend. It prevents business logic from being mixed directly with user interface code or database queries.

3.1.3. MVC Pattern

The frontend follows the Model-View-Controller or similar presentation pattern. Views display data to users, models represent application data, and controllers or components handle user interactions and coordinate communication with backend APIs.

This pattern was chosen because it supports a clear separation between data, interface, and interaction logic. It helps keep the user interface organized and easier to modify.

3.1.4. Service-Oriented Design

The system separates major business capabilities into services, such as user service, product service, cart service, order service, payment service, and delivery service. Each

service is responsible for a specific business area and exposes operations related to that area.

This pattern was chosen to reduce coupling between business functions. It also makes the system easier to test, reuse, and extend when new features or external integrations are added.

3.1.5. Repository Pattern

The repository pattern is used to isolate database operations from business logic. Services interact with repositories instead of directly writing database queries.

This pattern was chosen because it improves maintainability and testability. It allows the data access implementation to change without affecting the business logic.

3.1.6. Factory Pattern

The factory pattern is used for selecting and creating suitable payment gateway handlers, such as different payment methods or external providers.

This pattern was chosen because payment processing may involve multiple providers or methods. A factory helps the system add or replace payment gateways without changing the main payment workflow.

Overall, these architectural patterns provide a structured and flexible design. They support clear responsibility separation, easier testing, better maintainability, and future extension of the application.

3.2 Interaction Diagrams

3.2.1 CUD Product

Product Management is a core task for any Product Manager. It involves adding new products, updating their details, adjusting quantities, and deleting items. The system keeps a record (audit log) of every change made and strictly follows specific business rules

3.2.1.1 Add Product

Description: This diagram shows the step-by-step interaction between different parts of the system when running the "Add Product" Use Case.

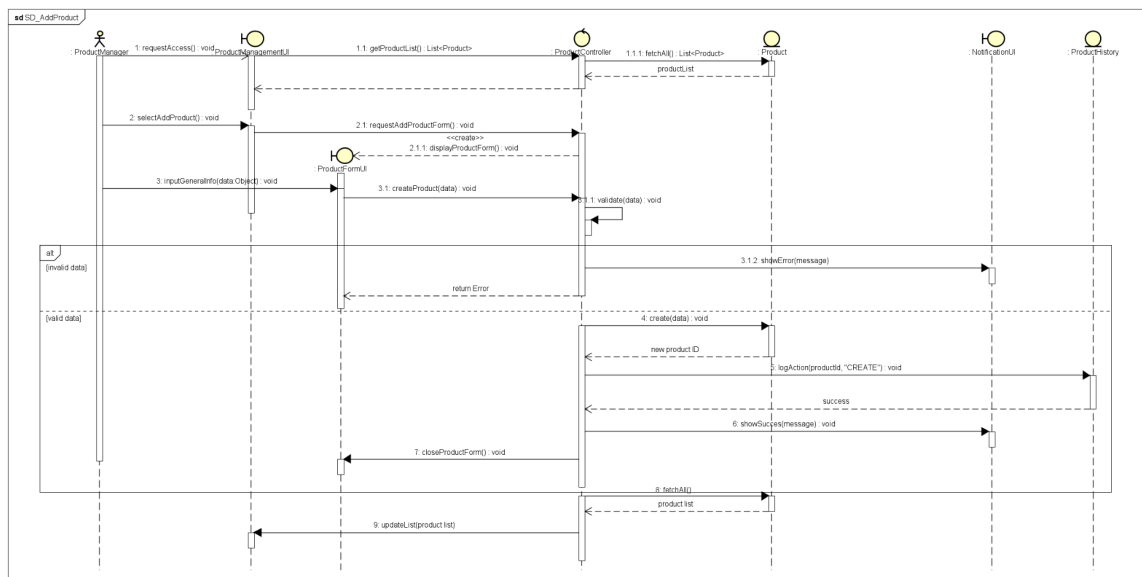
Pre-condition: The user has successfully logged in as a Product Manager and opened the "Add New Product" screen.

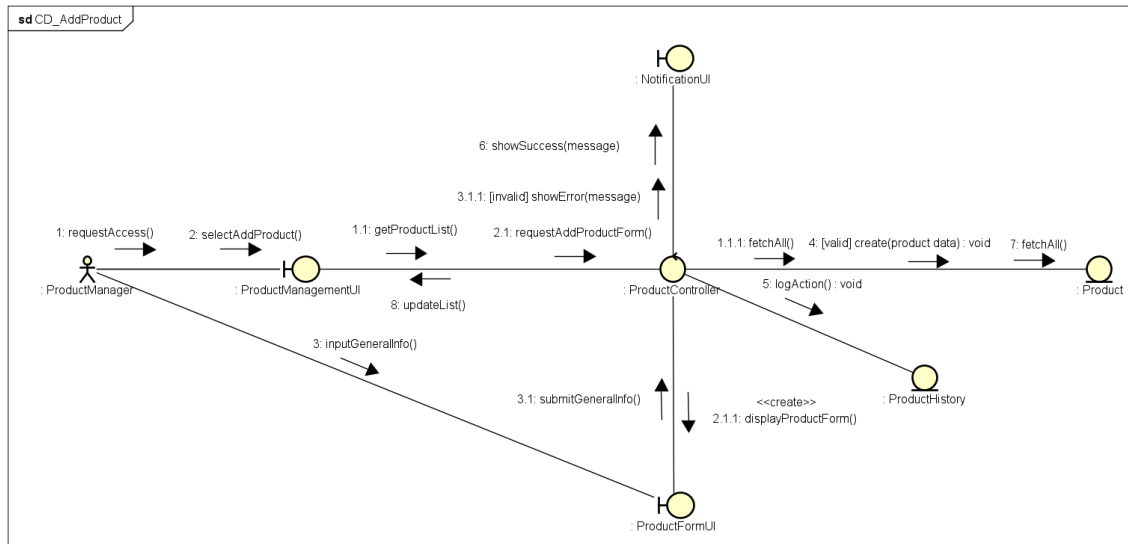
Main Flow:

- Receiving the request: The interface (Boundary - ProductFormUI) collects all the data entered by the user and sends a request to create a new product to the main controller (ProductController).
- Handling the logic: The controller (Control - ProductController) checks (validates) if the input data is correct. Depending on the product type (like a Book, CD, etc.), the system will check for specific required fields.
- Saving data: If the data is valid, the controller asks the system (Entity - Product) to create and save the new product's information.
- Audit logging: After successfully creating the product, the controller automatically tells the log (Entity - ProductLog) to record this action in the history.
- Feedback: The interface receives a success signal from the controller and shows a "Product added successfully" message to the Product Manager.

Business Rules:

- Unlike deleting products (which is limited to 20 items per day), there is no limit on how many products you can add.
- Every time a new product is added, it must be recorded in the History Log so it can be reviewed later if needed.





3.2.1.2 Update Product

Description: This diagram shows the step-by-step process when a Product Manager updates the details of an existing product.

Pre-condition: The user has successfully logged in as a Product Manager and selected a valid product to edit.

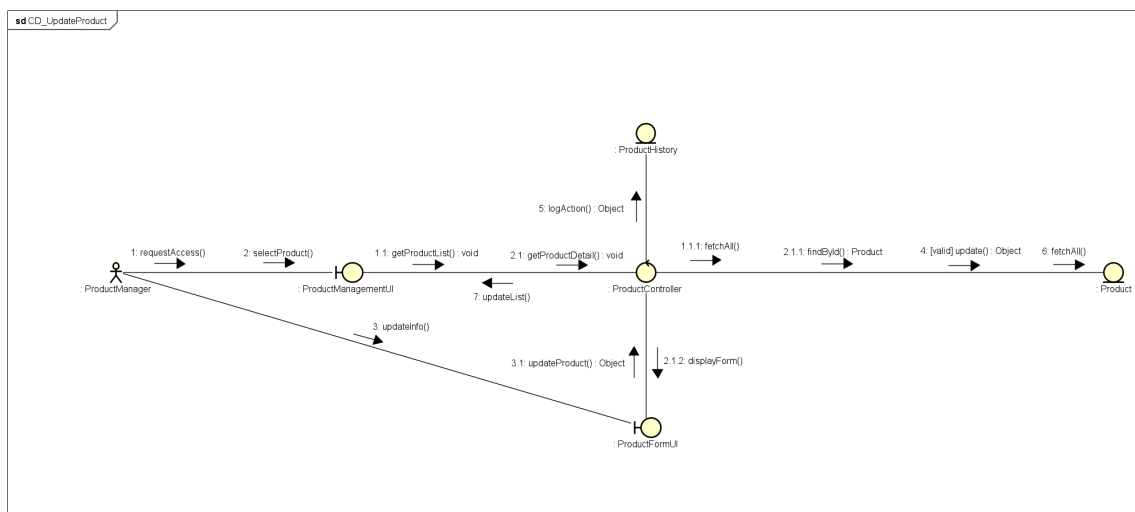
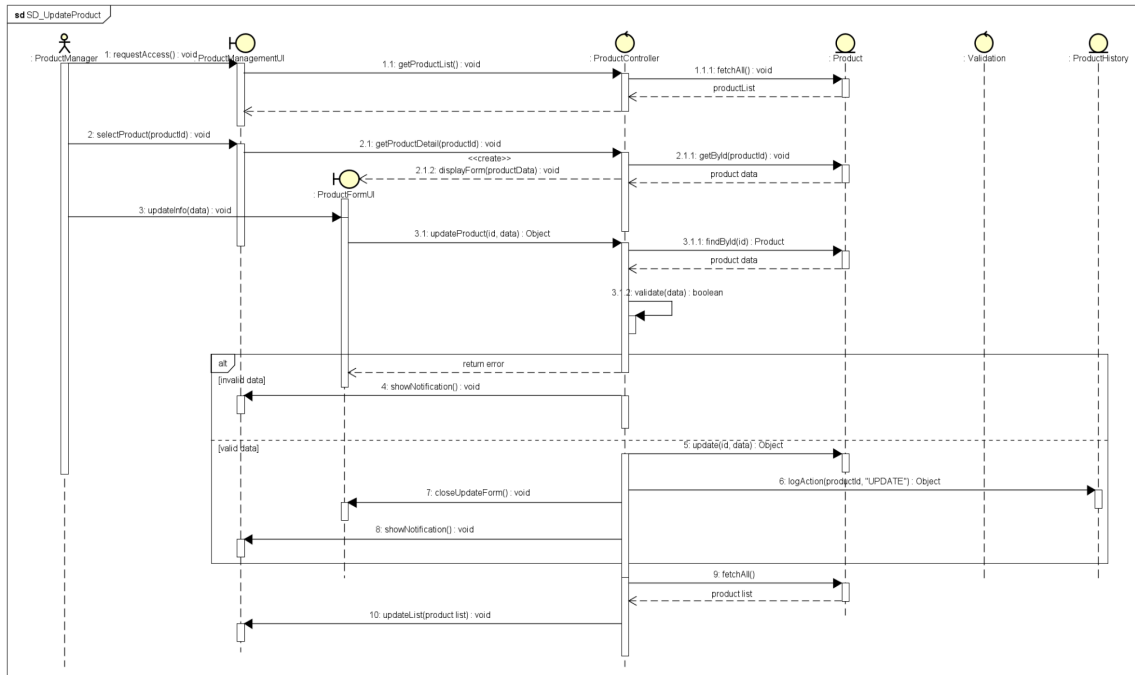
Main Flow:

- Receiving the request: The interface (Boundary - ProductFormUI) receives the new edits (which could be typing in standard text fields or uploading a new image) and sends them to the main controller (ProductController).
- Loading current info: The controller (ProductController) retrieves the product's current information from the system (Entity - Product) to compare and apply the changes.
- Saving data: The controller tells the system (Entity - Product) to process and save the updates.
- Audit logging: The update action is securely recorded in the history log (Entity - ProductLog).
- Feedback: The system sends a success signal back to the interface (Boundary) so the user knows the update worked.

Business Rules:

- Just like adding new items, there is no daily limit on how many products you can update.

- Any changes to the selling price must stay within an allowed range (30% to 150% of the original price). This strict rule is in place to prevent price dumping (selling too cheap) or price gouging (pricing too high), as outlined in the system specifications.



3.2.1.3 Adjust Quantity

Description: The "Adjust Quantity" flow is kept in a separate area on the detail screen. It is designed to help with inventory checks, such as adding new stock when it arrives or removing items that are damaged.

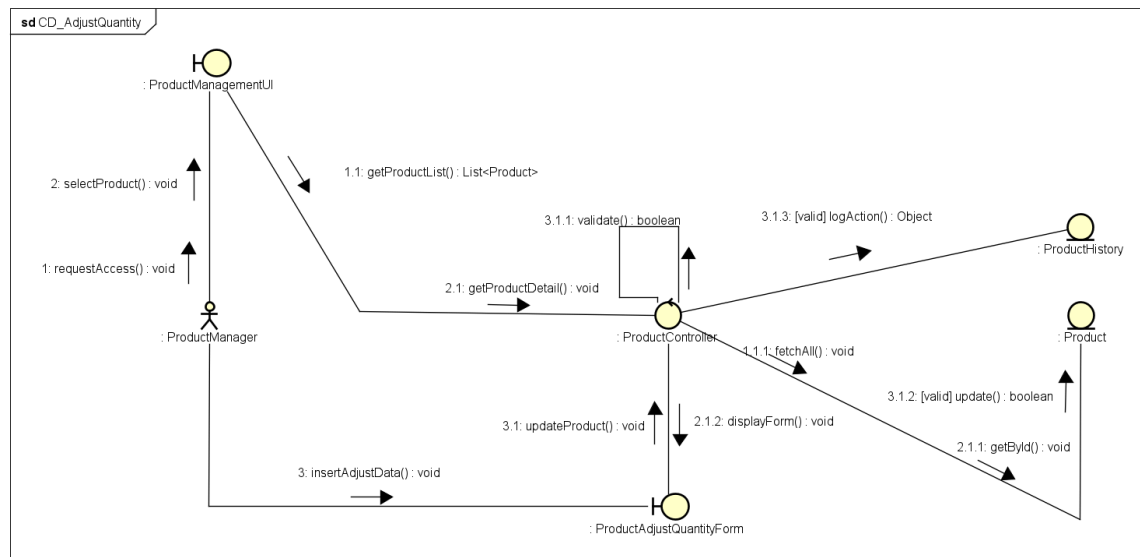
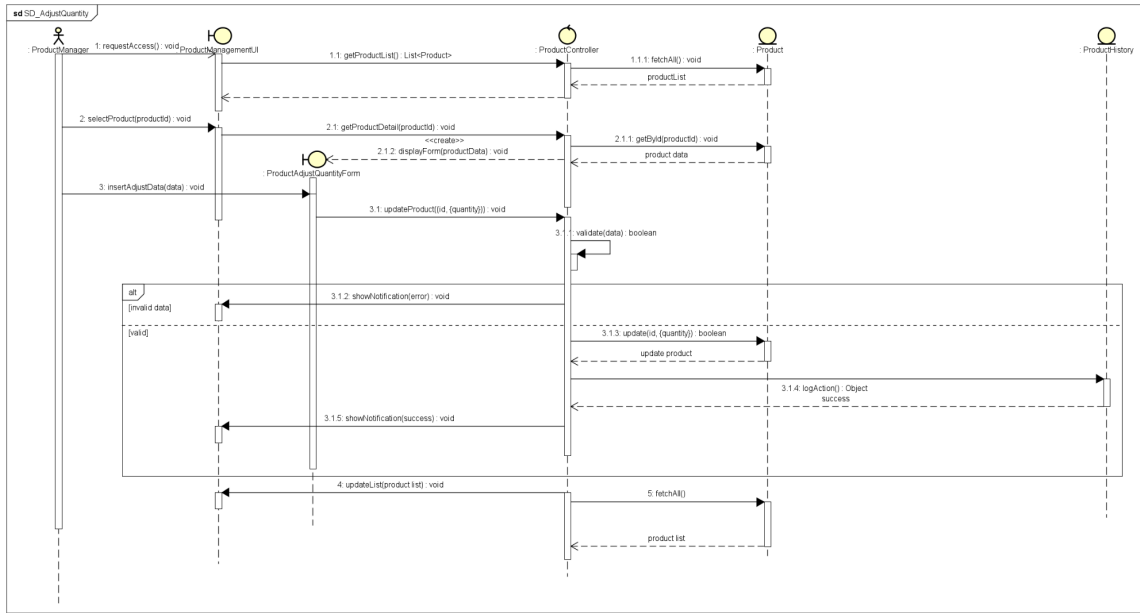
Pre-condition: The Product Manager is currently viewing a product's details (on the ProductDetailUI screen).

Main Flow:

- Receiving the request: The interface (Boundary - ProductDetailUI) opens a pop-up box (Modal) asking the user to type in the new quantity and provide a reason for the change.
- Updating data: The system reuses the standard update process (ProductController), but it focuses specifically on changing only the quantity field of the product (Entity - Product).
- Specific audit logging: This adjustment is securely recorded in the history log (Entity - ProductLog). It strictly requires the "Reason for change" to be included in the log (for example: "Damaged goods").
- Feedback: The system confirms the success, and the screen automatically updates to show the new quantity.

Business Rules:

- Any manual adjustments to the inventory require a clear, transparent reason (such as a system error, damaged goods, or adding new stock), as strictly required by the Problem Statement.
- The system will not process the update if the user leaves the reason field blank.



3.2.1.4 Delete Product

Description: The "Delete Product" flow is a complex process that is strictly guided by the system's security and data protection rules.

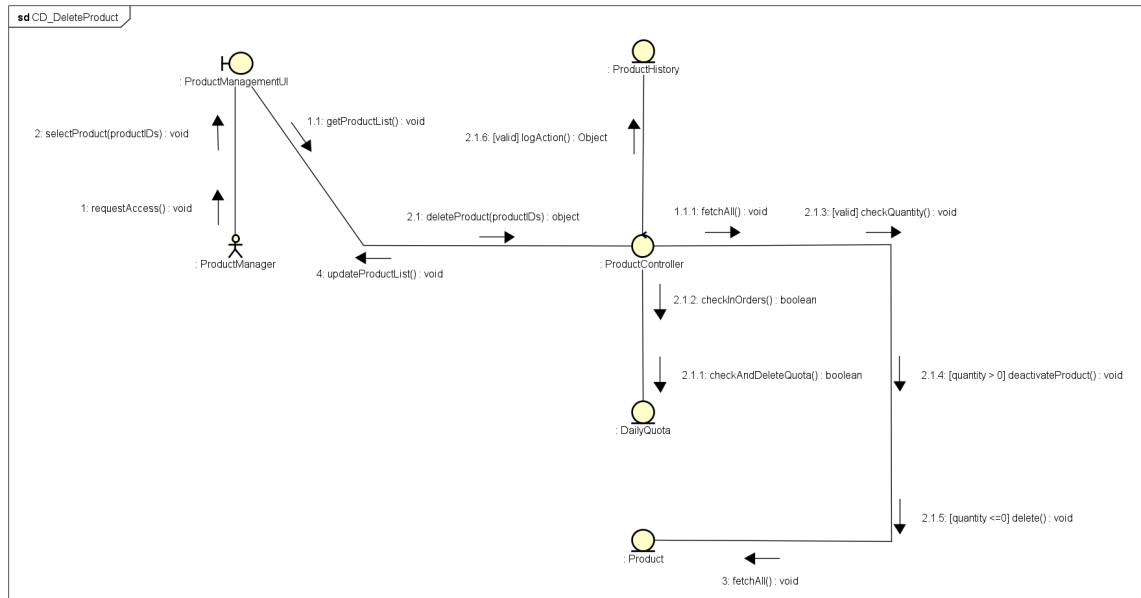
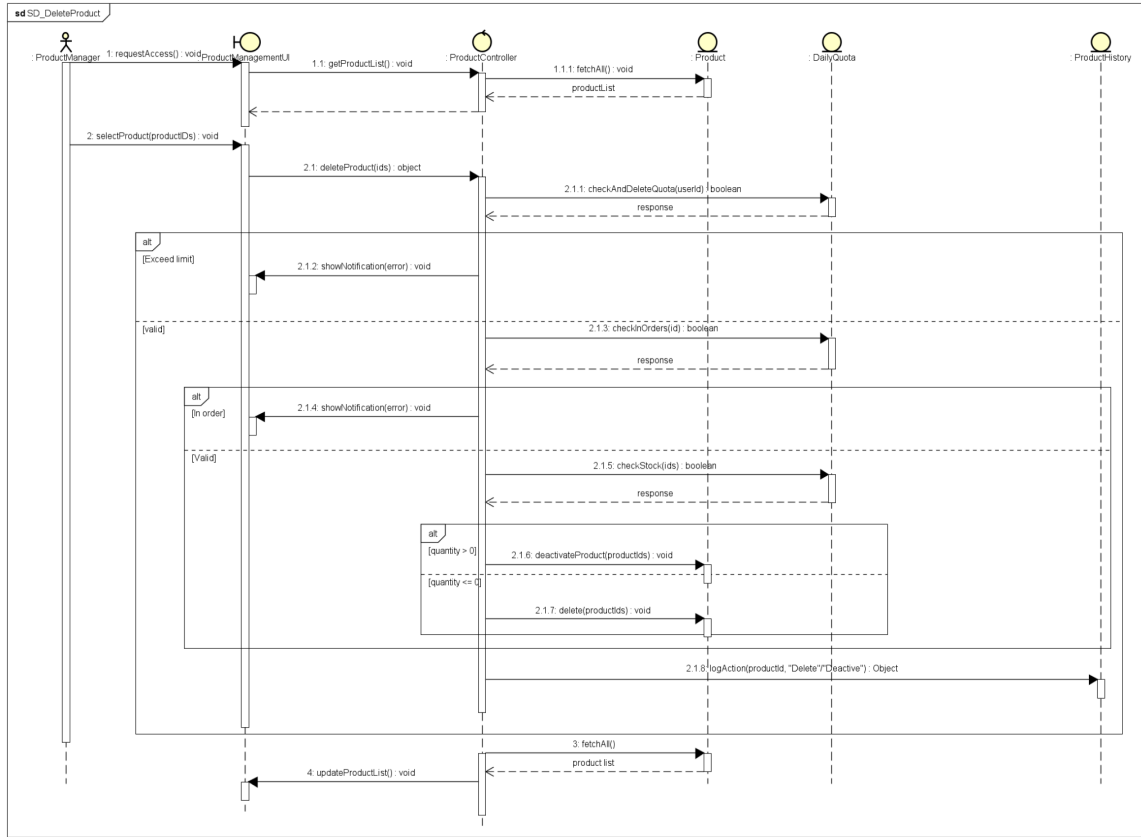
Pre-condition: The Product Manager is currently looking at the product list (ProductListUI) and selects the delete action (they can delete just one item or multiple items—up to 10 products at a time).

Main Flow:

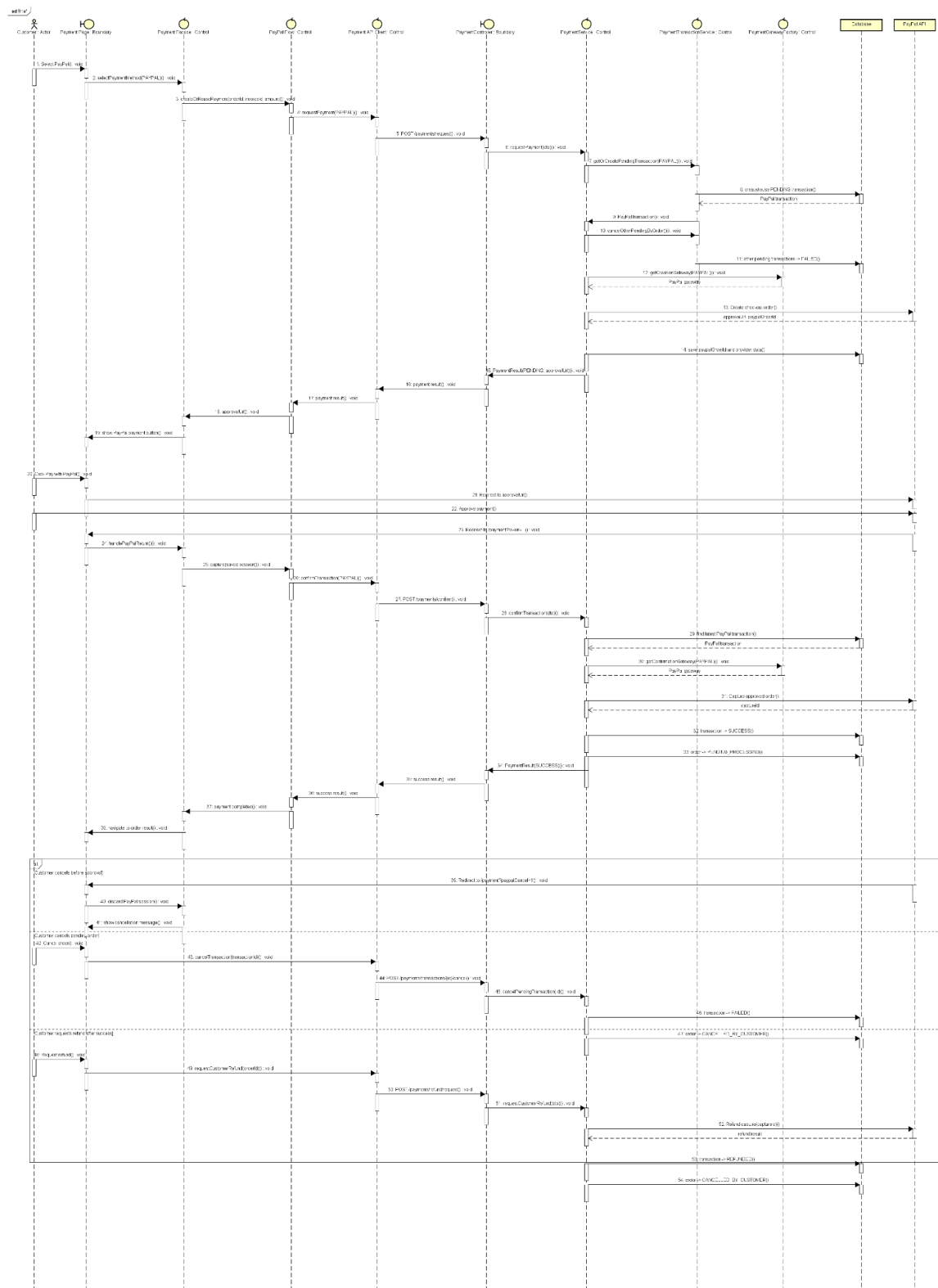
- Receiving the request: The interface (Boundary - ProductListUI) sends a delete command to the main controller (ProductController).
- Checking the daily limit (Quota): The controller checks with the daily limit tracker (Entity - DailyQuota) to make sure the Manager hasn't exceeded their allowed number of deletions for the day. If they are over the limit, the request is instantly rejected.
- Checking data connections: The controller checks if the product is completely out of stock (Stock = 0) and looks to see if it is tied to any past customer orders (Entity - Product).
- Executing the delete: Depending on what the system finds, the product will either be completely erased from the system (Hard Delete) or its status will simply change to "Deactivated" (Soft Delete) if it is still connected to past data.
- Audit logging: The controller tells the history log (Entity - ProductLog) to record a "DELETE" event.
- Updating the quota: The Manager's daily deletion count goes up in the DailyQuota tracker.
- Feedback: The system sends the result back to the interface so the product list on the screen can update.

Business Rules:

- Security Rule (Daily Quota): A Manager cannot delete more than 20 products per day to prevent any intentional damage or accidental mass deletions. Can only select a maximum of 10 products to delete at the exact same time.
- Data Integrity Rule (Soft vs. Hard Delete): A product can only be permanently erased from the database (Hard Delete) if there are exactly 0 items in stock AND it has never been part of an order. Otherwise, its status will just change to "Deactivated" (Soft Delete). This means it won't show up for sale anymore, but its information is safely kept for accounting and history checks.



3.2.2 Pay order by PayPal



Description

This use case describes the PayPal payment flow for an online order. The customer selects PayPal as the payment method, is redirected to PayPal to approve the payment, and then returns to the system. The system confirms and captures the PayPal payment, updates the payment transaction status, and moves the order to processing. The flow also covers customer cancellation and refund requests.

Preconditions

- The customer has created an order and reached the payment step.
- The order contains a valid invoice and payable amount.
- PayPal is available as a supported payment method.
- The system is configured with valid PayPal API credentials.
- The order has no completed payment transaction yet.
- The customer has access to a valid PayPal account or PayPal-supported payment source.

Main Flow

1. The customer selects PayPal as the payment method.
2. The payment page sends the selected payment method to the payment facade.
3. The system creates or reuses a pending PayPal payment transaction for the order and invoice.
4. The system sends a payment creation request to the PayPal API.
5. The payment service checks whether there is an existing pending PayPal transaction.
6. If a reusable pending transaction exists, the system reuses it; otherwise, it creates a new pending transaction.
7. The system cancels other pending payment transactions for the same order.
8. The system creates a PayPal gateway instance.
9. The PayPal gateway creates a checkout order through the PayPal API.
10. PayPal returns an approval URL and PayPal order ID.
11. The system saves the PayPal order ID and provider data.
12. The payment result is returned to the payment page with status PENDING and the approval URL.
13. The payment page displays the PayPal payment button.
14. The customer clicks the PayPal payment button.
15. The customer is redirected to PayPal.
16. The customer approves the payment on PayPal.

17. PayPal redirects the customer back to the system with a payment token.
18. The system handles the PayPal return and captures the saved session information.
19. The system sends a confirmation request for the PayPal transaction.
20. The payment service finds the latest PayPal transaction for the order.
21. The system confirms the payment with the PayPal gateway.
22. The PayPal gateway captures the approved PayPal order through the PayPal API.
23. PayPal returns a successful capture result.
24. The system updates the transaction status to SUCCESS.
25. The system updates the order status to PENDING_PROCESSING.
26. The system returns a successful payment result.
27. The customer is navigated to the order result page.

Business Rules

1. Only one active pending payment transaction should exist for an order at a time.
2. When PayPal is selected, other pending transactions for the same order must be marked as failed or cancelled.
3. A pending PayPal transaction may be reused if it is still valid and belongs to the same order.
4. The payment amount must match the invoice amount of the order.
5. The system must store the PayPal order ID and provider response data for later confirmation.
6. The order must not be moved to processing until the PayPal payment is successfully captured.
7. A PayPal payment is considered successful only after PayPal returns a successful capture result.
8. If the customer cancels before approving the payment, the system discards the PayPal session and shows a cancellation message.
9. If the customer cancels while the transaction is still pending, the pending transaction must be cancelled and the order status becomes CANCELLED_BY_CUSTOMER.
10. If the customer requests a refund after a successful payment, the system must request a refund from PayPal before marking the transaction as REFUNDED.
11. After a successful refund, the order status becomes CANCELLED_BY_CUSTOMER.
12. Failed, cancelled, or refunded transactions must not be reused for later payment attempts.

[illegible]

Description

This use case describes the VietQR payment flow for an online order. The customer selects VietQR as the payment method, the system creates or reuses a pending payment transaction, requests a QR code from the VietQR provider, and displays the QR code to the customer. The customer transfers money using the QR content. VietQR then sends a transaction callback to the system. The system verifies the callback, updates the payment transaction status, and moves the order to processing. The flow also covers customer cancellation while the payment is still pending.

Preconditions

- The customer has created an order and reached the payment step.
- The order contains a valid invoice and payable amount.
- VietQR is available as a supported payment method.
- The system is configured with valid VietQR API credentials.
- The system is configured with VietQR bank account and QR generation settings.
- The order has no completed payment transaction yet.
- The customer has access to a banking application that supports QR transfer.

Main Flow

1. The customer selects VietQR as the payment method.
2. The payment page sends the selected payment method to the payment facade.
3. The system starts a fresh VietQR payment flow for the current order.
4. The payment page sends a payment creation request to the backend.
5. The payment service validates the order, invoice, and payable amount.
6. The payment service checks whether there is an existing pending VietQR transaction.
7. If a reusable pending transaction exists, the system reuses it; otherwise, it creates a new pending transaction.
8. The system cancels other pending payment transactions for the same order.
9. The system creates or selects the VietQR payment gateway.
10. The VietQR gateway builds the QR payment request.
11. The system requests an access token from the VietQR provider.
12. VietQR returns an access token.
13. The system sends a QR generation request to the VietQR provider.
14. VietQR returns QR data, including QR code, QR content, and QR link if available.
15. The system saves the QR provider data into the payment transaction.

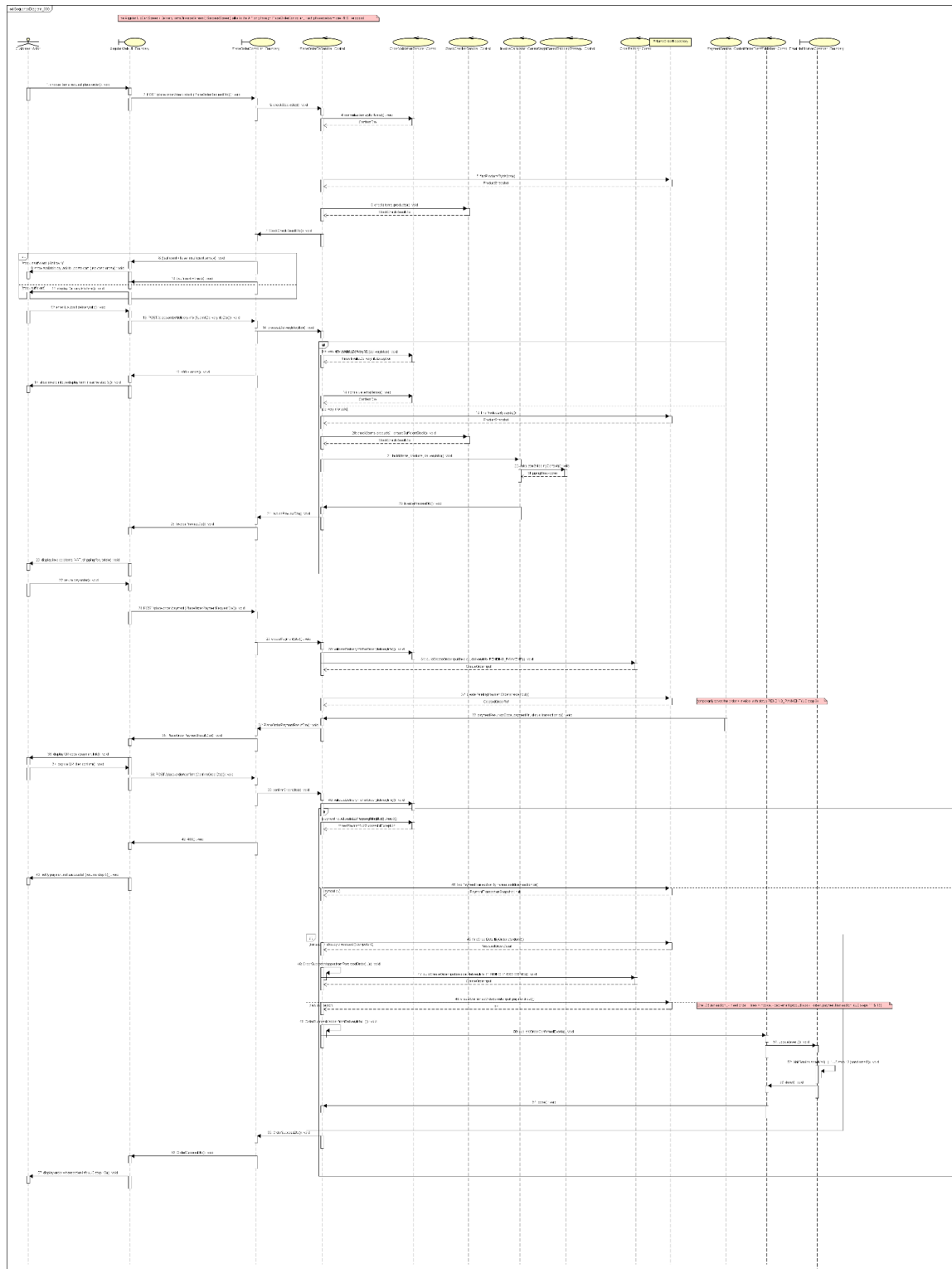
16. The payment result is returned to the payment page with status PENDING and the QR code.
17. The payment page displays the VietQR QR code to the customer.
18. The customer transfers money using the QR content.
19. VietQR sends a transaction callback to the system.
20. The system validates the callback authentication.
21. The callback service finds the matching VietQR payment transaction by QR content.
22. The system checks for duplicate callbacks.
23. The system validates the transaction amount and transaction type.
24. The system marks the payment transaction as SUCCESS.
25. The system cancels other pending transactions for the same order.
26. The system updates the order status to PENDING_PROCESSING.
27. The system returns an HTTP 200 callback acknowledgement to VietQR.
28. The frontend polls the latest payment transaction status.
29. The system returns the successful VietQR transaction status.
30. The payment page marks the payment as completed.
31. The customer is navigated to the order result page.

Business Rules

1. Only one active pending payment transaction should exist for an order at a time.
2. When VietQR is selected, other pending transactions for the same order must be marked as failed or cancelled.
3. A pending VietQR transaction may be reused if it is still valid and belongs to the same order.
4. The payment amount must match the invoice amount of the order.
5. The system must store the generated QR code, QR content, QR link, and provider data for later matching.
6. A VietQR payment is not considered successful immediately after QR generation.
7. A VietQR payment is considered successful only after the system receives and validates a VietQR transaction callback.
8. The callback transaction amount must match the payment transaction amount.
9. Duplicate callbacks must not create duplicate successful transactions.
10. The order must not be moved to processing until the VietQR callback is successfully processed.
11. If the customer cancels while the VietQR transaction is still pending, the pending transaction must be marked as failed and the order status becomes CANCELLED_BY_CUSTOMER.

12. If the VietQR transaction has already succeeded, cancellation from the payment page must not be allowed.
13. Expired or failed VietQR transactions must not be reused for later payment attempts.
14. The system must return a callback acknowledgement to VietQR after processing the callback.

3.2.4 Place order



Description

The Place Order use case describes how a Customer purchases the media products currently in the cart without needing to log in. Its main purpose is to help the Customer check whether the selected items are available, provide delivery information, review the invoice, choose a payment method, and complete the payment.

During the process, the Customer can move through each step, go back to edit previous information, or cancel the order at any time. After the payment is successful, the system records the order and payment transaction, updates the order status, sends the invoice to the Customer, and displays the final order summary.

Preconditions

- The Customer has at least one item in the cart.
- Each cart item references an existing product and a positive integer quantity.
- The AIMS software is running and reachable (the payment gateway — VietQR/PayPal Sandbox — is configured for the payment step).

Main Flow

1. The Customer requests to place an order for the items in the cart.
2. The system normalizes the cart items (merges duplicates by product) and checks product availability for every item.
3. The system displays the delivery-information form together with the current order/cart information.
4. The Customer enters and submits the delivery information (receiver name, phone, province, address, optional shipping instructions, email).
5. The system validates the delivery information.
6. The system calculates (or recalculates) the shipping fee and displays the invoice preview: line items, subtotal before VAT, subtotal incl. VAT, delivery fee, and total amount.
7. The Customer asks to pay the order.
8. The system temporarily saves the order and invoice with status PENDING_PAYMENT and displays the invoice information.
9. The system calls the Pay Order use case (default payment method = QR code) and displays the QR code / payment link.
10. After the Customer completes the payment, the system validates the payment result (transaction id + method) and ensures the transaction has not already been processed.
11. The system records the successfully paid order and the payment transaction, decrements product stock, and moves the order to PENDING_PROCESSING — all within a single atomic transaction.

12. The system sends the invoice and payment-transaction information to the Customer's e-mail.
13. The system displays the general order information (customer name, phone, province, address, total amount) and transaction information (transaction id, content, datetime).

Postcondition: A new order is created in the PENDING_PROCESSING state and its information is e-mailed to the Customer; if the payment is not successful, nothing is persisted.

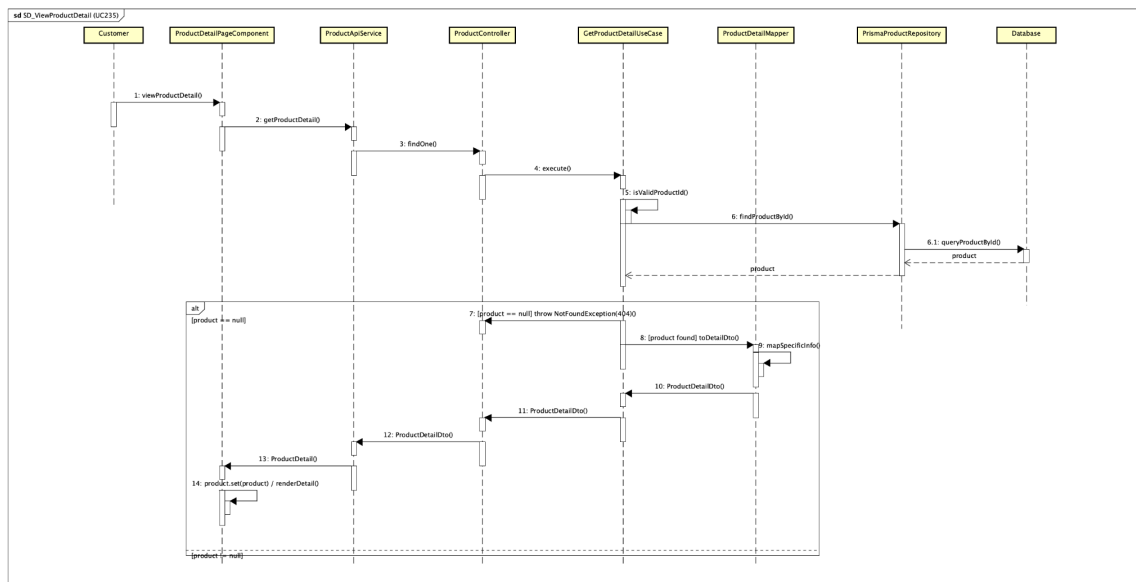
Business Rules

1. All product prices exclude a 10% VAT. $\text{subtotalAfterVat} = \text{subtotalBeforeVat} \times 1.10$.
2. An order may proceed only if, for every item, available stock $\geq$ requested quantity. A product whose status is DEACTIVATED/DELETED/INACTIVE/UNAVAILABLE counts as 0 available. If any item is insufficient, the system notifies the Customer with the available quantity of each affected product.
3. Delivery information validation: Receiver name required, letters only; Phone required, exactly 10 digits, may contain a single separator type (., space, or -) inside the number; Province required (from the provided list); Address required, non-empty; Shipping instructions optional, max 200 characters; Email required to create the order.
4. Shipping fee depends on total item weight and delivery location and is not taxed: Ha Noi / Ho Chi Minh City is 22,000 VND for the first 3 kg; elsewhere in Vietnam is 30,000 VND for the first 0.5 kg; plus 2,500 VND for every additional 0.5 kg block (rounded up).
5. If the items subtotal (before VAT) $> 100,000$ VND, the order qualifies for a free-shipping discount of up to 25,000 VND per order ($\text{finalShippingFee} = \max(0, \text{rawFee} - \text{discount})$). Total amount = subtotal include VAT + final delivery fee.
6. The default payment method is QR code (VietQR); the Customer may switch to PayPal. Other gateways may be added in the future.
7. Cart items are normalized before processing: productId and quantity must be positive integers; items with the same productId are merged by summing quantities.
8. Order lifecycle: an order is first persisted as PENDING_PAYMENT when payment is requested, then transitions to PENDING_PROCESSING after a successful payment.
9. After a successful payment the system displays the order + transaction summary and records both the paid order and the payment transaction. If payment fails, the

use case is notified that payment was not successful and resumes at the delivery-validation step.

10. Payment confirmation requires a non-empty transaction id and payment method; otherwise it is rejected as "payment not successful". The confirmation is idempotent - a transaction that was already processed returns the existing order instead of creating a duplicate.
11. Stock decrement on confirmation is atomic and oversell-safe: stock is only reduced when the order actually transitions to PENDING_PROCESSING, so duplicate/concurrent payment callbacks never decrement stock twice.
12. On success, the system e-mails the invoice and payment-transaction information to the Customer (notification is delivered through pluggable channels - e-mail today; SMS/Zalo/Push may be added without changing the order flow).

3.2.5 View Product Detail



Description

The View Product Detail use case (UC235) lets a Customer open a product's detail page from the catalog to view its full information: the common fields (barcode, title, current price, original value, description, image, weight, dimensions, stock quantity and availability status) and the type-specific information for a Book, Newspaper, CD or DVD. From this screen the Customer may also choose a quantity and add the product to the shopping cart. Viewing a product detail is public and does not require login. The sequence diagram above shows the realization of this flow across the frontend (Angular) and backend (NestJS) classes.

Preconditions

1. The system is running and the product catalog (home / product list) has been loaded.
2. The Customer has selected a product from the catalog, so a product id is available in the route (/products/:id).
3. Viewing a product detail is publicly accessible — no authentication is required.

Main flow

1. The Customer selects a product from the catalog.
2. ProductDetailPageComponent reads the route parameter :id (ngOnInit) and calls ProductApiService.getProductDetail(id).
3. ProductApiService sends HTTP GET /products/:id; ProductController.findOne(id) delegates to GetProductDetailUseCase.execute(id).
4. GetProductDetailUseCase validates the id, then loads the product via ProductQueryRepository.findProductById(id) (implemented by PrismaProductRepository).
5. ProductDetailMapper.toDetailDto(product) maps the common fields and dispatches mapSpecificInfo() by product type (Strategy) to build the type-specific section, producing a ProductDetailDto.
6. The ProductDetailDto is returned as a JSON response; ProductApiService.normalizeProductDetail() converts it into the ProductDetail model.
7. ProductDetailPageComponent sets the product signal; ProductDetailComponent renders the common fields and ProductSpecificInfoComponent renders the type-specific section.
8. The Customer may adjust the quantity and add the product to the cart when it is available and in stock.

Alternative flow

1. Invalid product id (not matching `^[1-9]\d*$`) → GetProductDetailUseCase throws BadRequestException (HTTP 400).
2. Product not found → NotFoundException (HTTP 404); ProductDetailPageComponent shows the 'product not found' state.

3. Backend unreachable or other error → the page shows an error message ('Cannot load product detail from backend.').
4. Product status is OUT_OF_STOCK / DEACTIVATED / UNAVAILABLE → add-to-cart is disabled with a notice.

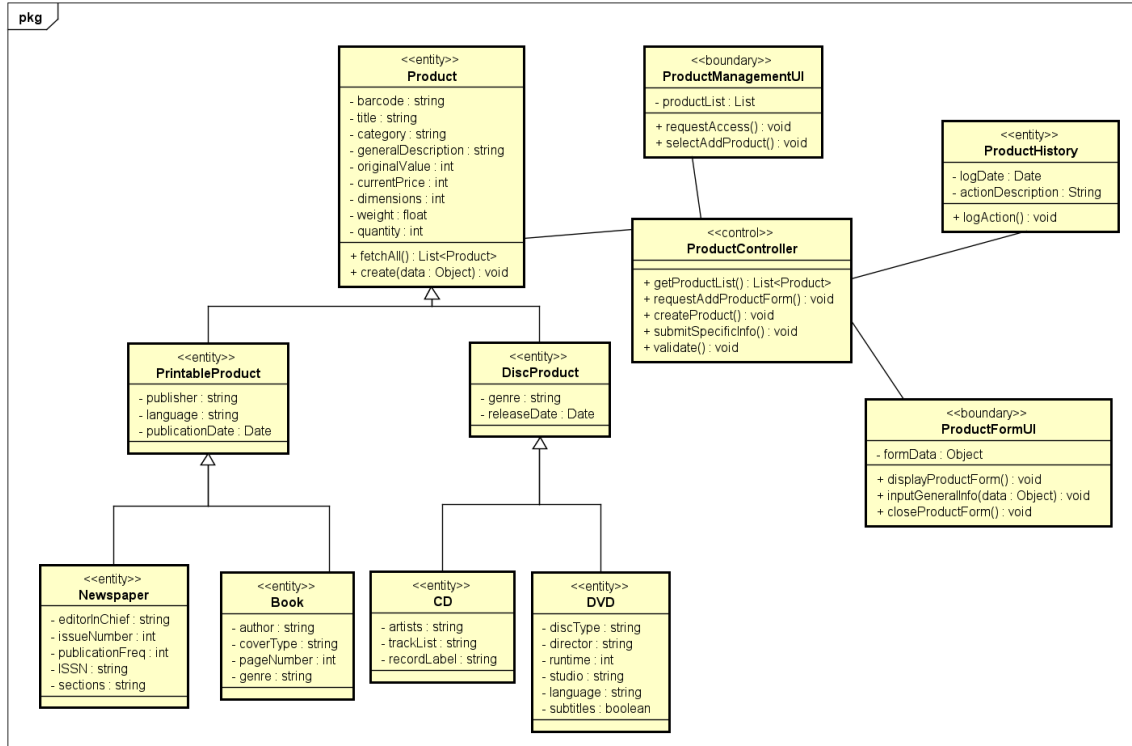
Business rules

1. Viewing a product detail is public; login is not required.
2. A product id must be a positive integer (regex `^[1-9]\d*$`); otherwise the request is rejected.
3. The information displayed depends on the product type — Book, Newspaper, CD and DVD each show their own specific fields.
4. A product is purchasable only when its status is available and `stockQuantity > 0`; status OUT_OF_STOCK, DEACTIVATED or UNAVAILABLE makes it nonpurchasable.
5. The add-to-cart quantity must be between 1 and the available stock quantity.
6. The availability status is derived from stock: when quantity = 0 the product is shown as OUT_OF_STOCK.

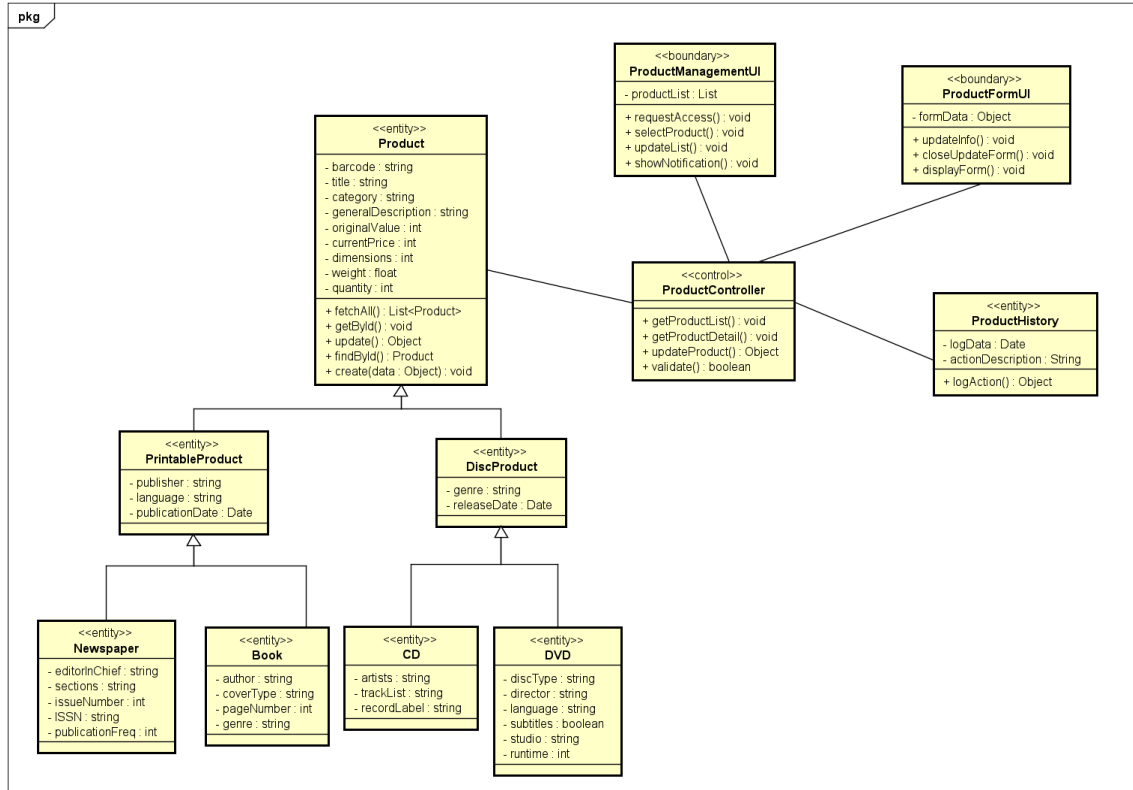
3.3 Analysis Class Diagrams

3.3.1 CUD Product

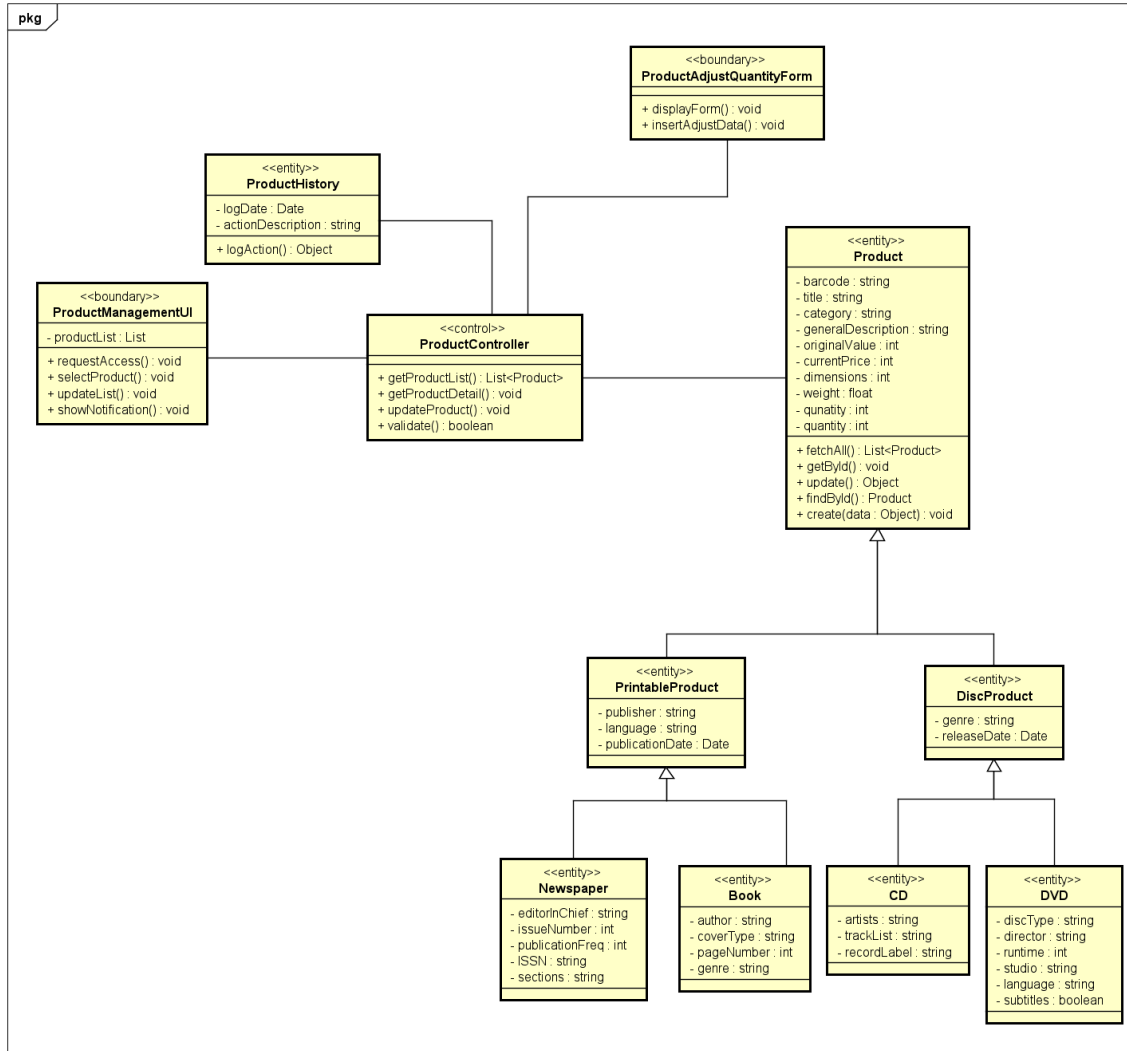
3.3.1.1. Add Product



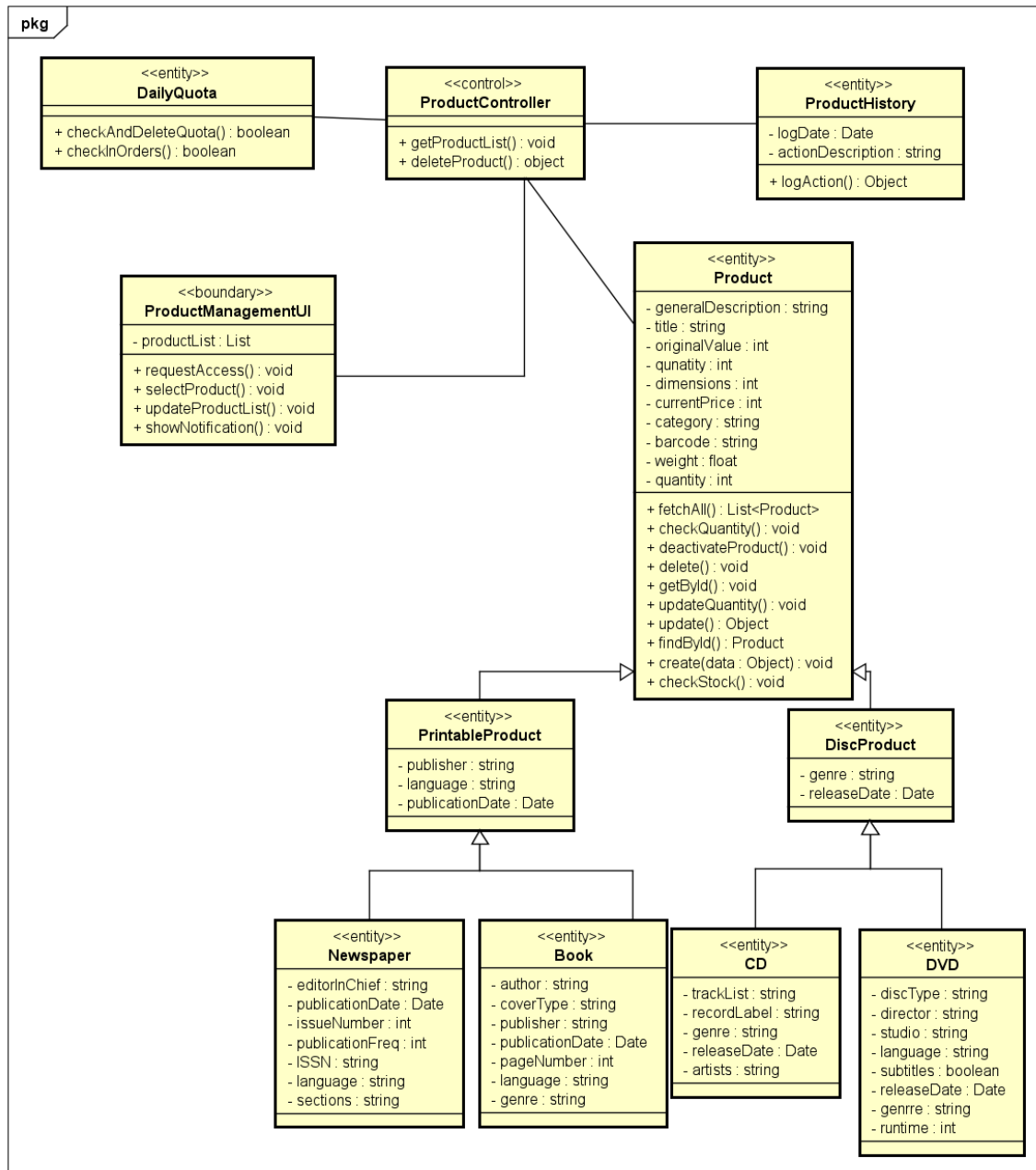
3.3.1.2. Update Product



3.3.1.2. Adjust Quantity

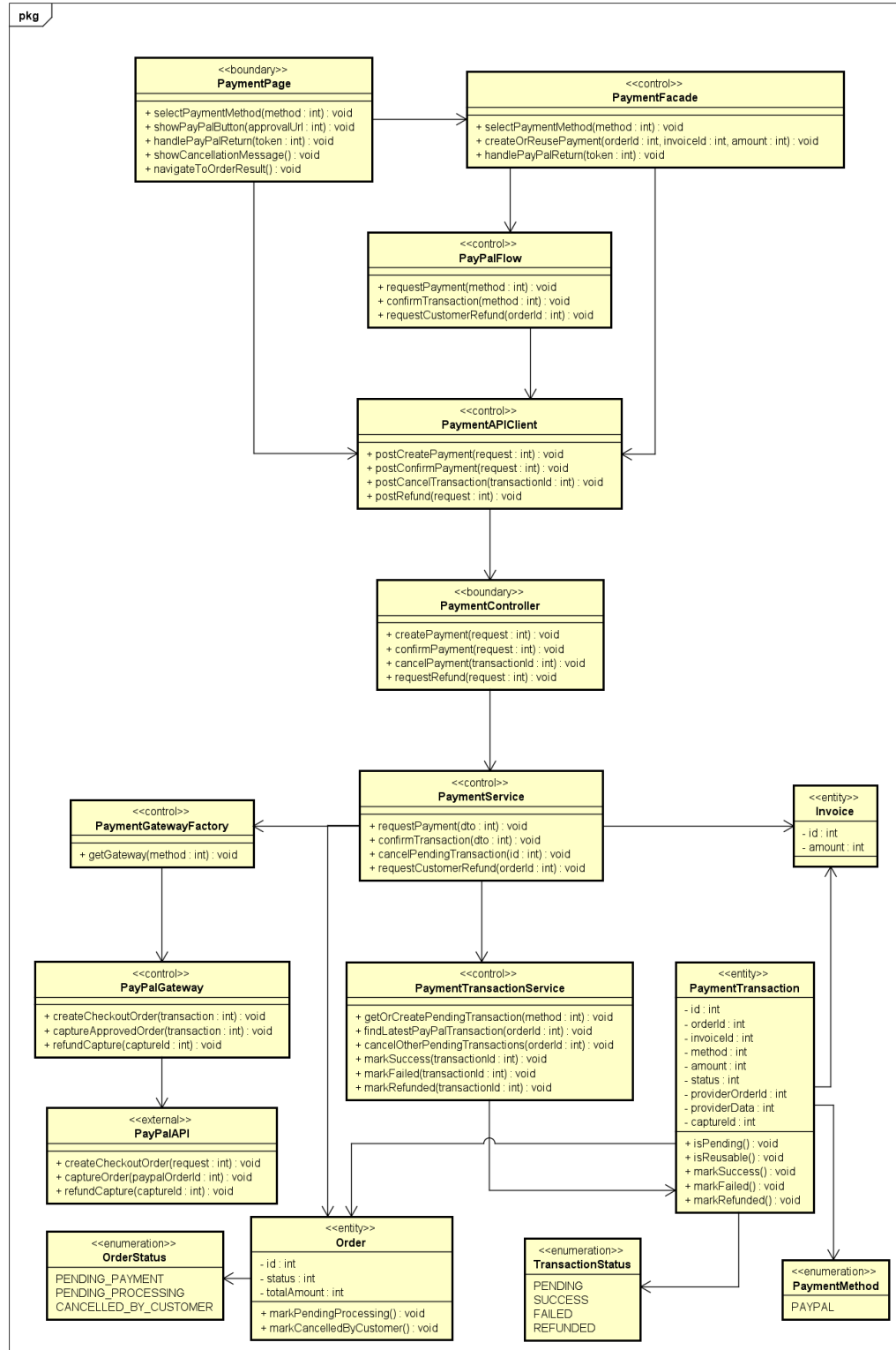


3.3.1.2. Delete Product

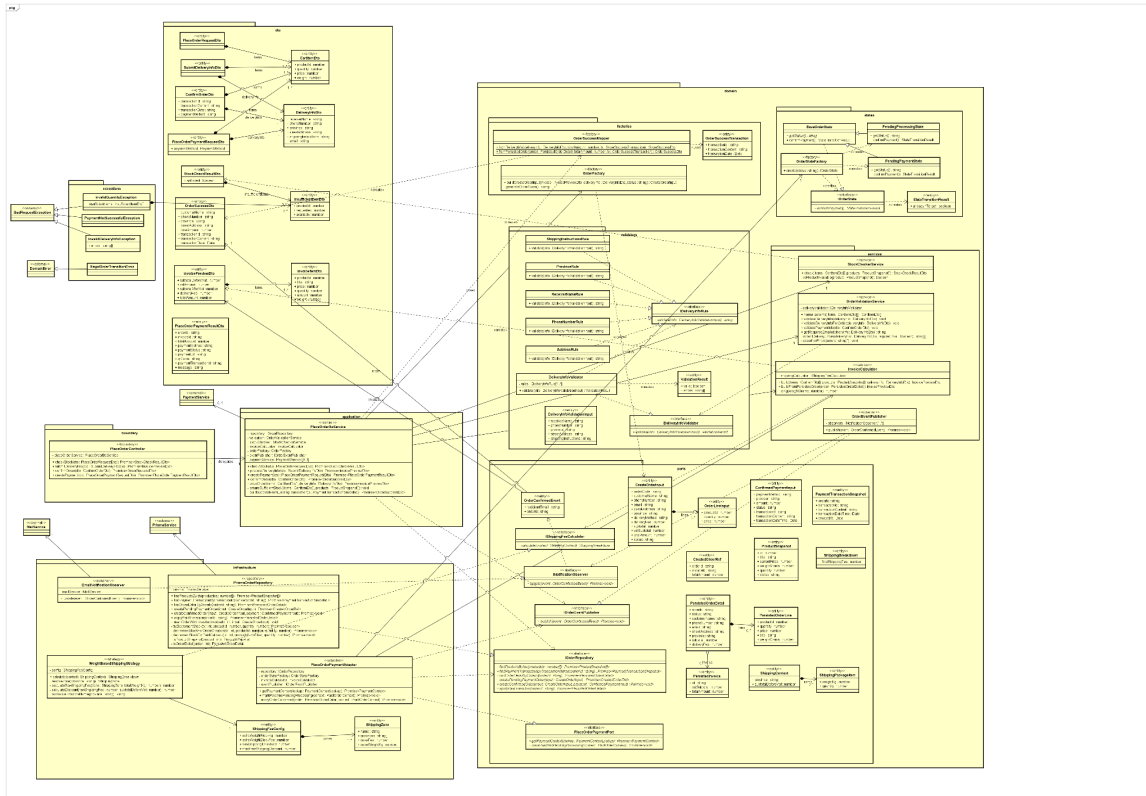


3.3.2 Pay order by VietQR

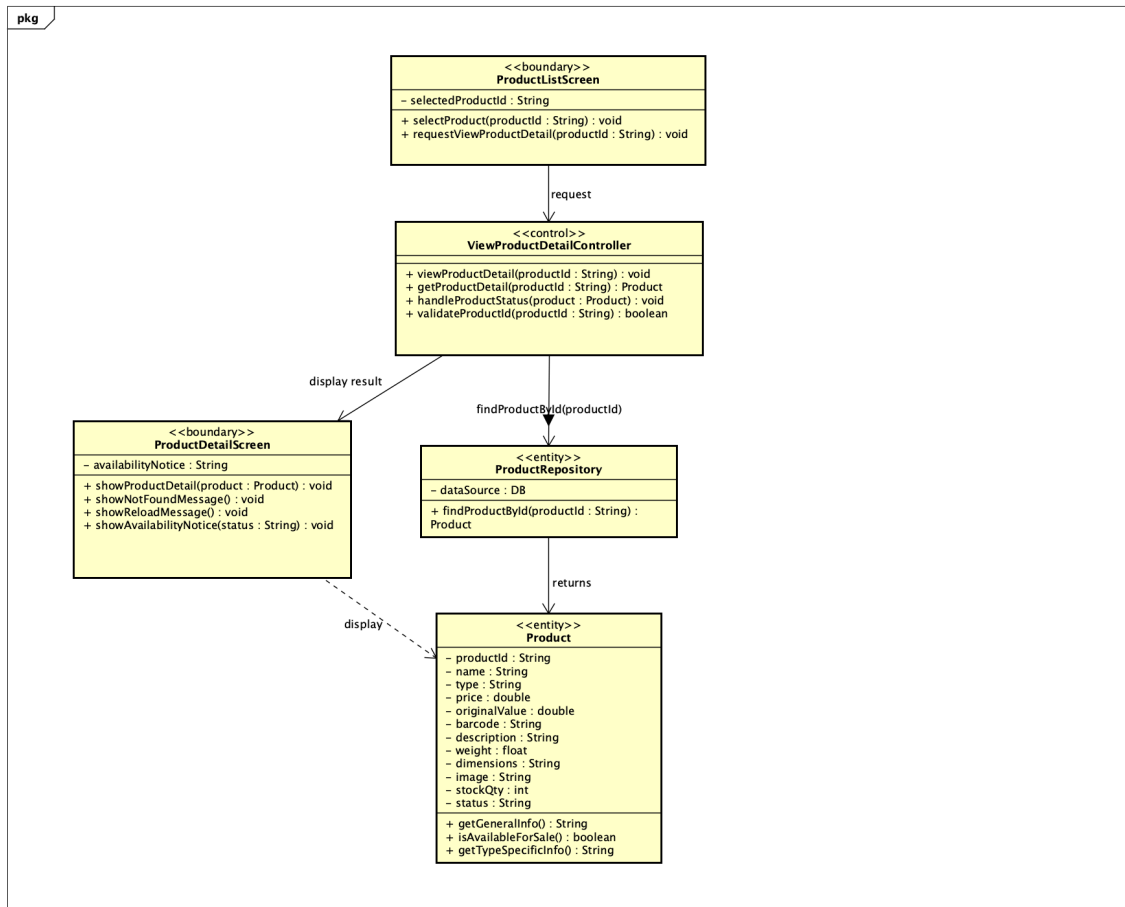
3.3.2 Pay order by PayPal



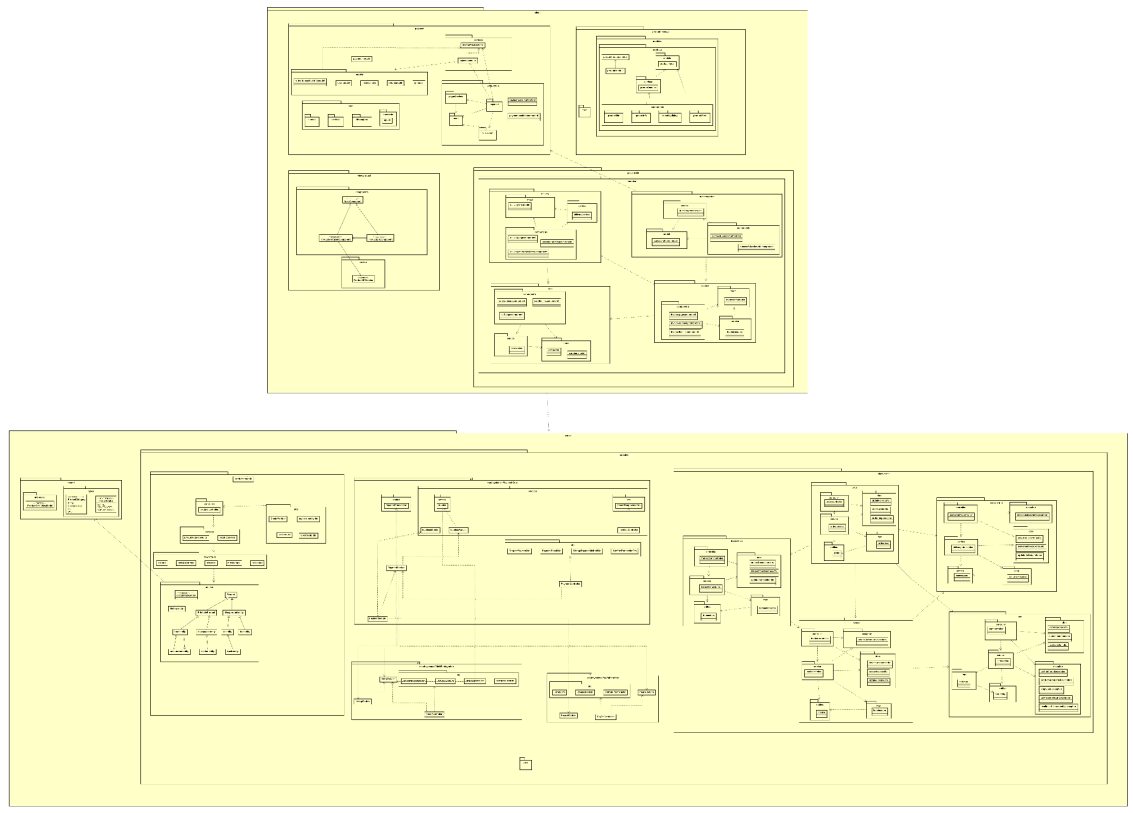
3.3.3 Place order



3.3.4 View Product Detail



3.4 Unified Analysis Class Diagram



3.5 Security Software Architecture

The system uses a layered security architecture to protect user accounts, business data, transactions, and administrative operations. Security controls are applied across the client application, backend services, database, and external integrations.

3.5.1 Authentication

The system requires users to authenticate before accessing protected features. Users log in with valid account credentials, and the authentication component verifies the submitted information against stored account records. Passwords are never stored in plain text; they are securely hashed before being saved.

After successful authentication, the system issues a secure session or authentication token. This token is used to identify the user in later requests and must be protected from unauthorized access. Unauthenticated users are only allowed to access public features, such as browsing publicly available product information.

3.5.2. Authorization

The system applies authorization rules after authentication to ensure that users can only perform actions permitted by their role and ownership of resources. Customers can access and manage only their own profile, cart, orders, payments, and related information. Staff or administrative users can access management functions according to their assigned roles.

Authorization checks are enforced on the backend for all protected operations, including account management, product management, order processing, payment handling, delivery updates, cancellations, refunds, and administrative configuration. The system also validates the state of business objects before allowing an action. For example, an order can only be cancelled, paid, processed, shipped, or refunded when it is in a valid status for that operation.

3.5.3. Encryption

The system protects sensitive data during transmission and storage. All communication between the client, backend services, database, and external services uses secure communication protocols such as HTTPS/TLS to prevent interception and tampering.

Sensitive information such as passwords, authentication tokens, personal user data, payment references, API keys, and database credentials must be protected. Passwords are stored using secure hashing. System secrets and external service credentials are stored in server-side configuration or a secure secrets manager, not in client-side code.

The system does not store unnecessary sensitive financial information. Payment and external service credentials are handled by trusted providers where applicable. Any sensitive data stored in the database should be minimized and protected according to its risk level.

3.5.4. Logging and Auditing

The system records important security and business events for monitoring, troubleshooting, and auditing. Logged events may include login attempts, failed authentication attempts, account changes, product updates, order creation, order status changes, payment events, cancellation requests, refund requests, delivery updates, and administrative actions.

Audit logs should include useful trace information such as user ID, action type, affected resource ID, timestamp, result status, and relevant business reference IDs. Logs must not expose sensitive information such as passwords, authentication tokens, API secrets, full payment credentials, or confidential personal data. Sensitive values should be masked or excluded.

The logging and auditing design helps detect abnormal behavior, investigate incidents, trace business operations, support dispute resolution, and maintain accountability for user and administrative actions.

4. Detailed Design

4.1 User Interface Design

4.1.1 Screen Configuration Standardization

4.1.1.1 Visual Foundation

4.1.1.1.1 Color Palette

The AIMS platform utilizes a clean, professional color system prioritizing clarity and structure.

Role	Color Hex	Usage
Primary Action	#0078D7 (Blue)	Primary buttons, active states, focus highlights.
High Energy	#FF5722 (Orange)	Critical CTAs (e.g., Add to Cart, Buy Now), alerts.
Background	#F5F5F5 (Light Gray)	Neutral foundation for all screens.
Surface	#FFFFFF (White)	Cards, modals, containers, and content areas.
Text Primary	#222222 (Dark Gray)	Main body text and headings.
Text Secondary	#666666 (Medium Gray)	Secondary text, labels, metadata

4.1.1.1.2 Typography System

Font Families: Manrope / Inter

- Headline 1: 24px Bold (Main page titles)
- Headline 2: 20px Semi-bold (Section headers)
- Headline 3: 18px Bold (Card titles, sub-sections)
- Headline 4: 16px Bold (Small headers, button text)
- Paragraph: 14px Regular (Body text, input labels)
- Caption: 12px Regular (Helper text, metadata)

4.1.1.1.3 Spatial System & Geometry

- Base Grid: 8px system. All margins, padding, and layout alignments must follow this scale:
 - + XS: 4px | S: 8px | M: 16px | L: 24px | XL: 32px | XXL: 64px
- Corner Radius: 4px standard across buttons, input fields, and cards.
- Border Weights: * Standard containers/inputs: 1px
 - + Active/Focus states: 2px

4.1.1.2. Component Specifications

4.1.1.2.1 Buttons

- Primary: Filled #0078D7, 4px radius, centered text.
- Secondary: Outlined #0078D7, 1px border, transparent background.
- High Energy: Filled #FF5722 (Strictly for conversion actions).
- Interaction States:
 - + Hover: Darken base color by 10%.
 - + Active (Click): Scale down to 98%.
 - + Disabled: 50% opacity, converted to grayscale.

4.1.1.2.2 Form Inputs

- Default State: 1px border #E0E0E0, 4px radius, 14px text.
- Validation States:
 - + Error Styling: 1px solid #F44336 (Red) border.
 - + Error Message: Displayed immediately below the field in #F44336 using 12px Caption typography.

4.1.1.3. Layout & Structure

4.1.1.3.1 Role-Based Architectural Shells

- Customer Interface: * Top Navigation (Logo, Search Bar, Cart/User Profile) -> Main -> Content -> Footer.
- Admin/PM Interface: * Left Sidebar (Navigation) + Top Header (Page Title, User Profile) + Main Content Area.
- Alignment Rule: Page titles must always be centered at the top of the main content area.

4.1.1.3.2 Modals & Secondary Screens

- Placement: Help screens, manuals, or detailed views must appear as centered popups.
- Overlay: A dark overlay with 80% opacity must cover the main screen, disabling all background interactions.

- Close Action: A prominent "X" close button must be standardized in the top-right corner of the modal.

4.1.1.4. Interaction & Flow Standards

4.1.1.4.1 Application Flow & Navigation

- Boot Sequence: Opening Screen → Splash Screen → Homepage.
- Back Navigation: Every internal or nested screen must feature a clear "Back" button returning the user to the exact previous state.

4.1.1.4.2 Keyboard & Focus Accessibility

- Tab Order: Forms and interactive elements must support sequential keyboard navigation (Tab / Shift+Tab) flowing naturally top-to-bottom, left-to-right.
- Standard OS Shortcuts: Do not implement custom keyboard shortcuts; rely on standard browser/OS behaviors.
- Escape Key: Must trigger the 'Close' action for any active modal or secondary screen.

4.1.1.5. Data & Logic Guidelines

4.1.1.5.1 Input Validation Protocol

All form submissions must pass a strict two-step validation sequence before processing:

- Presence Check: Verify the required field is not empty.
- Format/Logic Check: Verify the data conforms to the expected type, pattern, or range (e.g., valid email regex, positive numeric values for price).

4.1.1.5.2 Data Formatting

- Currency: Formatted as [Amount] VND (e.g., 100,000 VND).
- Numbers: Use comma separators for thousands (e.g., 1,250,000).
- Dates: Strict DD/MM/YYYY format.

4.1.1.6. Layout & Structure (The Screen Shells)

Our system doesn't use a one-size-fits-all interface. Depending on who the user is, the structural shell is fine-tuned to optimize the workspace and user flow.

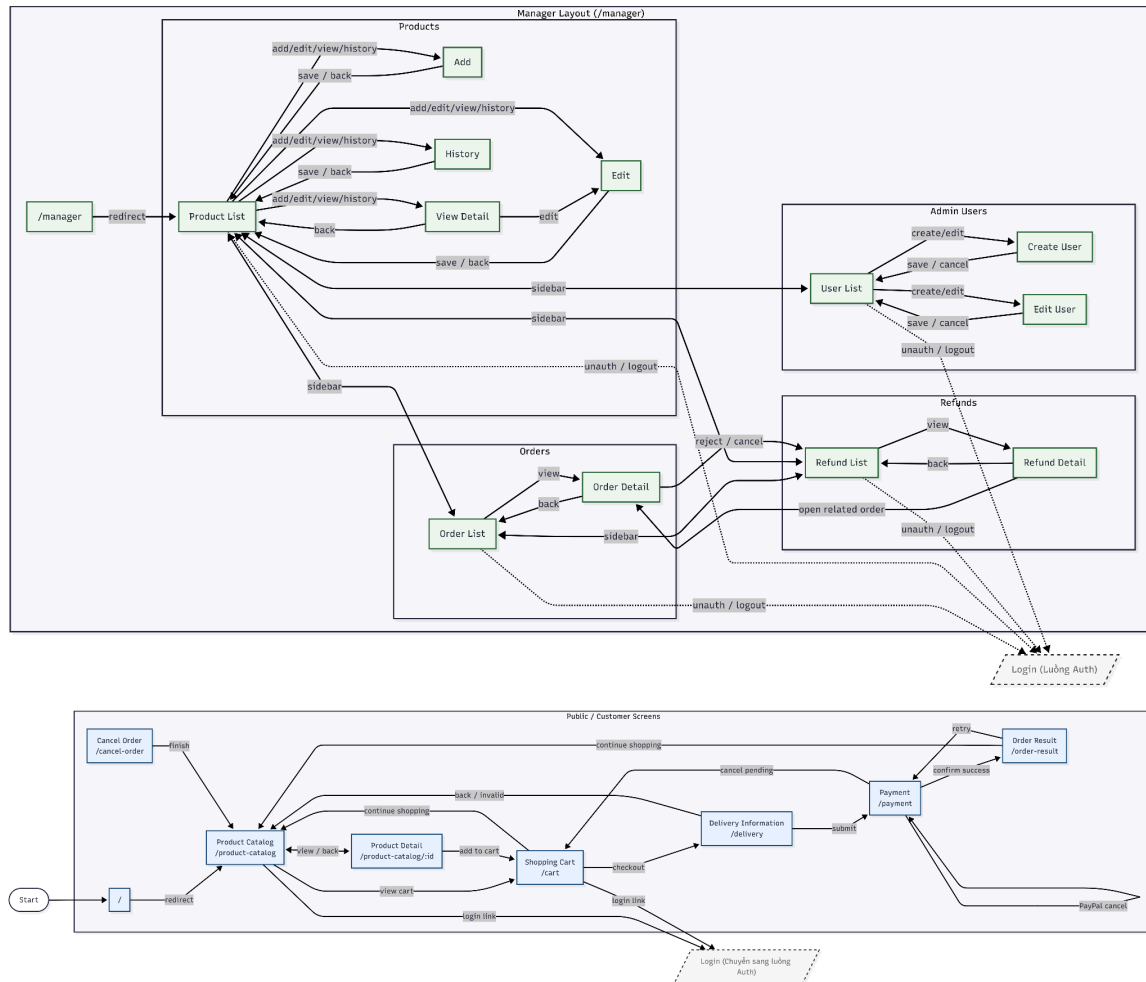
4.1.1.6.1. Admin & PM Layout (The Workspace)

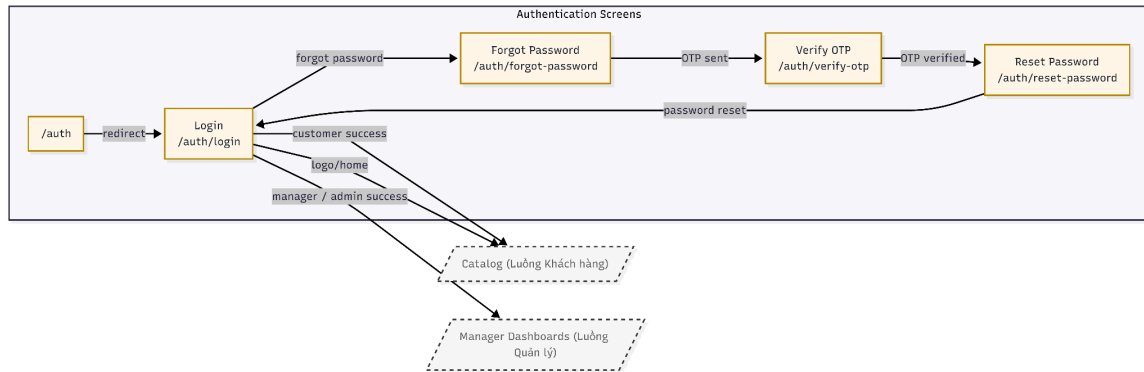
This environment is built for deep focus on data and management tools.

- Left Sidebar: Contains the main functional menus (Dashboard, Media Store, Settings, etc.). It stays pinned to the left so users can quickly switch between different modules.

- Top Header: Features the global search bar and account details (Profile/Notifications) tucked on the right. The Screen Title is positioned directly below this header, centered perfectly over the content area.
- Main Content Area: A spacious, clean white (#FFFFFF) canvas taking up the center stage. This is where all the heavy lifting happens—tables, input forms, and management lists.

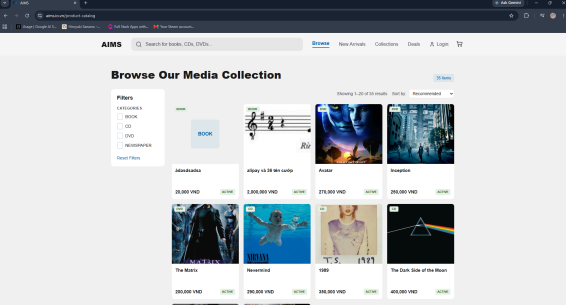
4.1.2 Screen Transition Diagrams





4.1.3 Screen Specifications

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Browse Media Screen	20/4/2026	HiếuBT	TúNHT	NghĩaTT

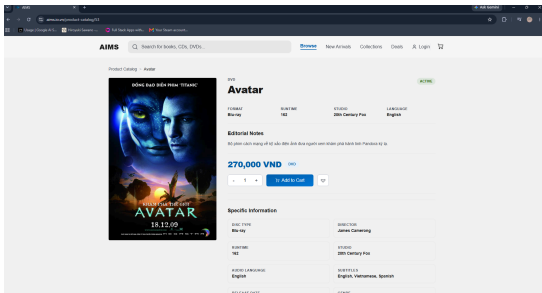
	Control	Operation	Function
	Search Bar	Input / Press Enter	Search for specific books, CDs, DVDs, or Vinyl across the database
	Navigation Links	Click	Navigate to Browse, New Arrivals, Collections, or Deals pages

	Cart Icon	Click	Navigate to the View Cart screen
	Category Checkboxes	Check / Uncheck	Filter the displayed media list by selected categories
	Price Range Slider	Drag / Input	Update the minimum and maximum price range to filter products
	Reset Filters Button	Click	Clear all applied category and price filters to default state
	Sort Dropdown	Select	Sort items by predefined criteria (e.g., Recommended, Price, Title)
	Product Card	Click	Navigate to the detailed information screen for that specific media item
	Pagination Links	Click	Display the requested page of media results

Item	Length	Type	Color	Remarks
Search placeholder	100 characters	Text	Gray	Left-justified
Category labels	50 characters	Text	Black	Left-justified
Price input fields	10 digits	Numeral / Text	Black	Right-justified, allows numeric input and suffix
Results count text	50 characters	Text	Gray	Left-justified
Product Badge	20 characters	Text	Blue	Uppercase, positioned top-left inside the product image area
Product Title	100 characters	Text	Black	Left-justified, Bold
Product Description	200 characters	Text	Gray	Left-justified, truncated with ellipsis if exceeding line limits
Product Price	20 digits	Numeral / Text	Black	Left-justified, Bold

Pagination numbers	5 digits	Numeral	Black / White	Center-justified, active page highlighted with a blue background
---------------------------	----------	---------	---------------	--

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Product Detail Screen	20/04/2026	HiếuBT	KhanhNT	NghĩaTT

	Control	Operation	Function
	Search bar	Input	Search for books, CDs, DVDs and other products
	Browse tab	Click	Navigate to Browse page
	New Arrivals tab	Click	Navigate to New Arrivals page
	Collections tab	Click	Navigate to Collections page
	Deals tab	Click	Navigate to Deals page
	Cart icon	Click	Navigate to shopping cart screen
	Breadcrumb navigation	Click	Navigate to previous category levels
	Product image area	Initial	Display product image
	Artist / Brand area	Initial	Display artist or brand name

	Product title area	Initial	Display product title
	Product subtitle area	Initial	Display product subtitle or edition
	Price area	Initial	Display product selling price
	Format tag	Initial	Display product format
	Decrease quantity button	Click	Decrease selected quantity
	Quantity field	Input	Display or edit selected quantity
	Increase quantity button	Click	Increase selected quantity
	Add to Cart button	Click	Add selected quantity of product to cart
	Wishlist button	Click	Add product to wishlist or favorites
	Format information area	Initial	Display product format information
	Runtime information area	Initial	Display runtime information
	Tracks information area	Initial	Display number of tracks
	Label information area	Initial	Display label information
	Editorial Notes area	Initial	Display editorial notes or product description

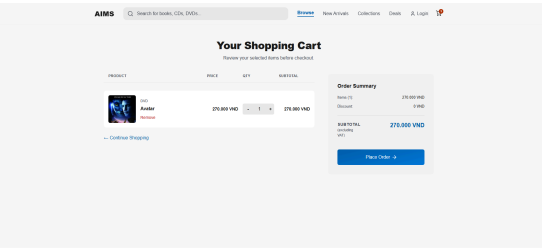
	Privacy Policy link	Click	Navigate to Privacy Policy page
	Terms of Service link	Click	Navigate to Terms of Service page
	Licensing link	Click	Navigate to Licensing page
	Contact link	Click	Navigate to Contact page

Item	Length	Type	Color	Remarks
Search bar	100 characters	Text	Gray	Input field, left-justified
Browse tab	20 characters	Link / Tab	Blue	Clickable
New Arrivals tab	20 characters	Link / Tab	Black	Clickable
Collections tab	20 characters	Link / Tab	Black	Clickable
Deals tab	20 characters	Link / Tab	Black	Clickable
Cart icon	1 icon	Button	Black / Orange badge	Clickable
Breadcrumb navigation	100 characters	Link / Text	Gray / Black	Clickable
Product image	1 image	Image	Original image color	Display only
Artist / Brand	50 characters	Text	Gray	Uppercase, left-justified
Product title	150 characters	Text	Black	Bold, left-justified
Product subtitle	100 characters	Text	Black	Left-justified
Price	20 characters	Currency	Blue	Bold, left-justified
Format tag	20 characters	Label	Gray	Display only
Decrease quantity button	2 characters	Button	Black / White background	Clickable
Quantity field	5 digits	Numerical	Black	Center-justified

Increase quantity button	2 characters	Button	Black / White background	Clickable
Add to Cart button	20 characters	Button	White text / Blue background	Clickable
Wishlist button	1 icon	Button	Black / White background	Clickable
Format information	30 characters	Text	Black	Left-justified
Runtime information	20 characters	Text	Black	Left-justified
Tracks information	10 digits	Numerical	Black	Left-justified
Label information	50 characters	Text	Black	Left-justified
Editorial Notes	1000 characters	Text	Gray / Black	Multi-line, left-justified
Privacy Policy link	30 characters	Link	Gray	Clickable
Terms of Service link	30 characters	Link	Gray	Clickable
Licensing link	20 characters	Link	Gray	Clickable
Contact link	20 characters	Link	Gray	Clickable

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Cart Screen	20/4/2026	<i>TùNHT</i>	<i>HiếuBT</i>	<i>KhanhNT</i>

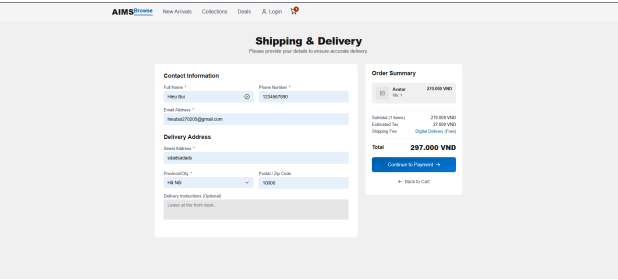
	Control	Operation	Function
--	----------------	------------------	-----------------

	Search Bar	Input / Press Enter	Search for specific media items across the database
	Navigation Links	Click	Navigate to Browse, New Arrivals, Collections, or Deals pages
	Cart Icon	Click	Reload or stay on the current View Cart screen
	Remove Button	Click	Remove the corresponding media item from the shopping cart
	Quantity Input Box	Input	Adjust the desired quantity of the specific item
	Place Order Button	Click	Proceed to the Checkout/Delivery Information screen
	Continue Shopping Link	Click	Navigate back to the previous product browsing screen or homepage

Item	Length	Type	Color	Remarks
Page Title	50 characters	Text	Black	Center-justified, Bold ("Your Shopping Cart")
Table Headers	20 characters	Text	Gray	Left/Center-justified, Uppercase (PRODUCT, PRICE, QTY, SUBTOTAL)
Product Image	N/A	Image	N/A	Left-aligned within the product row
Product Category	50 characters	Text	Gray	Left-justified, small text (e.g., "Vector Graphic")
Product Title	100 characters	Text	Black	Left-justified, Bold
Unit Price	20 digits	Numeral / Text	Gray	Left-justified, formatted with comma separators
Quantity Box	3 digits	Numeral	Black	Center-justified text within a light gray background box

Item Subtotal	20 digits	Numeral / Text	Black	Left-justified, Bold
Order Summary Title	50 characters	Text	Black	Left-justified, Bold ("Order Summary")
Summary Labels	50 characters	Text	Gray	Left-justified (e.g., "Items (3):", "Discount:")
Summary Values	20 digits	Numeral / Text	Black	Right-justified
Total Subtotal Label	50 characters	Text	Gray	Left-justified, includes "(excluding VAT)" subtext
Total Subtotal Value	20 digits	Numeral / Text	Blue	Right-justified, Bold, larger font

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Shipping & Delivery Screen	20/4/2026	HiếuBT	TúNHT	BìnhNT

	Control	Operation	Function
	Search Bar	Input / Press Enter	Search for specific media items across the database
	Navigation Links	Click	Navigate to Browse, New Arrivals, Collections, or Deals pages
	Cart Icon	Click	Navigate to the View Cart screen
	Full Name Input	Input	Enter the recipient's full name for the order

	Phone Number Input	Input	Enter the recipient's contact phone number
	Email Address Input	Input	Enter an email address to receive order confirmation and updates
	Street Address Input	Input	Enter the detailed delivery street address
	Province/ City Dropdown	Select	Choose the specific province, state, or city for delivery
	Postal/Zip Code Input	Input	Enter the postal code corresponding to the delivery address
	Delivery Instructions	Input	Enter optional notes or instructions for the delivery personnel
	Continue to	Click	Validate all required form fields and navigate to the

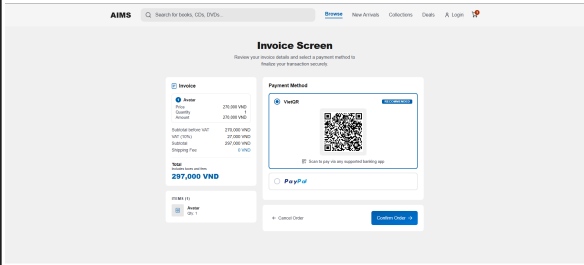
	Payment Button		payment processing screen
	Back to Cart Link	Click	Return to the Shopping Cart screen to review or modify items

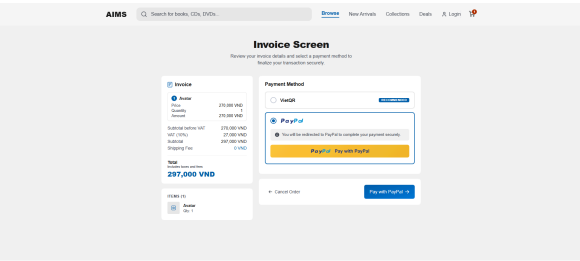
Item	Length	Type	Color	Remarks
Page Title	50 characters	Text	Black	Center-justified, Bold ("Shipping & Delivery")
Section Headings	50 characters	Text	Black	Left-justified, Bold (e.g., "Contact Information", "Delivery Address")
Input Labels	50 characters	Text	Gray	Left-justified, positioned above input fields
Valid Input Field	N/A	UI Element	Green/Gray	Input field displays a green checkmark icon when data format is validated

Invalid Input Field	N/A	UI Element	Red	Input field background turns light red, displays a red error icon when data is invalid
Error Message	100 characters	Text	Red	Left-justified, appears directly below the invalid input field (e.g., "Please enter a valid email address.")
Dropdown Menu	N/A	UI Element	Light Gray	Displays a chevron icon, opens a list of selectable regions
Summary Product Title	100 characters	Text	Black	Left-justified, small font
Summary Subtext	50 characters	Text	Gray	Left-justified, small font (e.g., "Digital License")
Summary Item Price	20 digits	Numerical / Text	Black	Right-justified, formatted with comma separators
Cost Breakdown Labels	50 characters	Text	Gray	Left-justified (e.g., "Subtotal (2 items)", "Estimated Tax")

Total Price Value	20 digits	Numeral / Text	Black	Right-justified, Bold, larger font
--------------------------	-----------	----------------	-------	------------------------------------

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Payment Screen	20/4/2026	<i>KhanhNT</i>	<i>TúNHT</i>	<i>HiếuBT</i>

	Control	Operation	Function
	Search Bar	Input / Press Enter	Search for specific media items across the database
	Navigation Links	Click	Navigate to Browse, New Arrivals, Collections, or Deals pages

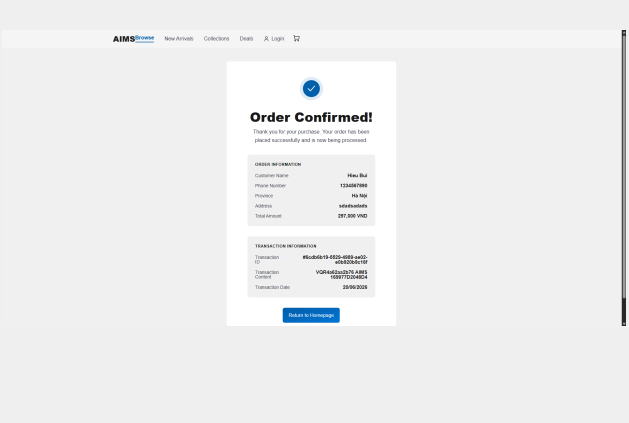
	Cart Icon	Click	Navigate back to the View Cart screen
	Method Radio Buttons	Select	Switch between available payment options (e.g., VietQR, PayPal)
	Cancel Order Link	Click	Abort the current transaction and return to the previous screen or homepage
	Confirm Payment Button	Click	Submit the selected payment details and finalize the transaction

Item	Length	Type	Color	Remarks
Page Title	50 characters	Text	Black	Center-justified, Bold ("Complete Your Purchase")
Page Subtitle	150 characters	Text	Gray	Center-justified

Section Titles	50 characters	Text	Black	Left-justified, Bold, with leading icons (e.g., "Invoice Summary", "Payment Method")
Invoice Labels	50 characters	Text	Gray	Left-justified (e.g., "Subtotal", "VAT (10%)")
Invoice Values	20 digits	Numeral / Text	Black	Right-justified, formatted with comma separators and "VND"
Total Amount Label	50 characters	Text	Black	Left-justified, Bold, includes subtext "Includes taxes and fees"
Total Amount Value	20 digits	Numeral / Text	Blue	Right-justified, Bold, larger font
Product Item Details	100 characters	Text	Black / Gray	Left-justified beside thumbnail (Title in black, Quantity in gray)
Selected Payment Box	N/A	UI Element	White / Blue	Background container with a solid blue highlight border
Recommended Badge	20 characters	Text	Blue	Uppercase text with light blue background

QR Code Image	N/A	Image	N/A	Center-aligned within the VietQR payment box
Payment Instructions	100 characters	Text	Gray	Center-justified below the QR code
Confirm Button	N/A	UI Element	Blue	Solid blue background with white text and a right-arrow icon

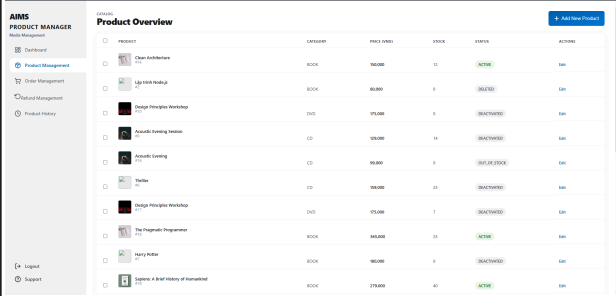
AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Order Confirmation Screen	20/4/2026	<i>BìnhNT</i>	<i>NghĩaTT</i>	<i>TúNHT</i>

	Control		Operation	Function
	Search Bar		Input / Press Enter	Search for specific media items across the database
	Navigation Links		Click	Navigate to Browse, New Arrivals, Collections, or Deals pages
	Cart Icon		Click	Navigate to the View Cart screen
	Return to Homepage Button		Click	Navigate back to the main default homepage of the application

Item	Length	Type	Color	Remarks
------	--------	------	-------	---------

Success Icon	N/A	Image/Icon	Blue / White	Center-justified, circular checkmark symbol
Page Title	50 characters	Text	Black	Center-justified, Bold, large font ("Order Confirmed!")
Confirmation Message	200 characters	Text	Gray	Center-justified paragraph
Order Details Container	N/A	UI Element	Light Gray	Background box for transaction information
Order Details Header	50 characters	Text	Black	Left-justified, Bold, Uppercase ("ORDER DETAILS")
Detail Labels	50 characters	Text	Gray	Left-justified (e.g., "Transaction ID", "Order Date")
Detail Values	50 characters	Text	Black	Right-justified, Bold (e.g., "#AIMS-123456")

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Product Management Screen	20/4/2026	KhanhNT	NghĩaTT	BìnhNT

	Control	Operation	Function
	Sidebar Navigation Links	Click	Navigate to different manager modules (Dashboard, Product Management, Order Management, Product History)
	Global Search Bar (Top)	Input / Press Enter	Perform a global search across the entire manager portal

	Notification / Settings / Profile Icons	Click	Open respective menus for alerts, system configurations, or user account settings
	Local Search Bar	Input / Press Enter	Filter the product inventory table by specific keywords or IDs
	Filter Button	Click	Open additional filtering options (e.g., by category, status, or stock level)
	Add Product Button	Click	Navigate to the screen or modal to create a new product entry
	Header Checkbox	Check / Uncheck	Select or deselect all product rows currently displayed on the page

	Row Checkbox	Check / Uncheck	Select or deselect an individual product for bulk actions
	Delete Selected Button	Click	Remove the currently selected products from the inventory (activates when items are selected)
	Pagination Buttons	Click	Navigate through different pages of the product inventory list

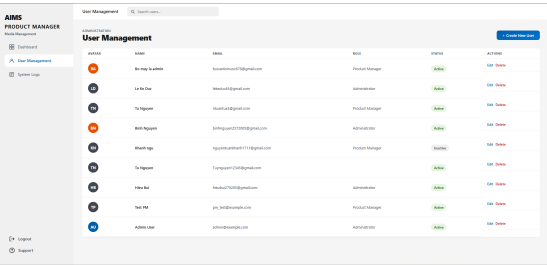
Item	Length	Type	Color	Remarks
Sidebar Active State	N/A	UI Element	Blue	Active menu item is highlighted with blue text and a left border indicator
Breadcrumb Label	50 characters	Text	Gray	Left-justified, Uppercase ("INVENTORY")

Page Title	50 characters	Text	Black	Left-justified, Bold, Large font ("Product Management")
Table Headers	20 characters	Text	Gray	Left/Center-justified, Uppercase
Product Image/Icon	N/A	Image	N/A	Left-aligned thumbnail within the product row
Product Title	100 characters	Text	Black	Left-justified, Bold
Product ID	20 characters	Text	Gray	Left-justified, positioned below the title (e.g., "ID: #P-84729")
Category Text	50 characters	Text	Gray	Left-justified
Price Value	20 digits	Numeral	Black	Right/Center-justified, formatted with comma separators
Stock Value	20 characters	Numeral/Text	Black/Gray	Center-justified, displays quantity and type (e.g., "∞ Digital")

Status Badge (Active)	20 characters	UI Element	Blue	Light blue background with dark blue text
Status Badge (Draft/Inactive)	20 characters	UI Element	Gray	Light gray background with dark gray text
Pagination Summary	50 characters	Text	Gray	Left-justified (e.g., "Showing 1 to 4 of 24 results")

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: User Management Screen	20/4/2026	<i>NghĩaTT</i>	<i>HiếuBT</i>	<i>TúNHT</i>

	Control	Operation	Function
--	----------------	------------------	-----------------

	Sidebar Navigation Links	Click	Navigate to different administrator modules (Dashboard, User Management, Role Management, System Logs)
	Global Search Bar (Top)	Input / Press Enter	Perform a global search across the entire administrator portal
	Notification / Settings / Profile Icons	Click	Open respective menus for alerts, system configurations, or user profile settings
	Create New User Button	Click	Open a modal or navigate to a dedicated screen to register a new user account
	Local Search Bar	Input / Press Enter	Search the specific user table by typing a name or email address
	Filter Button	Click	Open advanced filtering options (e.g., by role, status, or date added)

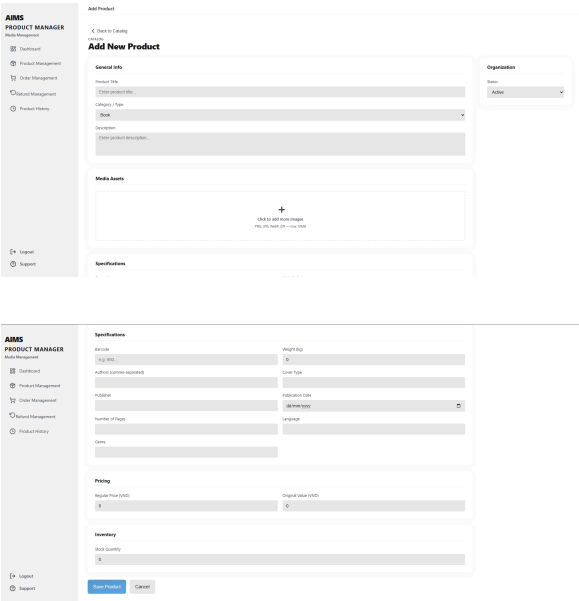
	Export Button	Click	Download the current user list data (typically as a CSV or Excel file)
	Quick Edit Icon (Actions)	Click	Open the specific user's profile or a modal to quickly edit their details or roles
	More Options Icon (Kebab)	Click	Open a dropdown menu with additional row-specific actions (e.g., Block User, Delete User, Reset Password)
	Pagination Buttons	Click	Navigate through different pages of the user table
	View Security Report Link	Click	Navigate to the detailed System Logs or Security module

Item	Length	Type	Color	Remarks
------	--------	------	-------	---------

Page Title	50 characters	Text	Black	Left-justified, Bold, Large font ("User Management")
Page Subtitle	100 characters	Text	Gray	Left-justified, below the page title
Table Headers	20 characters	Text	Gray	Left/Center-justified, Uppercase
User Avatar (Initials)	3 characters	UI Element	Varies	Circular background with white initials
User Name	50 characters	Text	Black	Left-justified, Bold
User Email	100 characters	Text	Gray	Left-justified, smaller font below the user name
Role Badge (Admin)	20 characters	UI Element	Blue	Light blue background with dark blue text
Role Badge (Standard)	20 characters	UI Element	Gray	Light gray background with dark gray text (e.g., Editor, Viewer)

Status Badge (Active)	20 characters	UI Element	Green	Light green background with a green dot and text
Status Badge (Blocked)	20 characters	UI Element	Red	Light red background with a red dot and text
Last Login String	50 characters	Text	Gray	Left-justified (e.g., "Today, 09:41 AM")
Widget Titles	50 characters	Text	Black	Bold (e.g., "Recent Security Activity", "System Secure")
Security Activity Item	100 characters	Text	Black	Left-justified text with a colored indicator dot
Security Subtext/Time	50 characters	Text	Gray	Small text indicating IP address, action details, or timestamp

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Add Product Screen	20/04/2026	KhanhNT	BinhNT	NghĩaTT

	Control	Operation	Function
	Back to Catalog link	Click	Return to the Product Catalog screen without saving the current form
	Product Title field	Input	Enter the product title
	Category dropdown	Select	Choose the product category
	Status dropdown	Select	Choose the current availability status of the product
	Description textarea	Input	Enter a textual description of the product
	Barcode (UPC/EAN) field	Input	Enter the unique barcode / numeric identifier of the product
	Original Price (excl. VAT) field	Input	Enter the original price before VAT
	Current Price (excl. VAT) field	Input	Enter the current selling price before VAT

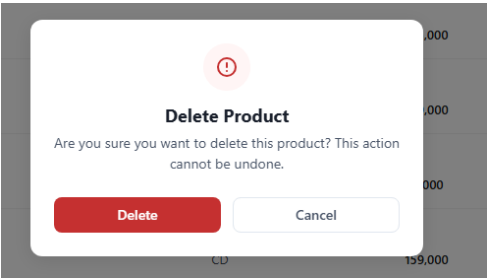
	Current Price validation message	Display	Show validation error when current price is not between 30% and 150% of the original price
	Publisher field	Input	Enter publisher information for book products
	Publication Date field	Input / Pick date	Enter or select the publication date
	Language field	Input	Enter the language of the book
	Dimensions field	Input	Enter physical dimensions in H x W x L format
	Weight field	Input	Enter product weight in kilograms
	Stock Quantity field	Input	Enter available inventory quantity
	Cancel button	Click	Cancel product creation and return to previous/catalog screen
	Save Product button	Click	Validate all required fields and save the new product

	Required-field indicator (*)	Display	Mark mandatory fields that must be completed before saving
--	------------------------------	---------	--

Item	Length	Type	Color	Remarks
Product Title	255 characters	Text	Gray	Required, left-justified
Category	50 characters	Dropdown list	Gray	Required, selectable value
Status	20 characters	Dropdown list	Gray	Required, selectable value
Description	1000 characters	Textarea	Gray	Optional, multi-line, left-justified
Barcode (UPC/EAN)	13 digits	Numerical / Text numeric	Gray	Required, unique identifier
Original Price (excl. VAT)	10 digits + 2 decimals	Decimal / Currency	Gray	Required, right-justified
Current Price (excl. VAT)	10 digits + 2 decimals	Decimal / Currency	Gray / Red on error	Required, right-justified, validated against Original Price
Publisher	255 characters	Text	Gray	Optional, left-justified
Publication Date	10 characters	Date	Gray	Optional, format mm/dd/yyyy
Language	50 characters	Text	Gray	Optional, left-justified

Dimensions (H x W x L)	50 characters	Text	Gray	Optional, format text
Weight (kg)	6 digits + 2 decimals	Decimal	Gray	Optional, non-negative
Stock Quantity	10 digits	Integer	Gray	Required, default value = 0, right-justified

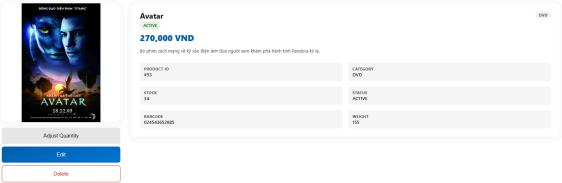
AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Delete Product Screen	20/04/2026	TúNHT	HiếuBT	BìnhNT

	Control	Operation	Function
	Warning icon area	Initial	Display warning status for delete action
	Dialog title	Initial	Display the title “Delete Product”
	Confirmation message area	Initial	Display the confirmation message for deleting the selected product
	Cancel button	Click	Close the dialog and cancel the delete action
	Delete button	Click	Delete the selected product from the system

Item	Length	Type	Color	Remarks
Dialog title	50 characters	Text	Black	Center-left justified
Confirmation message	200 characters	Text	Gray	Left-justified

Warning icon	1 icon	Icon	Red	Display only
Cancel button	10 characters	Button	Blue text / White background	Clickable
Delete button	10 characters	Button	White text / Red background	Clickable

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Product Detail Screen	20/04/2026	HiếuBT	BìnhNT	TúNHT

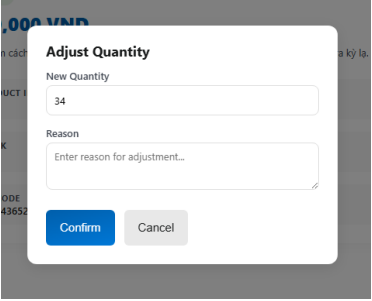
	Control	Operation	Function
	Back to Catalog link	Click	Return to Product Catalog screen
	Product image area	Initial	Display product cover image
	SKU area	Initial	Display product SKU
	Format area	Initial	Display product format
	Category tags area	Initial	Display product categories / labels
	Product title area	Initial	Display product title
	Author area	Initial	Display author name
	Description area	Initial	Display product description
	Edit icon button	Click	Navigate to Edit Product screen

	Publisher area	Initial	Display publisher information
	Publication date area	Initial	Display publication date
	Pages area	Initial	Display number of pages
	Language area	Initial	Display product language
	ISBN-10 area	Initial	Display ISBN-10 code
	ISBN-13 area	Initial	Display ISBN-13 code
	Dimensions area	Initial	Display product dimensions
	Weight area	Initial	Display product weight
	In Stock area	Initial	Display current stock quantity
	Reorder Point area	Initial	Display reorder point quantity
	Adjust Quantity button	Click	Open quantity adjustment function
	Current Price area	Initial	Display current selling price
	Original Price area	Initial	Display original price

Item	Length	Type	Color	Remarks
Back to Catalog link	20 characters	Link	Blue	Clickable
Product image	1 image	Image	Original image color	Display only
SKU	20 characters	Text	Black	Left-justified

Format	20 characters	Text	Black	Left-justified
Category tags	30 characters each	Label	Gray / Orange	Display only
Product title	150 characters	Text	Black	Bold, left-justified
Author	100 characters	Text	Gray	Left-justified
Description	1000 characters	Text	Gray	Multi-line, left-justified
Edit icon button	1 icon	Button	Black	Clickable
Publisher	100 characters	Text	Black	Left-justified
Publication date	30 characters	Date/Text	Black	Left-justified
Pages	10 digits	Numerical	Black	Left-justified
Language	30 characters	Text	Black	Left-justified
ISBN-10	10 characters	Text/Numerical	Black	Left-justified
ISBN-13	13 characters	Text/Numerical	Black	Left-justified
Dimensions	30 characters	Text	Black	Left-justified
Weight	20 characters	Text	Black	Left-justified
In Stock	10 digits	Numerical	Black	Left-justified
Reorder Point	10 digits	Numerical	Black	Left-justified
Adjust Quantity button	20 characters	Button	White text / Blue background	Clickable
Current Price	20 characters	Currency	Blue	Bold, left-justified
Original Price	20 characters	Currency	Gray	Strikethrough, left-justified

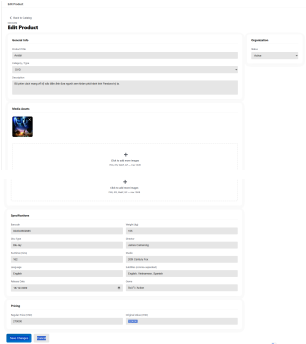
AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Stock Adjustment Dialog	20/04/2026	TùNHT	NghĩaTT	BìnhNT

	Control	Operation	Function
	Dialog title	Initial	Display the title “Stock Adjustment”
	Product name area	Initial	Display the selected product name
	Close icon button	Click	Close the dialog without saving changes
	Adjustment Quantity field	Input	Enter quantity to increase or decrease stock
	Quantity instruction text	Initial	Display guidance that negative values deduct stock
	Reason for Adjustment field	Input	Enter the reason for inventory adjustment
	Cancel button	Click	Close the dialog and cancel the stock adjustment
	Confirm button	Click	Confirm and save the stock adjustment

Item	Length	Type	Color	Remarks
Dialog title	50 characters	Text	Black	Bold, left-justified
Product name	100 characters	Text	Gray	Left-justified
Close icon button	1 icon	Button	Gray	Clickable
Adjustment Quantity	10 digits	Numerical	Gray	Input field, accepts positive and negative values
Quantity instruction text	100 characters	Text	Gray	Display only, left-justified

Reason for Adjustment	500 characters	Textarea	Gray	Multi-line, left-justified
Cancel button	10 characters	Button	Blue text / White background	Clickable
Confirm button	10 characters	Button	White text / Blue background	Clickable

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Edit Product Screen	20/04/2026	KhanhNT	BìnhNT	NghĩaTT

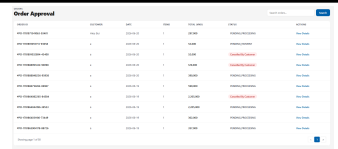
	Control	Operation	Function
	Back to Catalog link	Click	Return to Product Catalog screen
	Cancel button	Click	Cancel editing and discard changes
	Save Changes button	Click	Save updated product information
	Title field	Input	Edit product title
	Description field	Input	Edit product description
	Add Media link	Click	Add new media asset
	Primary image area	Initial	Display current primary media asset
	Add Image area	Click	Upload or add product image
	Current Price field	Input	Edit current selling price
	Compare at Price field	Input	Edit original / compare price

	Category dropdown	Select	Select product category
	Status dropdown	Select	Select product status
	Tags field	Input	Edit product tags
	ISBN field	Input	Edit product ISBN
	Publisher field	Input	Edit publisher information
	Author field	Input	Edit author name
	Format dropdown	Select	Select product format
	SKU field	Input	Edit product SKU
	Available Quantity field	Input	Edit available product quantity

Item	Length	Type	Color	Remarks
Back to Catalog link	20 characters	Link	Blue	Clickable
Cancel button	15 characters	Button	Blue text / White background	Clickable
Save Changes button	20 characters	Button	White text / Blue background	Clickable
Title	150 characters	Text	Gray	Left-justified
Description	2000 characters	Textarea	Gray	Multi-line, left-justified
Add Media link	15 characters	Link	Blue	Clickable
Primary image	1 image	Image	Original image color	Display only
Add Image area	1 area	Upload control	Gray	Clickable
Current Price (\$)	10 digits + 2 decimals	Currency	Gray	Right-justified

Compare at Price (\$)	10 digits + 2 decimals	Currency	Gray	Right-justified
Category	50 characters	Dropdown	Gray	Selectable value
Status	20 characters	Dropdown	Gray	Selectable value
Tags	255 characters	Text	Gray	Separate tags with commas
ISBN	20 characters	Text / Numerical	Gray	Left-justified
Publisher	100 characters	Text	Gray	Left-justified
Author	100 characters	Text	Gray	Left-justified
Format	30 characters	Dropdown	Gray	Selectable value
SKU	50 characters	Text	Gray	Left-justified
Available Quantity	10 digits	Numerical	Gray	Right-justified

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Order Approval Screen	20/04/2026	TúNHT	NghĩaTT	KhanhNT

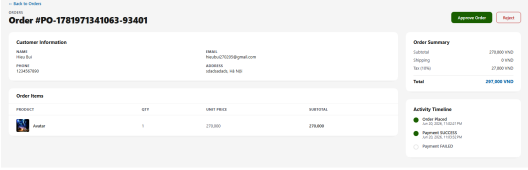
	Control	Operation	Function
	Search field	Input	Search orders by ID or customer name
	Order ID column	Initial	Display order ID
	Customer Name column	Initial	Display customer name
	Date column	Initial	Display order date
	Total Amount column	Initial	Display total order amount
	Approve button	Click	Approve the selected order

	Reject button	Click	Reject the selected order
	Current page button	Click	Display the selected page of orders
	Previous page button	Click	Navigate to previous page
	Next page button	Click	Navigate to next page
	Pagination area	Initial	Display page navigation and current page status
	Showing results text	Initial	Display current visible order range and total number of orders

Item	Length	Type	Color	Remarks
Search field	100 characters	Text	Gray	Input field, left-justified
Order ID	20 characters	Text	Blue	Clickable / left-justified
Customer Name	100 characters	Text	Black	Left-justified
Date	10 characters	Date	Black	Format dd/mm/yyyy
Total Amount (VND)	15 digits	Currency	Black	Right-justified
Approve button	10 characters	Button	White text / Blue background	Clickable
Reject button	10 characters	Button	White text / Orange background	Clickable
Current page button	5 characters	Button	White text / Blue background	Clickable
Other page button	5 characters	Button	Black text / White background	Clickable
Previous page button	2 characters	Icon/Button	Gray	Clickable
Next page button	2 characters	Icon/Button	Black	Clickable

Showing results text	30 characters	Text	Gray	Display only, left-justified
----------------------	---------------	------	------	------------------------------

AIMS Software	Date of creation	Approved by	Reviewed by	Person in charge
Screen specification: Order Detail Screen	20/04/2026	NghĩaTT	KhanhNT	BìnhNT

	Control	Operation	Function
	Breadcrumb link	Click	Return to Order Approval screen
	Reject button	Click	Reject the current order
	Approve button	Click	Approve the current order
	Customer Information area	Initial	Display customer information
	Company / Name field	Initial	Display company or customer name
	Email field	Initial	Display customer email
	Phone field	Initial	Display customer phone number
	Shipping Address field	Initial	Display shipping address
	Order Status area	Initial	Display order status information
	Date Placed field	Initial	Display date and time the order was placed
	Payment Method field	Initial	Display selected payment method

	Current Status field	Initial	Display current order status
	Order Items table	Initial	Display ordered product items
	Product column	Initial	Display ordered product information
	Category column	Initial	Display product category
	Price column	Initial	Display item unit price
	Qty column	Initial	Display ordered quantity
	Subtotal column	Initial	Display item subtotal
	Activity Timeline area	Initial	Display order activity history
	Order Summary area	Initial	Display order financial summary
	Subtotal summary field	Initial	Display subtotal amount
	VAT summary field	Initial	Display VAT amount
	Shipping Fee summary field	Initial	Display shipping fee
	Total Amount summary field	Initial	Display total order amount

Item	Length	Type	Color	Remarks
Breadcrumb link	40 characters	Link	Gray / Black	Clickable
Reject button	10 characters	Button	Orange text / White background	Clickable

Approve button	10 characters	Button	White text / Blue background	Clickable
Company / Name	100 characters	Text	Black	Left-justified
Email	100 characters	Text	Black	Left-justified
Phone	20 characters	Text	Black	Left-justified
Shipping Address	255 characters	Text	Black	Multi-line, left-justified
Date Placed	20 characters	Date/Time	Black	Format dd/mm/yyyy hh:mm
Payment Method	30 characters	Text	Black	Left-justified
Current Status	30 characters	Label	Yellow / Black	Display only
Product information	150 characters	Text	Black	Includes product name and format
Category	50 characters	Text	Gray	Left-justified
Price (VND)	15 digits	Currency	Black	Right-justified
Qty	10 digits	Numerical	Black	Center or right-justified
Item Subtotal (VND)	15 digits	Currency	Black	Right-justified
Activity Timeline entry	150 characters	Text	Gray / Black	Multi-line, display only
Subtotal amount	15 digits	Currency	Black	Right-justified
VAT amount	15 digits	Currency	Black	Right-justified
Shipping Fee	15 digits	Currency	Black	Right-justified
Total Amount	15 digits	Currency	Blue	Bold, right-justified

4.2.1 Conceptual Data Modeling

91

DVD (sub-class of DiscProduct)	Video disc	id, discType, director, studio, subtitles
PrintableProduct (sub-class of Product)	Common attributes of printed media	id, publisher, language, publishDate
Book (sub-class of PrintableProduct)	Book	id, coverType, nbPages, genre
Newspaper (sub-class of PrintableProduct)	Newspaper / periodical	id, editorInChief, issueNumber, publicationFreq, ISSN, sections
Author	A book's author	id, name

Sales / order processing

Entity	Role	Key attributes
Order	A customer's placed order with its delivery info	id (PK, plus a unique business orderId), customerName, phoneNumber, email, streetAddress, province, deliveryMethod, deliveryFee, subtotal, status, createdAt, updatedAt
OrderProduct (associative entity / line item)	One product line inside an order	quantity, price (price snapshot captured at purchase time)
Invoice	The priced bill produced for an order	id, vatSubtotal, totalAmount
Transaction (PaymentTransaction)	The payment record for the invoice	id, amount, status, method, content (the implementation also stores provider, gateway IDs, QR data, and timestamps)

Security and audit

Entity	Role	Key attributes
User	An administrator or product manager account	id, userName, email (unique), hashedPass, status
Role	A privilege role	id, roleName

UserRole (associative entity)	Resolves the User <-> Role many-to-many	(userId, roleId)
BookAuthor (associative entity)	Resolves the Book <-> Author many-to-many	(bookId, authorId)
ProductLog	Audit entry for a product add / edit / delete	id, action, createdDate

Relationships:

Relationship	Type / Cardinality	Meaning
CD / DVD is-a DiscProduct; Book / Newspaper is-a PrintableProduct; DiscProduct / PrintableProduct is-a Product	Generalization (1:1 subtype links), total & disjoint	Every Product is exactly one leaf type (CD, DVD, Book, or Newspaper). Subtypes share the parent's primary key and inherit its attributes, so common product data is stored once.
Order has Product	N : M (realised by OrderProduct)	An order contains many products and a product appears in many orders; the line stores quantity and the purchase-time price.
Order has Invoice	1 : 1	Each order produces exactly one invoice carrying the VAT subtotal and total amount.
Invoice has Transaction	1 : 1 (in the ERD)	The invoice is settled by a payment transaction.
Book written-by Author	N : M (realised by BookAuthor)	A book may have several authors; an author writes many books.
User has Role	N : M (realised by UserRole)	A user can hold multiple roles (e.g. administrator and product manager); a role is shared by many users (principle of least privilege).
Product has ProductLog	1 : N	Every change to a product is recorded as a log entry, so the full add/edit/delete history can be queried.

ProductLog created-by User	N : 1	Each log entry is attributed to the user who performed the action.
----------------------------	-------	--

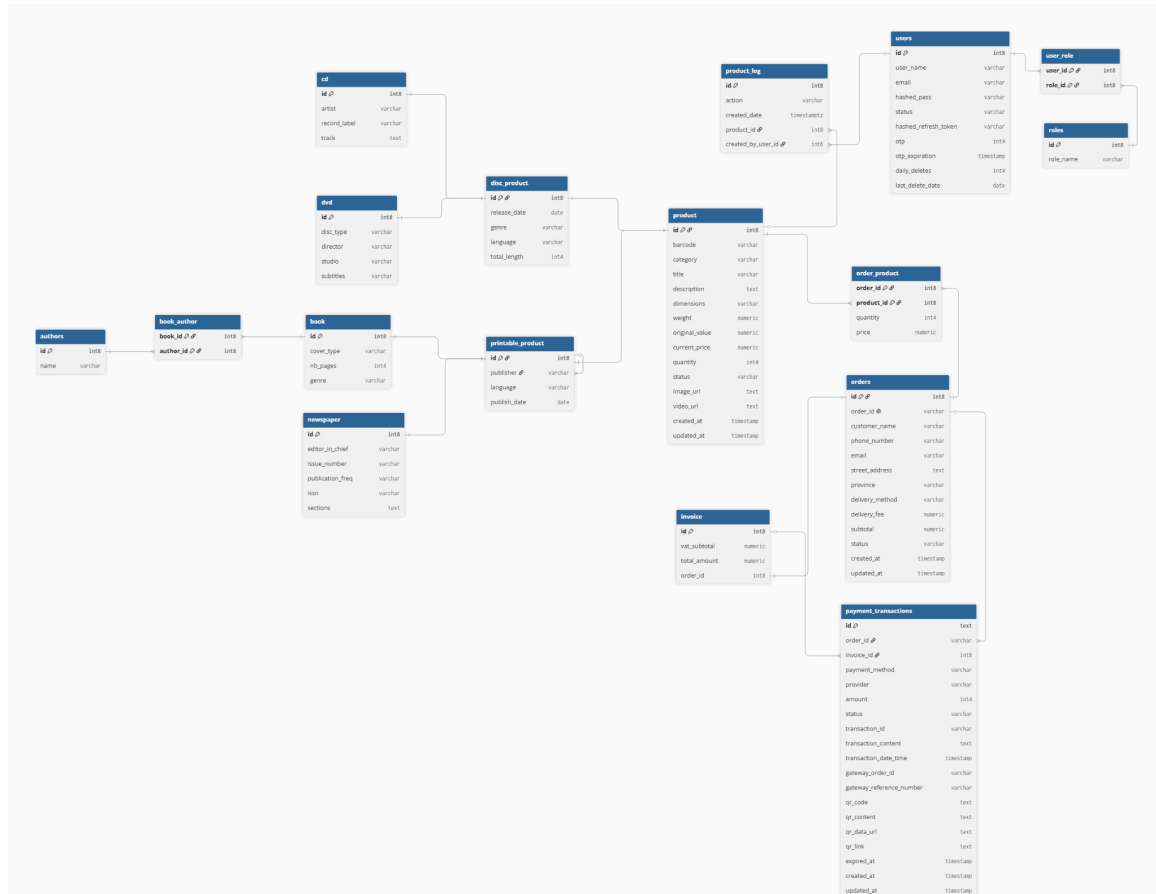
4.2.2 Database Design

4.2.2.1 Database Management System

The AIMS application utilizes PostgreSQL as its primary Database Management System. The decision to select PostgreSQL was driven by its strong reputation as a highly reliable, open-source relational database system. Because AIMS is an e-commerce platform handling financial transactions, order processing, and real-time inventory management, the strict ACID (Atomicity, Consistency, Isolation, Durability) compliance provided by PostgreSQL is essential to guarantee absolute data integrity.

PostgreSQL excels in managing complex relationships and concurrent operations, which directly supports the system's strict requirement to serve up to 1,000 simultaneous customers without performance degradation. Furthermore, the database integrates seamlessly with Prisma ORM, which is used throughout the NestJS backend to provide type-safe database queries and automated schema migrations. This combination enforces a highly structured relational data model that efficiently links users, physical media products, and order histories while maintaining strict foreign key constraints.

4.2.2.2 Database Diagram



4.2.2.3 Database Detail Design

CD

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, same as ID of DiscProduct of which type is CD
2.			artist	VARCHAR(100)	Yes	Name of the artist or performer
3.			recordLabel	VARCHAR(100)	Yes	Record label of the CD
4.			track	TEXT	Yes	List of tracks in the CD

DVD

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, same as ID of DiscProduct of which type is DVD
2.			discType	VARCHAR(45)	Yes	Disc type of the DVD
3.			director	VARCHAR(100)	Yes	Director of the DVD

4.			studio	VARCHAR(100)	Yes	Production studio or manufacturer
5.			subtitles	VARCHAR(255)	Yes	Available subtitles of the DVD

DiscProduct

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, same as ID of Product of which type is DiscProduct
2.			releaseDate	DATE	Yes	Release date of the disc product
3.			genre	VARCHAR(100)	Yes	Genre of the disc product
4.			language	VARCHAR(45)	Yes	Language of the disc product
5.			totalLength	INTEGER	Yes	Total length of the disc product in minutes

Invoice

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, auto increment
2.			VATsubtotal	NUMERIC(12,2)	Yes	VAT subtotal of the invoice
3.			totalAmount	NUMERIC(12,2)	Yes	Final total amount of the invoice
4.		x	orderId	BIGINT	Yes	Order ID associated with this invoice

Transaction

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	TEXT	Yes	Payment transaction id
2.		x	order_id	VARCHAR	Yes	Order ID associated with this payment transaction
3.		x	invoice_id	INT8	Yes	InvoiceID associated with this payment transaction
4.			payment_method	VARCHAR	Yes	Payment method, e.g., credit card, VietQR
5.			Provider	VARCHAR	Yes	Payment provider name

6.			amount	INT4	Yes	Payment amount
7.			status	VARCHAR	Yes	Payment transaction status, e.g, pending, success,failed,expired
8.			transaction_id	VARCHAR	Yes	Transaction ID returned from the payment provider or bank transaction
9.			transaction_content	TEXT	Yes	Transaction content or payment note
10.			transaction_date_time	TIMESTAMP	Yes	Date and time when the transaction is completed or recorded
11.			gateway_order_id	VARCHAR	No	Order ID created by the external payment gateway
12.			gateway_reference_number	VARCHAR	No	Reference number returned by the payment gateway
13.			qr_code	TEXT	No	QR code data returned from VietQR provider
14.			qr_content	TEXT	No	Payment content embedded in the QR code
15.			qr_data_url	TEXT	No	QR code image in data URL format
16.			qr_link	TEXT	No	QR code link returned by provider
17.			expired_at	TIMESTAMP	No	Expiration time of the pending payment transaction
18.			created_at	TIMESTAMP	Yes	Timestamp when the payment was created
19.			updated_at	TIMESTAMP	Yes	Timestamp when the payment transaction was last updated

User

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, auto increment
2.			userName	VARCHAR(100)	Yes	Username used for authentication
3.			email	VARCHAR(255)	Yes	User email address
4.			hashedPass	VARCHAR(255)	Yes	Hashed password of the user
5.			status	VARCHAR(30)	Yes	User account status, e.g., active, inactive, blocked
6.			hashed_refresh_token	VARCHAR(255)	Yes	Token to active silent refresh

7.			otp	INTEGER	NO	One time password when create account or change pass
8.			otp_expiration	DATE	No	Expire date of otp code
9.			daily_deletes	INTEGER	No	Quota deleted each day
10.			last_delete_date	DATE	No	Date final delete product

Role

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, auto increment
2.			roleName	VARCHAR(100)	Yes	Name of the role assigned to users

UserRole

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x	x	userId	BIGINT	Yes	User ID, foreign key referencing "User"(id)
2.	x	x	roleId	BIGINT	Yes	Role ID, foreign key referencing Role(id)

Product

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	Product ID, auto increment
2			barcode	VARCHAR(50)	Yes	Product barcode
3			category	VARCHAR(50)	Yes	Product category
4			title	VARCHAR(150)	Yes	Product title
5			description	TEXT	Yes	Product description
6			dimensions	VARCHAR(100)	No	Product dimensions
7			weight	NUMERIC(10,2)	Yes	Product weight
8			originalValue	NUMERIC(12,2)	Yes	Original value of product
9			currentPrice	NUMERIC(12,2)	Yes	Current selling price
10			quantity	INTEGER	Yes	Number of products in stock
11			status	VARCHAR(30)	Yes	Product status
12			image_url	TEXT	Yes	Product image URL

13			video_url	TEXT	Yes	Product video URL
14			createdAt	TIMESTAMP	Yes	Created date
15			updatedAt	TIMESTAMP	No	Updated date

Printable Product

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	ID
2			publisher	VARCHAR(150)	Yes	Publisher name
3			language	VARCHAR(50)	Yes	Publication language
4			publishDate	DATE	Yes	Publication date

Book

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	ID
2			coverType	VARCHAR(50)	Yes	Book cover type
3			nbPages	INTEGER	Yes	Number of pages
4			genre	VARCHAR(100)	Yes	Book genre

Newspaper

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	ID
2			editorInChief	VARCHAR(100)	Yes	Editor-in-chief name
3			issueNumber	VARCHAR(50)	Yes	Newspaper issue number
4			publicationFreq	VARCHAR(50)	Yes	Publication frequency
5			ISSN	VARCHAR(20)	Yes	International Standard Serial Number
6			sections	TEXT	Yes	Newspaper sections

Authors

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	Author ID, auto increment
2			name	VARCHAR(100)	Yes	Author name

Book_Author

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x	x	bookId	BIGINT	Yes	ID of book
2	x	x	authorId	BIGINT	Yes	ID of author

Orders

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		id	BIGSERIAL	Yes	Order ID, auto increment
2			orderId	VARCHAR(50)	Yes	Order code
3			customerName	VARCHAR(100)	Yes	Customer name
4			phoneNumber	VARCHAR(20)	Yes	Customer phone number
5			email	VARCHAR(150)	Yes	Customer email
6			streetAddress	TEXT	Yes	Customer street address
7			province	VARCHAR(100)	Yes	Customer province
8			deliveryMethod	VARCHAR(50)	Yes	Delivery method
9			deliveryFee	NUMERIC(12,2)	Yes	Delivery fee
10			subtotal	NUMERIC(12,2)	Yes	Order subtotal
11			status	VARCHAR(30)	Yes	Order status
12			createdAt	TIMESTAMP	Yes	Created date
13			updatedAt	TIMESTAMP	No	Updated date

Order_product

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x	x	orderId	BIGINT	Yes	ID of order
2	x	x	productId	BIGINT	Yes	ID of product
3			quantity	INTEGER	Yes	Quantity of product in order
4			price	NUMERIC(12,2)	Yes	Product price at order time

ProductLog

#	PK	FK	Column Name	Data type	Mandatory	Description
1.	x		id	BIGSERIAL	Yes	ID, auto increment
2.			action	VARCHAR(100)	Yes	Action performed on the product, e.g., create, update, delete

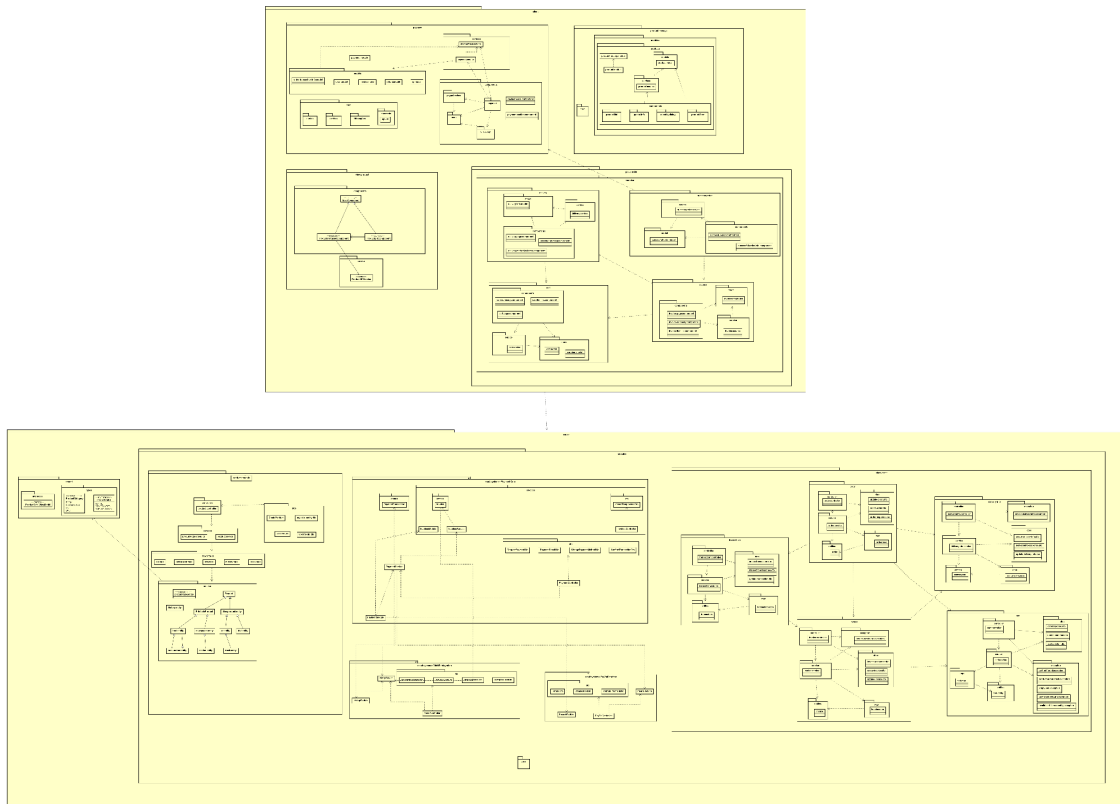
3.			createdDate	TIMESTAMP WITH TIME ZONE	Yes	Date and time when the log record is created
4.		x	productId	BIGINT	Yes	Product ID associated with this log
5.		x	createdByUserId	BIGINT	Yes	User ID of the person who created this log

4.3 Non-Database Management System Files

Our AIMS application does not utilize any non-DBMS files for storing business logic, transactional records, or application data. All core system data, including user accounts, product details, shopping carts, and order histories, are structured and stored exclusively within the PostgreSQL Relational Database Management System. Furthermore, media assets such as product images are processed directly from memory buffers and uploaded to the external Cloudinary service, entirely bypassing the local file system.

4.4 Class Design

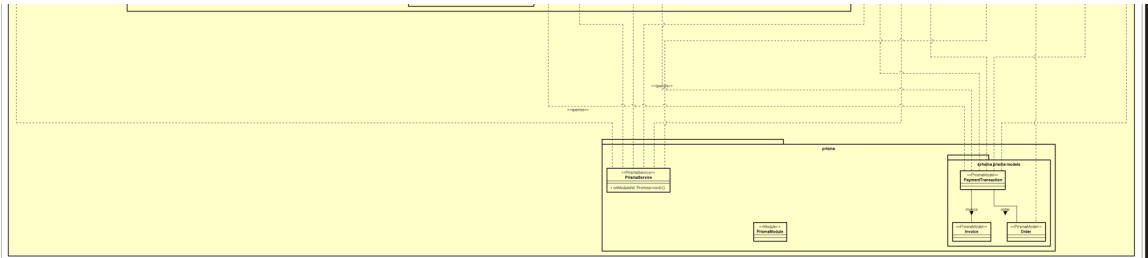
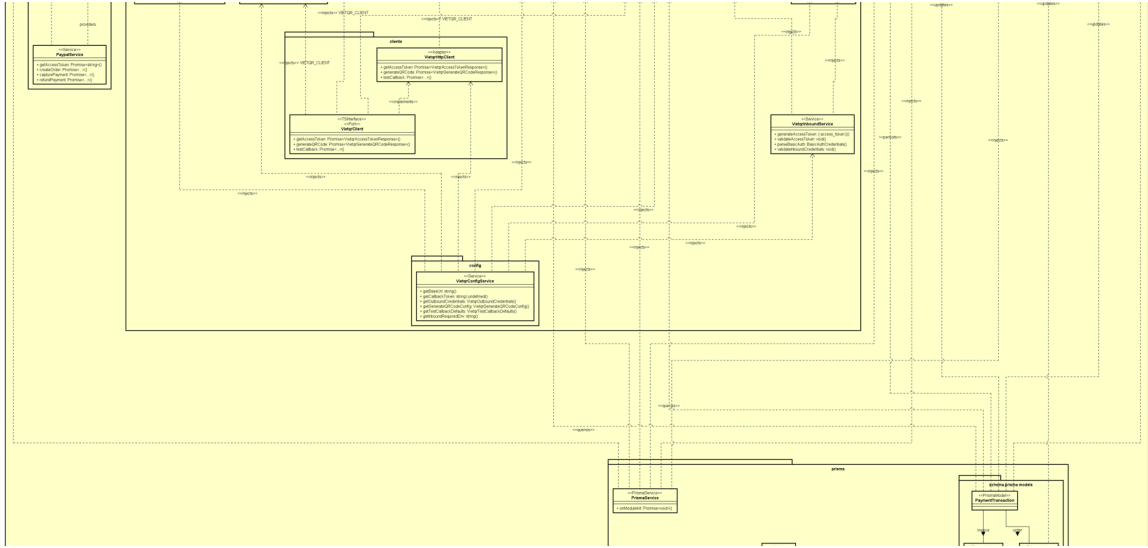
4.4.1 General Class Diagram



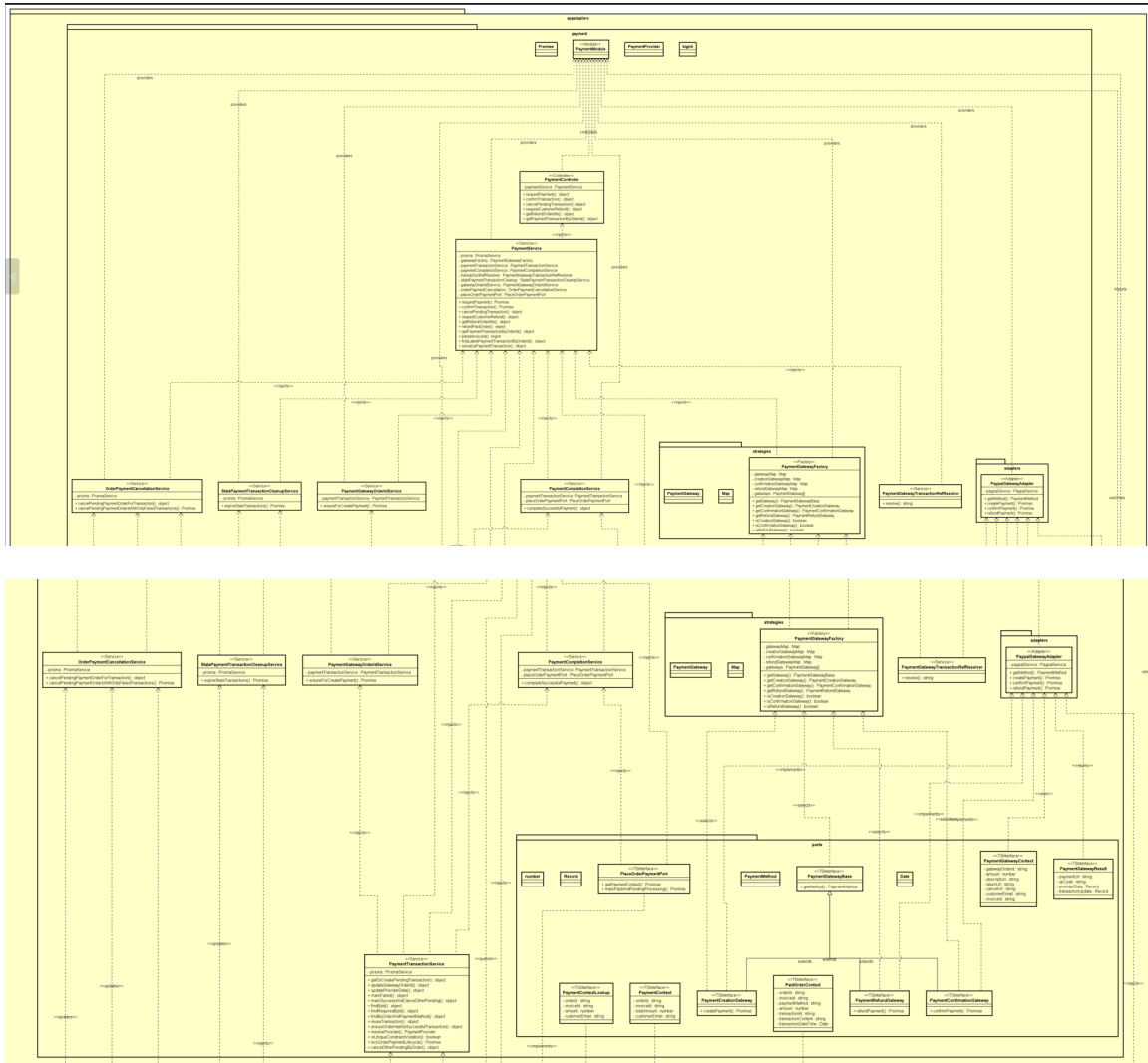
4.4.2 Class Diagrams

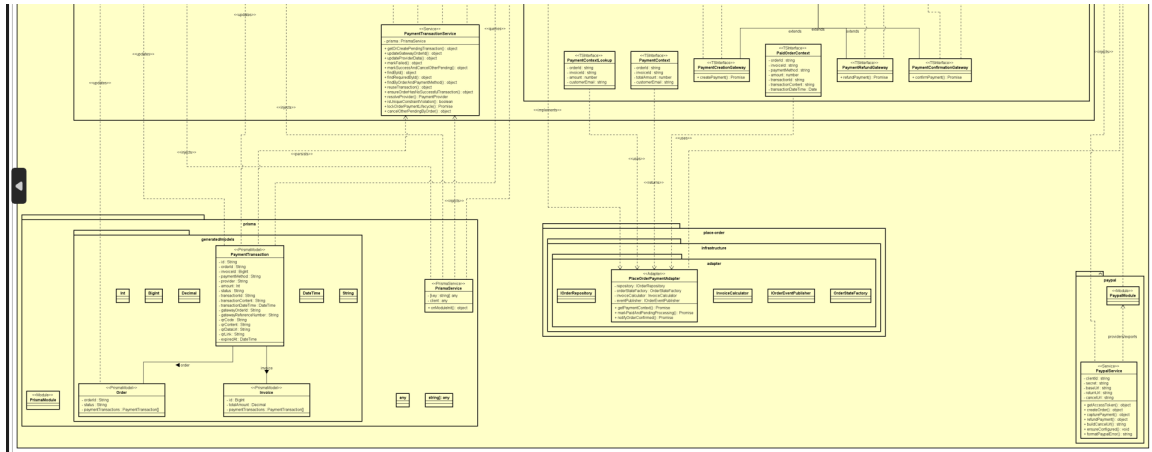
4.4.2.1 Class Diagram for Package Payment + Subsystem Vietqr





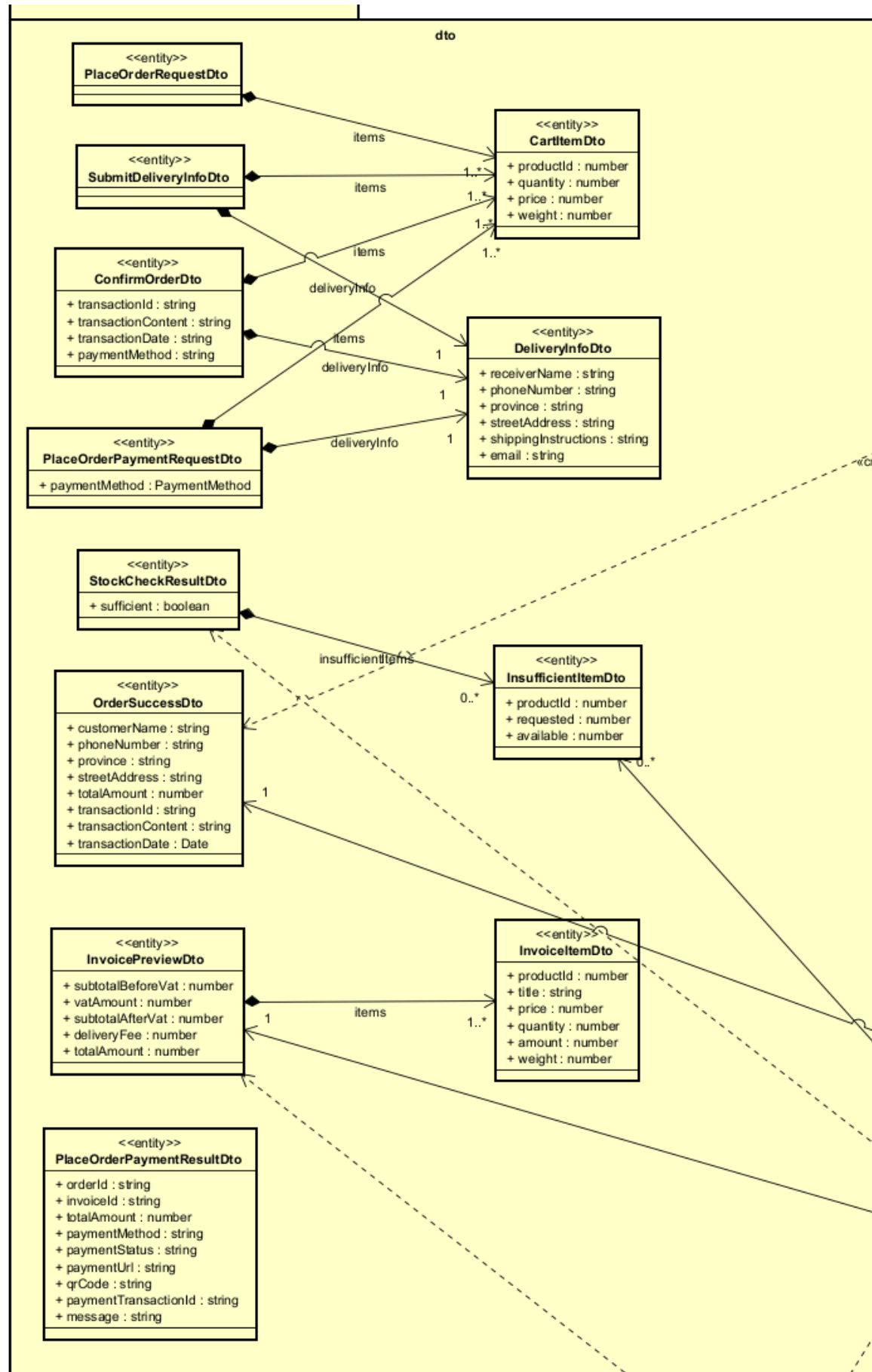
4.4.2.2 Class Diagram for Package Payment + Subsystem Paypal



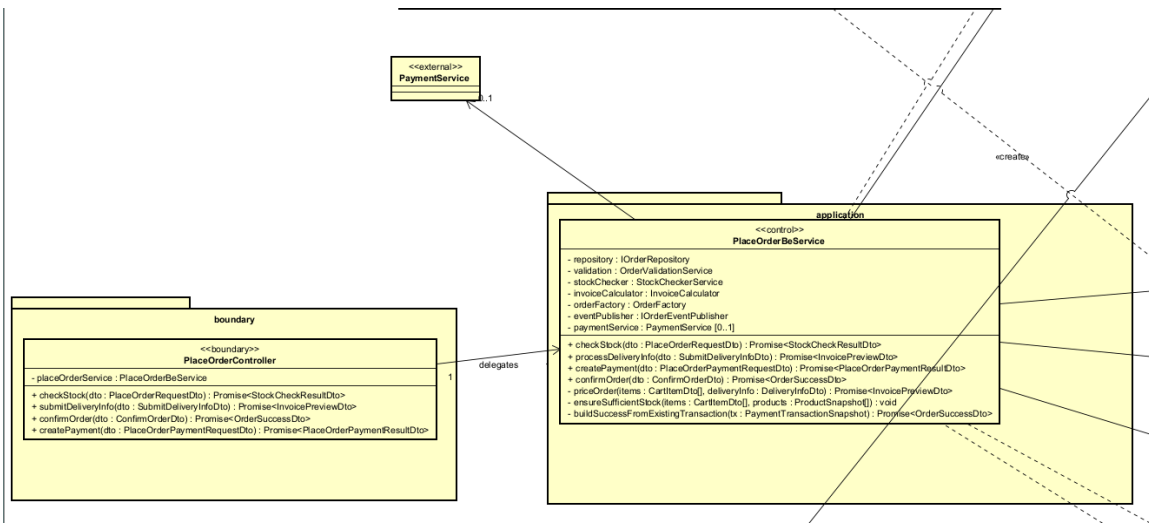


4.4.2.3 Class Diagram for Package Place Order

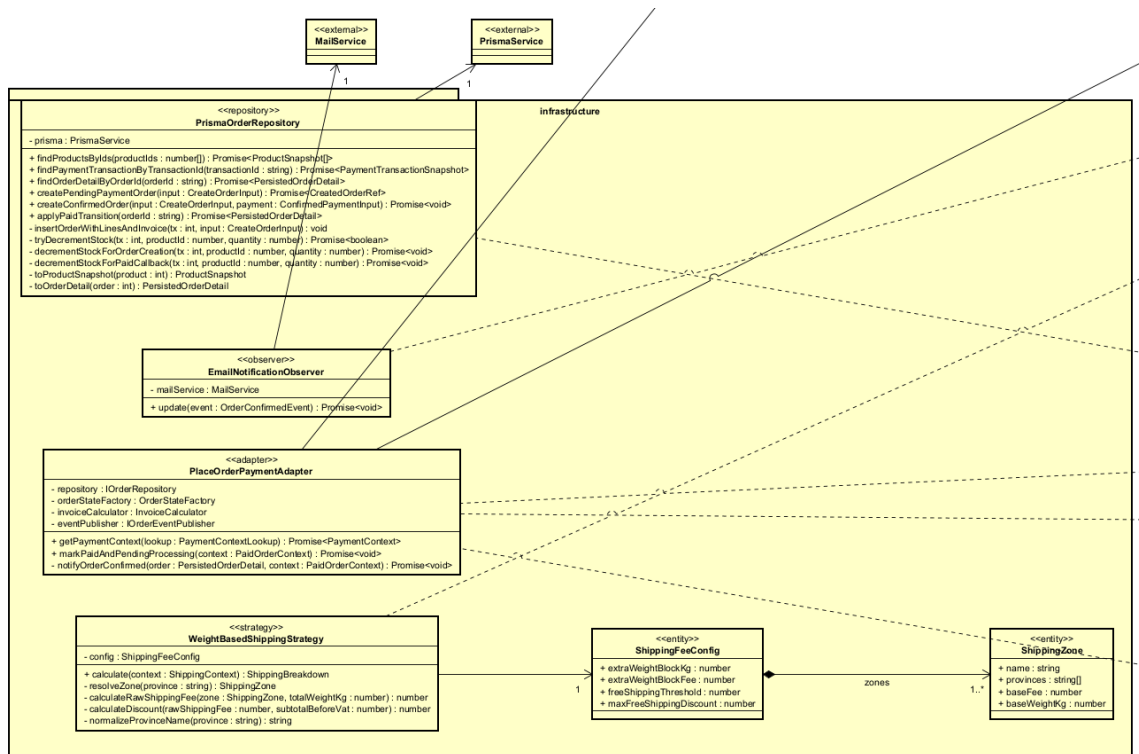
Package DTO:



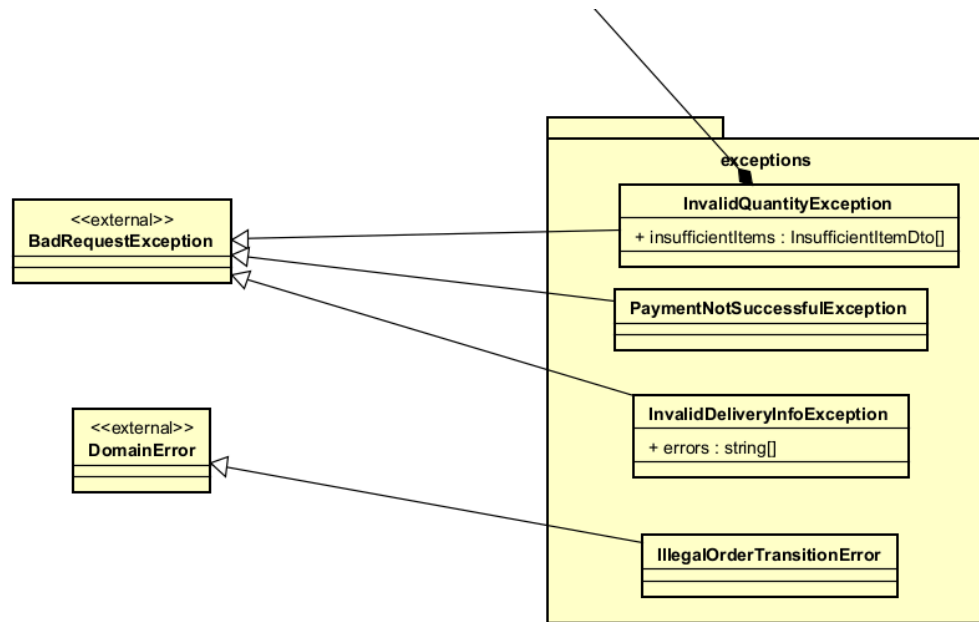
Package Application:



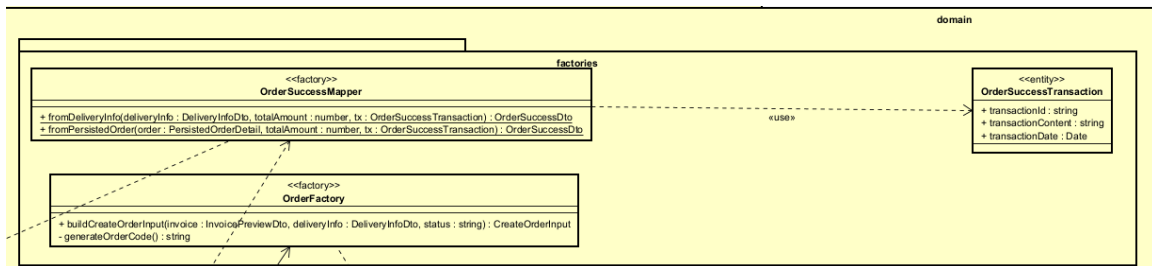
Package Infrastructure:



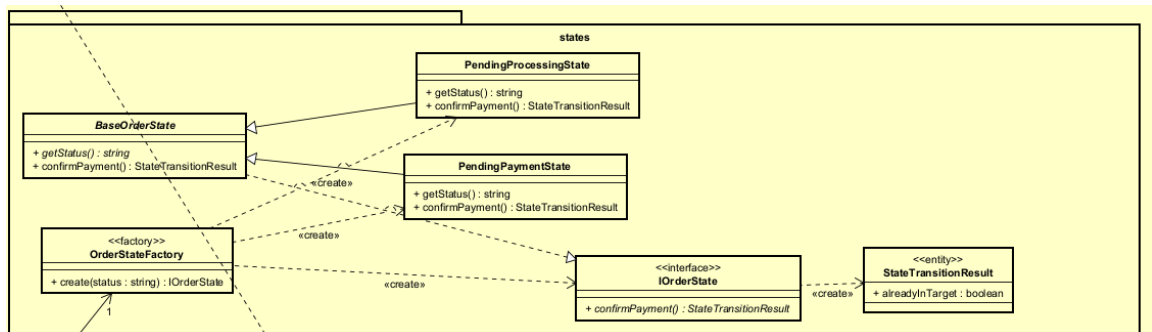
Package Exception:



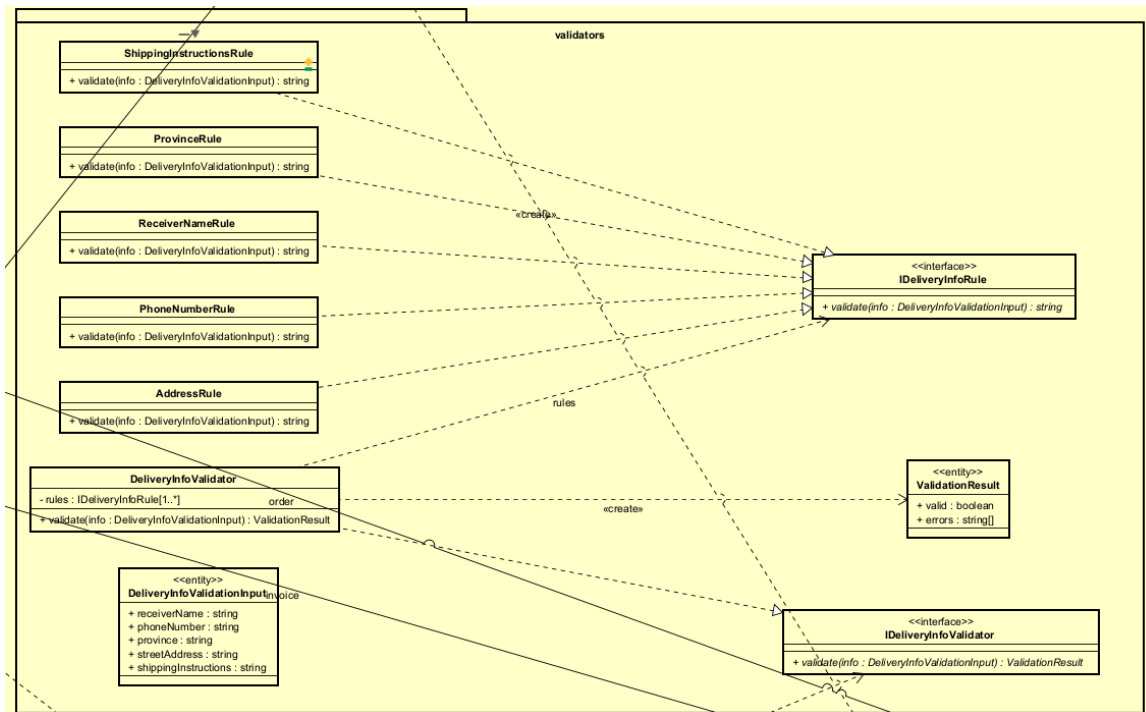
Package Domain/Factories:



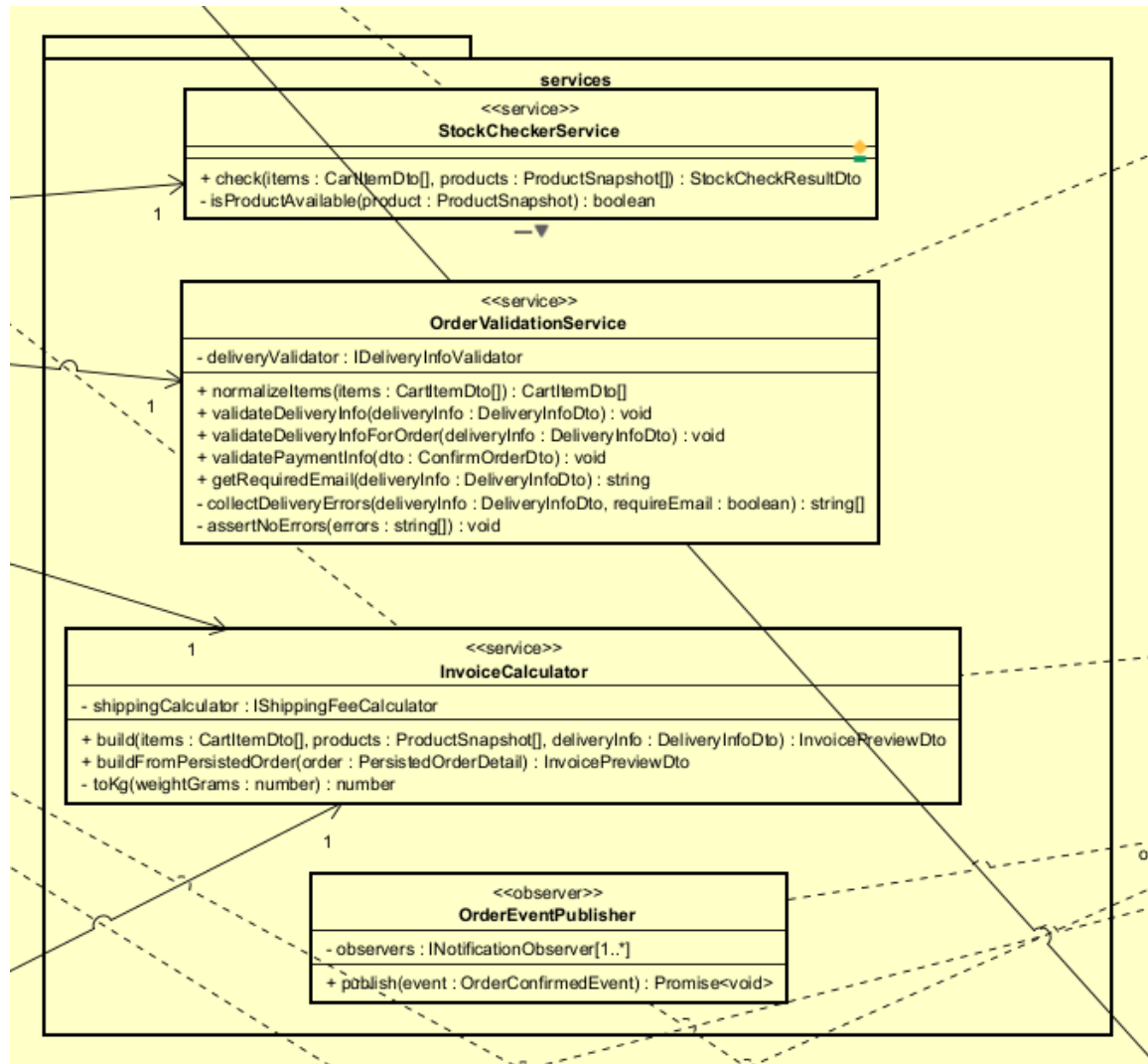
Package Domain/States:



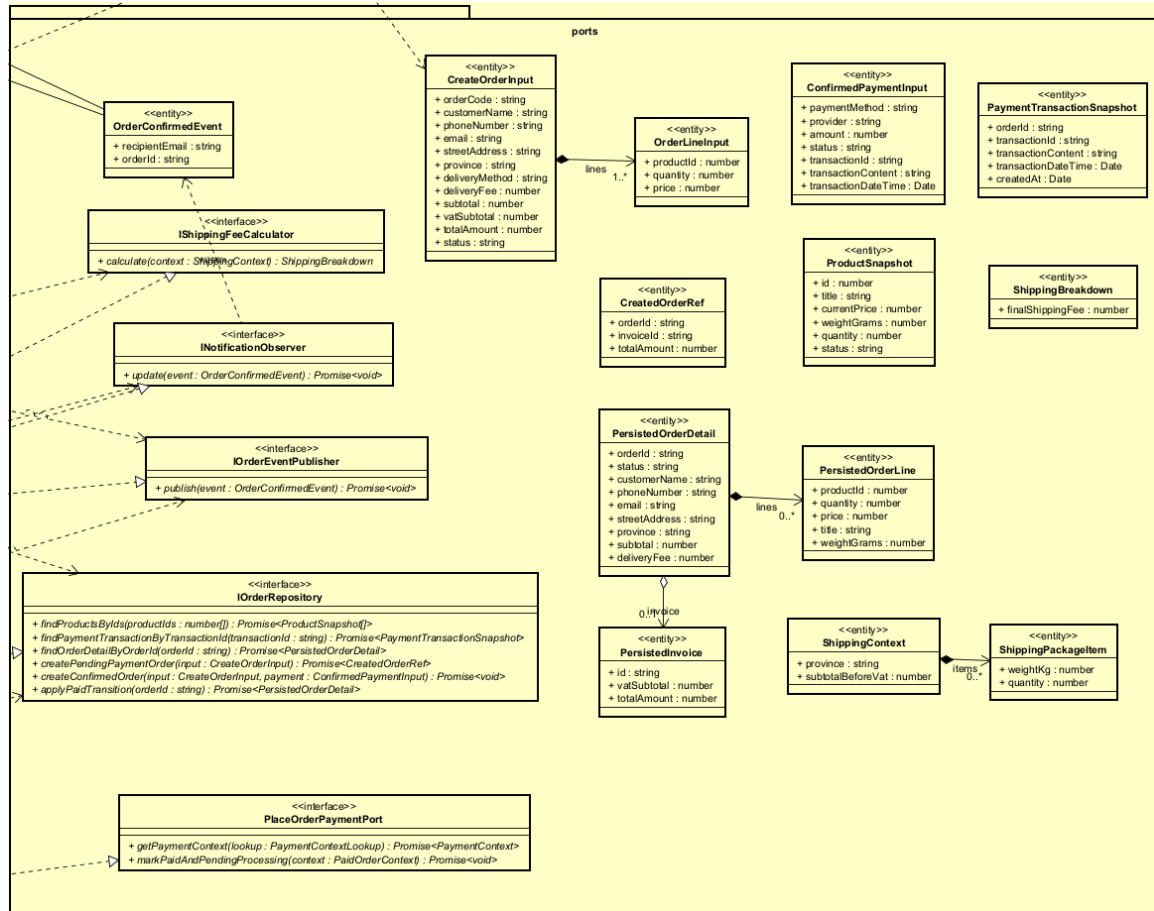
Package Domain/Validators:



Package Domain/Services:



Package Domain/Ports:



4.4.2.4 Class Diagram for View Product Detail

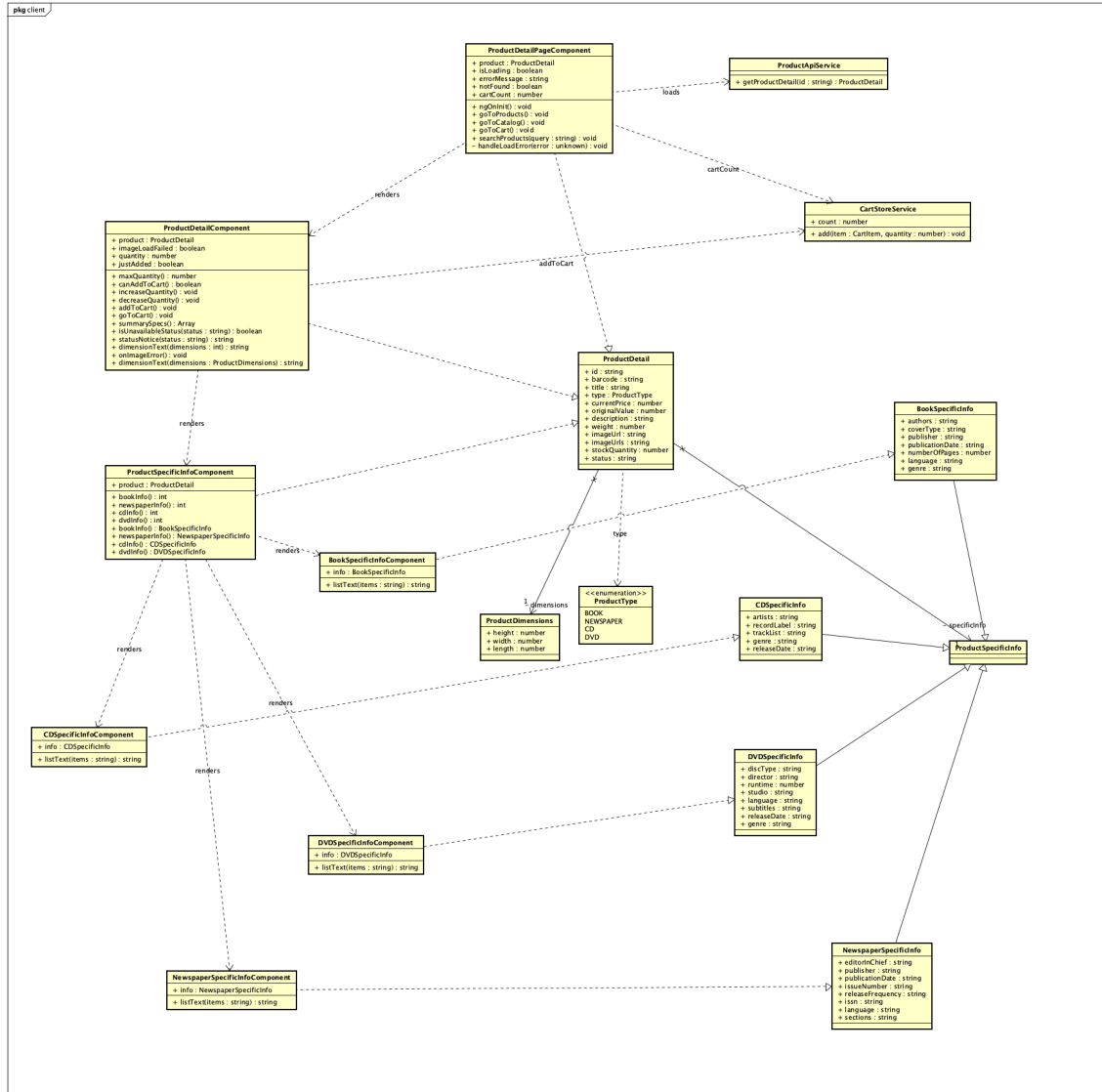


Figure: Frontend class diagram – View Product Detail (UC235)

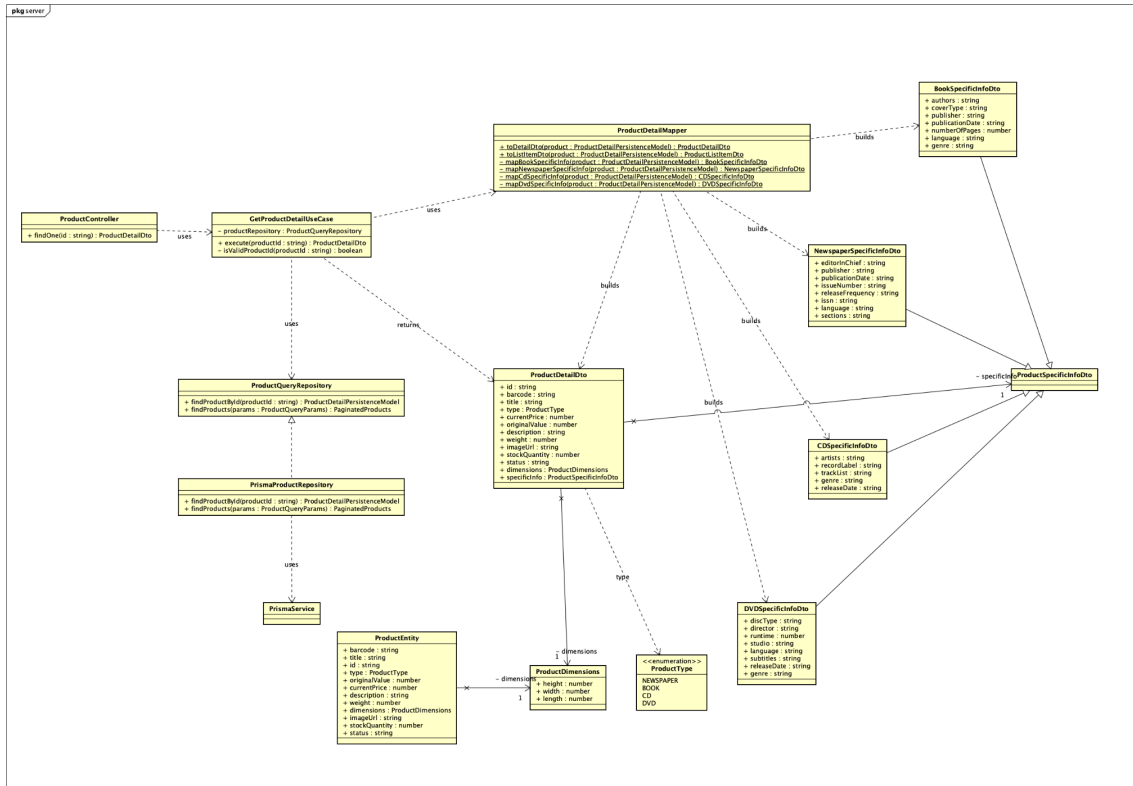


Figure: Backend class diagram – View Product Detail (UC235)

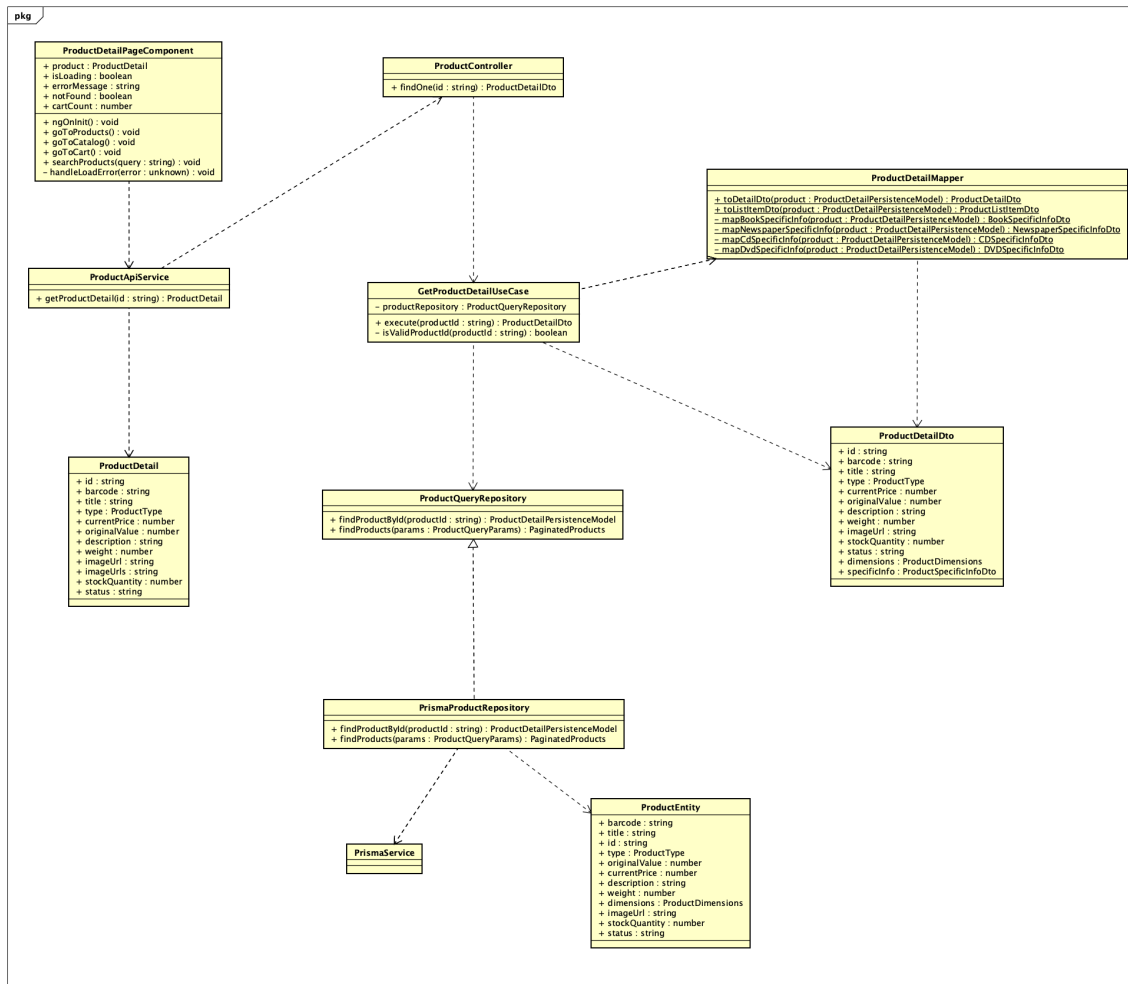
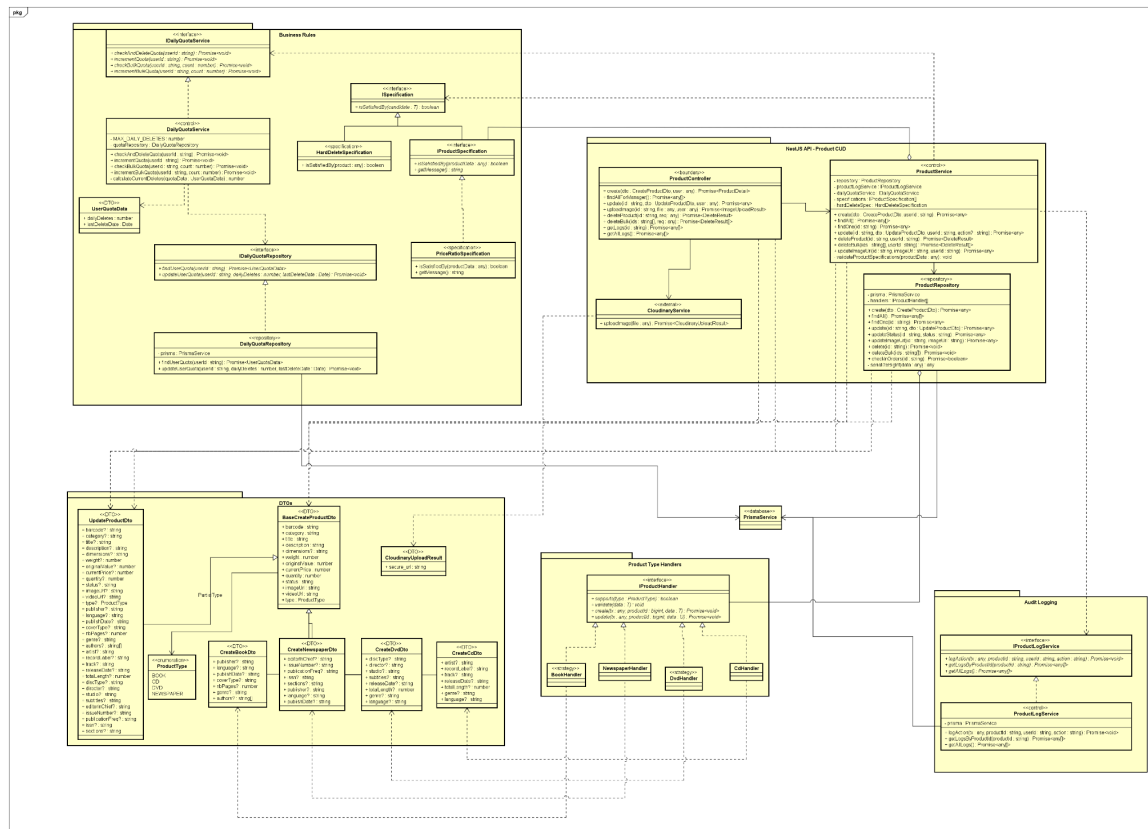
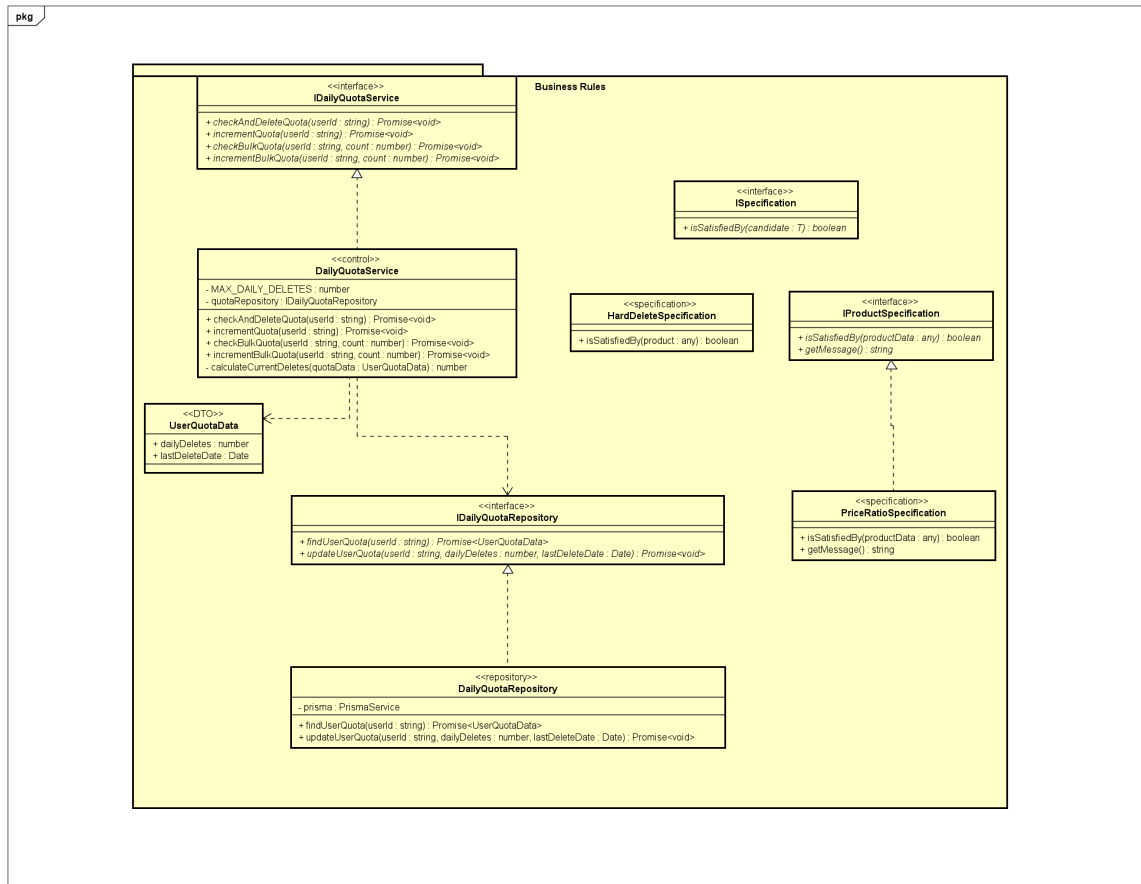


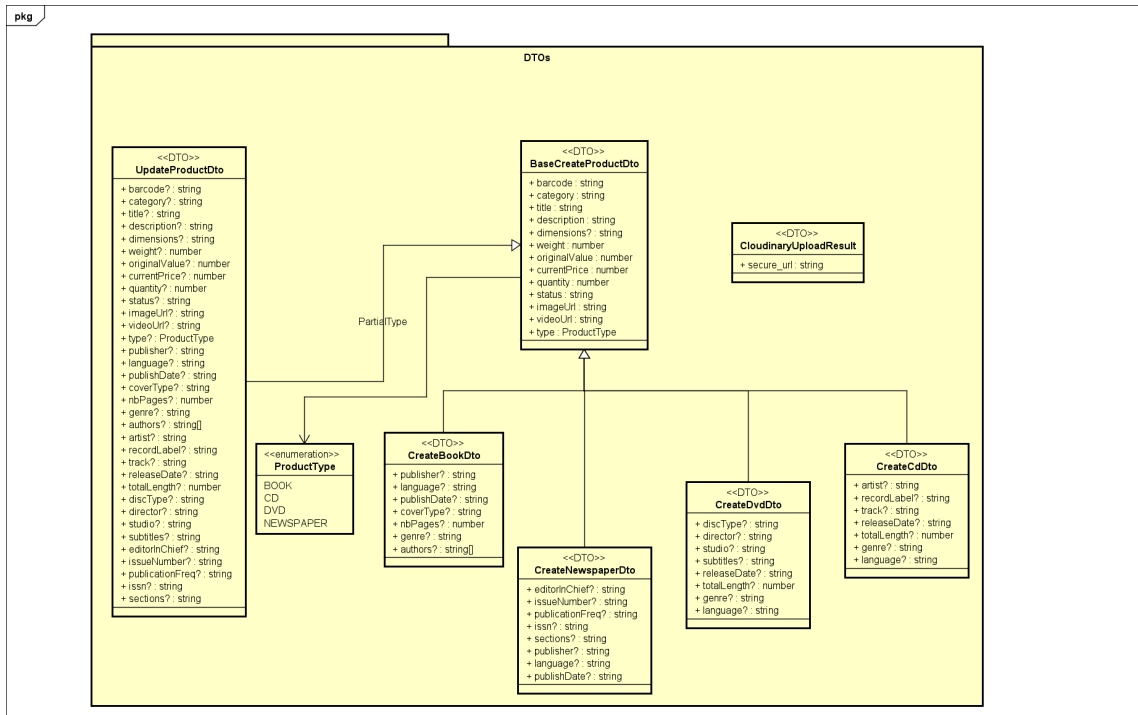
Figure: Backend-Frontend connection – View Product Detail (UC235)

4.4.2.5 CUD Product

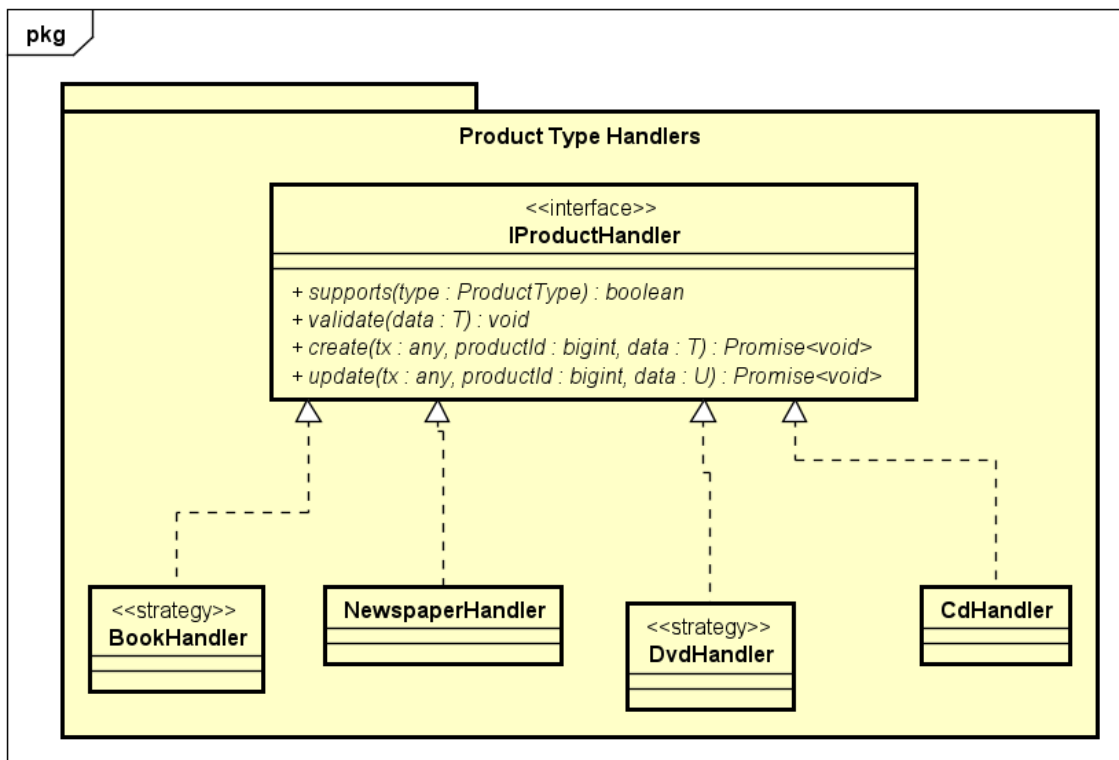


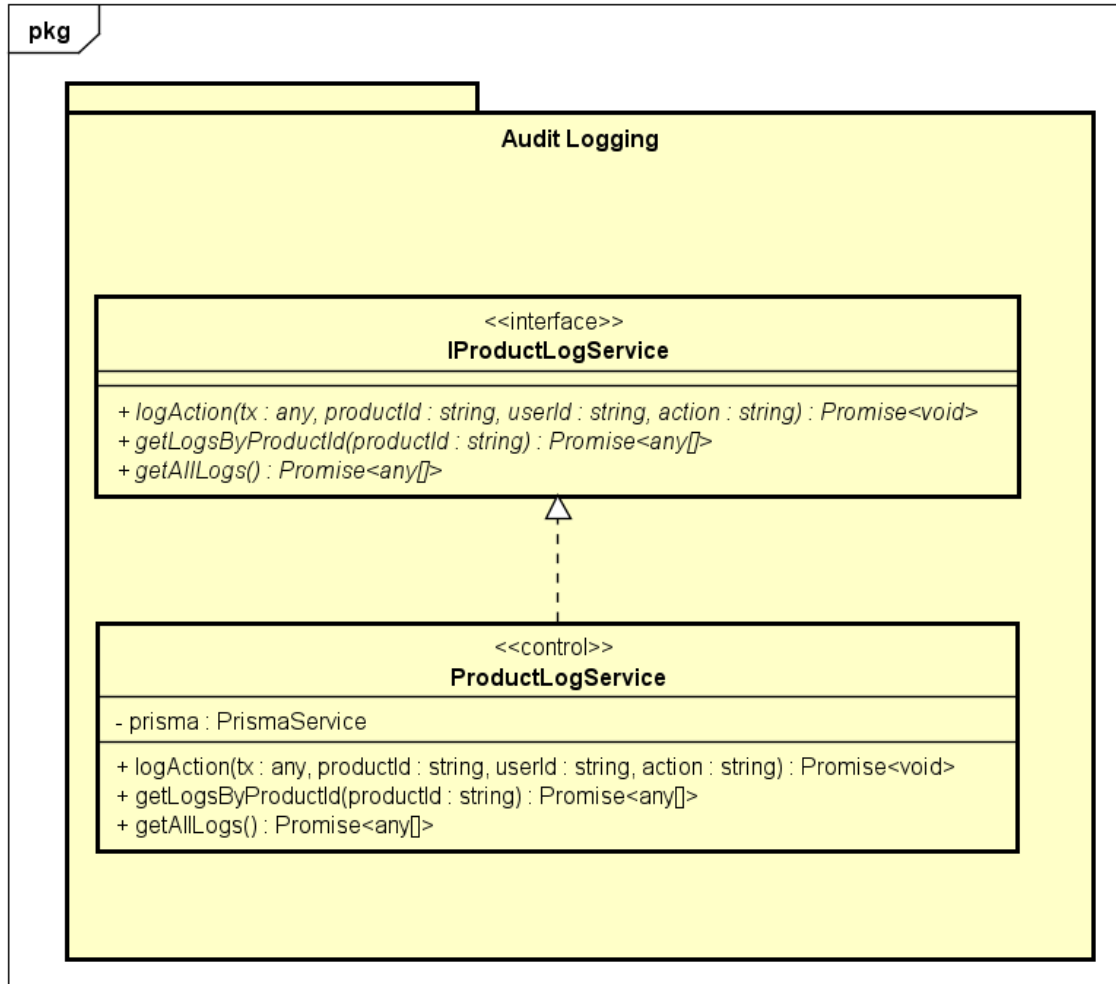


Package business rule

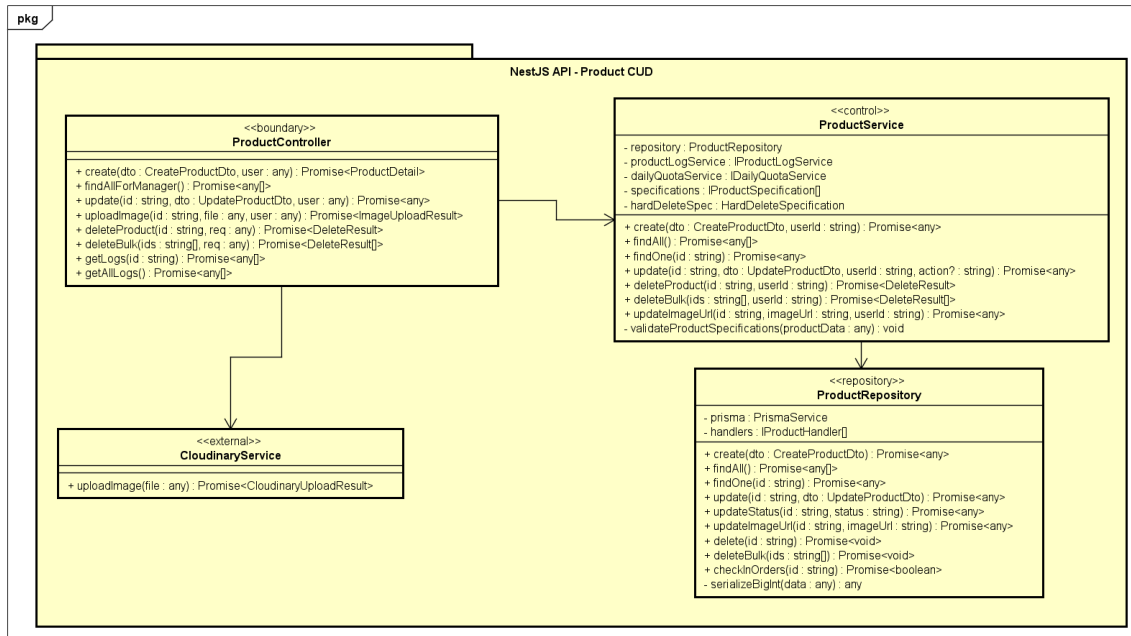


DTO package





ProductLog package



Core package CUD product

4.4.3 Class Design

4.4.3.1 Class PaymentService

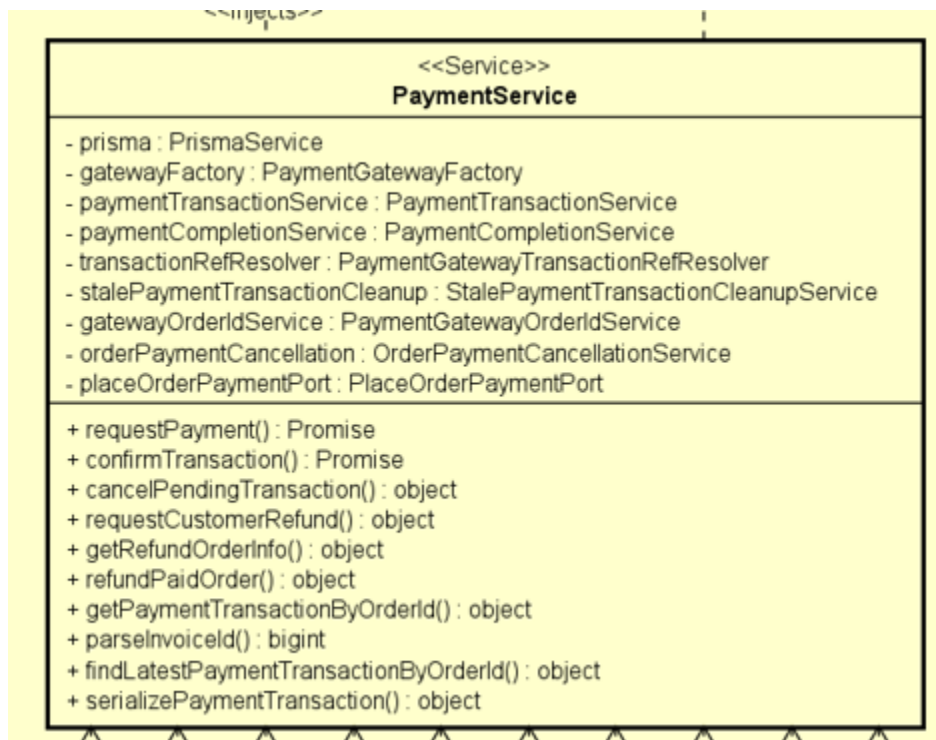


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	prisma	PrismaService	Injected by NestJS	Provides database access.
2	gatewayFactory	PaymentGatewayFactory	Injected by NestJS	Selects the correct payment gateway by payment method.
3	paymentTransactionService	PaymentTransactionService	Injected by NestJS	Manages payment transaction lifecycle rules.
4	paymentCompletionService	PaymentCompletionService	Injected by NestJS	Completes successful payments and updates order status.

5	transactionRefResolver	PaymentGatewayTransactionRefResolver	Injected by NestJS	Resolves the transaction reference used by the gateway.
6	stalePaymentTransactionCleanup	StalePaymentTransactionCleanupService	Injected by NestJS	Expires stale pending payment transactions.
7	gatewayOrderIdService	PaymentGatewayOrderIdService	Injected by NestJS	Ensures gateway order ID before payment creation.
8	orderPaymentCancellation	OrderPaymentCancellationService	Injected by NestJS	Cancels pending payment orders.
9	placeOrderPaymentPort	PlaceOrderPaymentPort	Injected by NestJS	Connects payment flow with place-order domain.

Table 3. Operation design

#	Name	Return type	Description
1	requestPayment(dto)	Promise<PaymentResult Dto>	Creates a payment request for VietQR or PayPal.
2	confirmTransaction(dto)	Promise<PaymentResult Dto>	Manually confirms a payment transaction. PayPal uses this flow; VietQR is confirmed by callback only.
3	cancelPendingTransaction(transactionId)	Promise<any>	Cancels a pending payment transaction and related order state.
4	requestCustomerRefund(dto)	Promise<any>	Handles customer refund request and cancels the order if allowed.
5	getRefundOrderInfo(orderId)	Promise<any>	Returns order and payment information for refund display.

6	refundPaidOrder(orderId, reason)	Promise<any>	Performs refund logic through the payment gateway if supported.
7	getPaymentTransactionByOrderId(orderId, paymentMethod?)	Promise<any>	Returns the latest payment transaction for an order.

Parameter:

- dto: Request DTO containing payment, confirmation, or refund data.
- transactionId: Internal payment transaction ID.
- orderId: Public order ID.
- reason: Refund reason.
- paymentMethod: Optional payment method filter.

Exception:

- BadRequestException if amount mismatches, invoice ID is invalid, confirmation is unsupported, or order cannot be cancelled.
- NotFoundException if order or payment transaction is not found.

Method:

- requestPayment() validates order payment context, creates/reuses a pending transaction, selects gateway, creates payment, and stores provider data.
- confirmTransaction() finds latest transaction, rejects VietQR manual confirmation, calls gateway confirmation, then marks payment as successful.
- refundPaidOrder() tries automatic refund through gateway; if unsupported, marks refund as required.

4.4.3.2 Class *PaymentController*

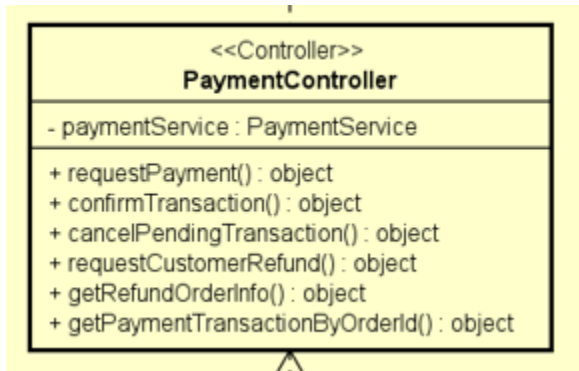


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paymentService	PaymentService	Injected by NestJS	Handles payment business logic.

Table 3. Operation design

#	Name	Return type	Description
1	requestPayment()	Promise<PaymentResult Dto>	Endpoint for creating a payment request.
2	confirmTransaction()	Promise<PaymentResult Dto>	Endpoint for confirming a transaction.
3	cancelPendingTransaction()	Promise<any>	Endpoint for cancelling a pending payment transaction.
4	requestCustomerRefund()	Promise<any>	Endpoint for requesting customer refund.

5	getRefundOrderInfo()	Promise<any>	Endpoint for fetching refund-related order information.
6	getPaymentTransactionByOrderId()	Promise<any>	Endpoint for fetching latest payment transaction by order.

Parameter:

- requestPaymentDto: Payment request data.
- confirmTransactionDto: Payment confirmation data.
- transactionId: Internal transaction ID.
- dto: Refund request data.
- orderId: Public order ID.
- paymentMethod: Optional payment method query.

Exception:

- Exceptions are delegated from PaymentService.

Method:

- This class only exposes REST endpoints and delegates business logic to PaymentService.

4.4.3.3 Class *PaymentTransactionService*

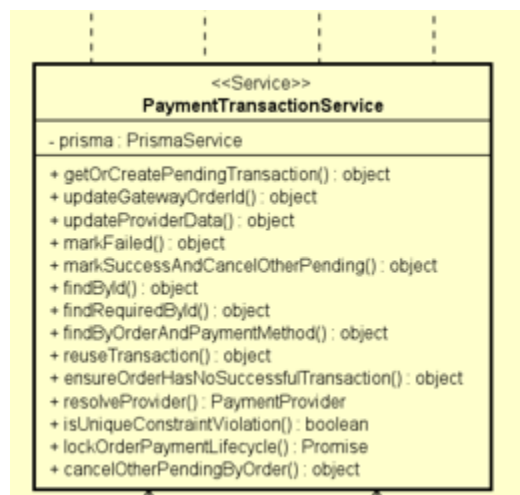


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	prisma	PrismaService	Injected by NestJS	Provides database access for payment transactions.

Table 3. Operation design

#	Name	Return type	Description
1	getOrCreatePendingTransaction(input)	Promise<any>	Creates a new pending transaction or reuses an existing one.
2	updateGatewayOrderId(id, gatewayOrderId)	Promise<any>	Updates gateway order ID for a transaction.
3	updateProviderData(id, data)	Promise<any>	Updates provider-specific transaction fields.
4	markFailed(id)	Promise<any>	Marks a transaction as failed.
5	markSuccessAndCancelOtherPending(input)	Promise<any>	Marks one transaction successful and fails other pending transactions of the same order.
6	findById(id)	Promise<any>	Finds a transaction by ID.

7	cancelOtherPendingByOrder(orderId, activeTransactionId)	Promise<any>	Marks other pending transactions of the order as failed.
---	---	--------------	--

Parameter:

- input: Transaction creation or success-marking data.
- id: Payment transaction ID.
- gatewayOrderId: Gateway-specific order ID.
- data: Provider update data.
- orderId: Public order ID.
- activeTransactionId: Transaction that should remain active.

Exception:

- BadRequestException if the order already has a successful payment.
- NotFoundException if the transaction is not found.
- Other database exceptions may be thrown by Prisma.

Method:

- The service centralizes transaction lifecycle rules across VietQR and PayPal.
- When one transaction succeeds, other pending transactions of the same order are marked as failed.

4.4.3.4 Class PaymentGatewayFactory

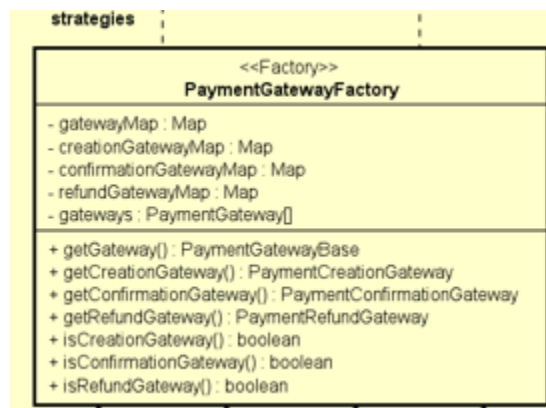


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	gateways	PaymentGateway[]	Injected by NestJS	List of available payment gateway adapters.
2	gatewayMap	Map<PaymentMethod, PaymentGatewayBase>	Initialized in constructor	Stores all gateways by payment method.
3	creationGatewayMap	Map<PaymentMethod, PaymentCreationGateway>	Initialized in constructor	Stores gateways that support payment creation.
4	confirmationGatewayMap	Map<PaymentMethod, PaymentConfirmationGateway>	Initialized in constructor	Stores gateways that support manual confirmation.
5	refundGatewayMap	Map<PaymentMethod, PaymentRefundGateway>	Initialized in constructor	Stores gateways that support refund.

Table 3. Operation design

#	Name	Return type	Description
1	getGateway(method)	PaymentGatewayBase	Returns the gateway for the selected method.

2	getCreationGateway(method)	PaymentCreationGateway	Returns a gateway that supports payment creation.
3	getConfirmationGateway(method)	PaymentConfirmationGateway	Returns a gateway that supports manual confirmation.
4	getRefundGateway(method)	PaymentRefundGateway	Returns a gateway that supports automatic refund.

Parameter:

- method: Selected payment method.

Exception:

- BadRequestException if the payment method is unsupported or does not support the requested operation.

Method:

- The constructor groups gateway adapters by supported capability.
- Public methods retrieve the required gateway type for the payment flow.

4.4.3.5 Class *PaymentCompletionService*



Table 2. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	paymentTransactionService	PaymentTransactionService	Injected by NestJS	Updates payment transaction status.
2	placeOrderPaymentPort	PlaceOrderPaymentPort	Injected by NestJS	Updates the related order after payment success.

Table 3. Operation design

#	Name	Return type	Description
1	completeSuccessfulPayment(input)	Promise<any>	Marks transaction as successful and marks the order as paid.

Parameter:

- input.transactionId: Internal payment transaction ID.
- input.data: Provider transaction data.
- input.fallbackProviderTransactionId: Fallback external transaction ID.

Exception:

- May throw exceptions from PaymentTransactionService or PlaceOrderPaymentPort.

Method:

- Marks the transaction as SUCCESS.
- Cancels other pending transactions of the same order.
- Marks the order as paid and pending processing.

4.4.3.6 Class *VietqrGatewayAdapter*



Table 2. Attribute design

#	Name	Data type	Default value	Description
1	vietqrService	VietqrService	Injected by NestJS	Service used to generate VietQR payment QR codes.

Table 3. Operation design

#	Name	Return type	Description
1	getMethod()	PaymentMethod	Returns VIETQR.
2	createPayment(context)	Promise<PaymentGatewayResult>	Generates a VietQR payment QR code and provider data.

Parameter:

- context.gatewayOrderId: Gateway order ID used in QR content.
- context.invoiceId: Invoice ID.
- context.amount: Payment amount.
- context.customerEmail: Customer email, if needed by gateway context.

Exception:

- Exceptions from VietqrService.generateQrCode() are propagated.

Method:

- Builds normalized QR content from AIMS {gatewayOrderId}.
- Calls VietqrService.generateQrCode().
- Returns QR code, QR link, QR content, expiry time, and transaction update data.

4.4.3.7 Class *VietqrService*

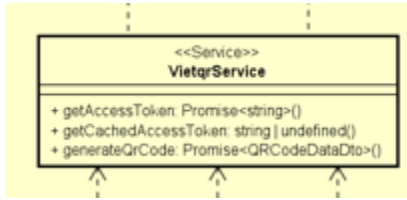


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	vietqrClient	VietqrClient	Injected by NestJS	Client used to call VietQR HTTP APIs.
2	tokenService	VietqrTokenService	Injected by NestJS	Manages VietQR access token retrieval and retry.
3	qrRequestBuilder	VietqrQrRequestBuilder	Injected by NestJS	Builds VietQR QR generation request payload.

Table 3. Operation design

#	Name	Return type	Description
1	getAccessToken()	Promise<string>	Returns a VietQR access token.
2	getCachedAccessToken()	string undefined	Returns the currently cached token, if any.
3	generateQrCode(request)	Promise<QRCodeDataDto>	Generates a VietQR QR code for a payment request.

Parameter:

- request: Contains order ID, invoice ID, amount, description, return URL, and cancel URL.

Exception:

- BadRequestException if amount is invalid.
- UnauthorizedException if VietQR authentication fails.
- BadGatewayException if QR generation fails.

Method:

- Validates amount.
- Builds the VietQR request payload.
- Calls VietQR API with access-token retry support.
- Maps VietQR response to QRCodeDataDto.

4.4.3.8 Class *VietqrTokenService*



Table 2. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	string undefined	undefined	Cached VietQR access token.
2	accessTokenExpiredAt	Date undefined	undefined	Expiration time of the cached token.
3	vietqrClient	VietqrClient	Injected by NestJS	Client used to request tokens from VietQR.
4	configService	VietqrConfigService	Injected by NestJS	Provides VietQR credentials and configuration.

Table 3. Operation design

#	Name	Return type	Description
1	getAccessToken()	Promise<string>	Returns cached token or requests a new one.
2	withAccessTokenRetry(operation)	Promise<T>	Executes an operation with token retry if authentication fails.
3	getCachedAccessToken()	string undefined	Returns the cached token.

Parameter:

- operation: Function that receives an access token and performs a VietQR API operation.

Exception:

- UnauthorizedException if token retrieval fails.

Method:

- Reuses cached token if still valid.
- Clears token and retries once when the VietQR API returns authentication errors.

4.4.3.9 Class *VietqrCallbackService*



Table 2. Attribute design

#	Name	Data type	Default value	Description

1	prisma	PrismaService	Injected by NestJS	Used to find and update payment transactions.
2	configService	VietqrConfigService	Injected by NestJS	Provides expected VietQR transaction type.
3	responseMapper	VietqrCallbackResponseMapper	Injected by NestJS	Maps callback data to response DTO.
4	paymentCompletionService	PaymentCompletionService	Injected by NestJS	Completes successful payment after valid callback.

Table 3. Operation design

#	Name	Return type	Description
1	confirmTransactionFromCallback(callback)	Promise<{ status, refTransactionId, duplicate }>	Confirms a VietQR payment from bank callback data.

Parameter:

- callback: VietQR transaction sync data including transaction ID, amount, content, reference number, transaction type, and transaction time.

Exception:

- BadRequestException if transaction type or amount does not match.
- NotFoundException if no matching payment transaction is found.

Method:

- Validates transaction type.

- Checks duplicate callback by transaction ID or reference number.
- Finds matching VietQR transaction by order ID or QR content.
- Completes payment using PaymentCompletionService.

4.4.3.10 Class VietqrController



Table 2. Attribute design

#	Name	Data type	Default value	Description
1	vietqrCallbackService	VietqrCallbackService	Injected by NestJS	Handles VietQR callback confirmation.
2	responseMapper	VietqrCallbackResponseMapper	Injected by NestJS	Builds VietQR callback response format.
3	sandboxService	VietqrSandboxService	Injected by NestJS	Supports VietQR test callback flow.

Table 3. Operation design

#	Name	Return type	Description
1	syncTransaction(transactionSyncDto)	Promise<any>	Receives VietQR bank callback and confirms payment.
2	testCallback(vietqrTestCallbackDto)	Promise<any>	Sends test callback request to VietQR sandbox.

Parameter:

- transactionSyncDto: VietQR bank callback payload.
- vietqrTestCallbackDto: Sandbox callback test payload.

Exception:

- Exceptions are caught in syncTransaction() and mapped to VietQR error response.
- testCallback() propagates exceptions from VietqrSandboxService.

Method:

- syncTransaction() is protected by VietqrCallbackAuthGuard.
- It delegates confirmation to VietqrCallbackService and returns success/error response using mapper.

4.4.3.11 Class VietqrHttpClient



Table 2. Attribute design

#	Name	Data type	Default value	Description
1	configService	VietqrConfigService	Injected by NestJS	Provides VietQR base URL and configuration.

Table 3. Operation design

#	Name	Return type	Description
1	getAccessToken(username, password)	Promise<VietqrAccessTokenResponse>	Requests VietQR access token using Basic

			Authentication
2	generateQRCode(accessToken, request)	Promise<VietqrGenerateQRCodeResponse>	Calls VietQR QR generation API.
3	testCallback(accessToken, request)	Promise<{ status?: string; message?: string }>	Calls VietQR sandbox test callback API.

Parameter:

- username: VietQR username.
- password: VietQR password.
- accessToken: Bearer token for VietQR API.
- request: VietQR API request body.

Exception:

- Axios errors may be thrown when VietQR API requests fail.

Method:

- This class is a low-level HTTP client and does not contain business logic.

4.4.3.12 Class *VietqrConfigService*

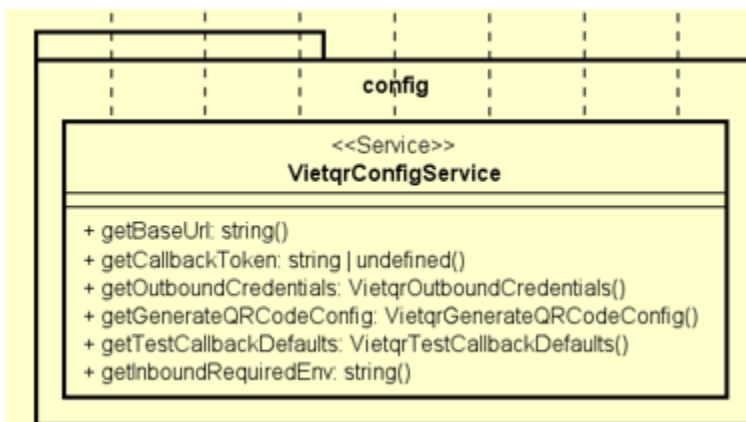


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	None	-	-	This service reads configuration from environment variables and does not store mutable state.

Table 3. Operation design

#	Name	Return type	Description
1	get(key)	string undefined	Reads an environment variable.
2	getBaseUrl()	string	Returns VietQR base URL.
3	getReturnUrl()	string undefined	Returns VietQR return URL.
4	getCallbackToken()	string undefined	Returns static callback token.
5	getOutboundCredentials()	VietqrOutboundCredentials	Returns VietQR username and password.
6	getGenerateQRCodeConfig()	VietqrGenerateQRCodeConfig	Returns validated QR generation config.
7	getTestCallbackDefaults()	VietqrTestCallbackDefaults	Returns default sandbox callback fields.
8	getTransType()	string	Returns expected transaction type.
9	getInboundRequiredEnv(key)	string	Returns required inbound config value.

Parameter:

- key: Environment variable name.

Exception:

- UnauthorizedException if outbound credentials are missing.
- BadRequestException if payment configuration is missing or invalid.
- InternalServerErrorException if required inbound config is missing.

Method:

- Reads environment variables.
- Validates bank code, bank account, user bank name, QR type, and transaction type.

4.4.3.13 Class VietqrQrRequestBuilder

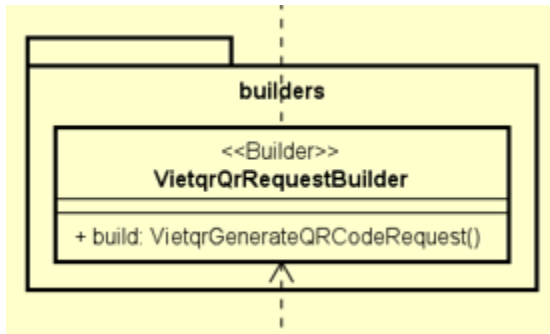


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	configService	VietqrConfigService	Injected by NestJS	Provides QR generation configuration.

Table 3. Operation design

#	Name	Return type	Description
1	build(request)	VietqrGenerateQRCodeRequest	Builds the request payload required by VietQR QR generation API.

Parameter:

- request: Payment QR request containing amount, order ID, description, and return URL.

Exception:

- May throw configuration-related exceptions from VietqrConfigService.

Method:

- Normalizes QR content.
- Combines payment request data with VietQR environment configuration.

4.4.3.14 Class *VietqrInboundService*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	configService	VietqrConfigService	Injected by NestJS	Provides inbound authentication configuration.

Table 3. Operation design

#	Name	Return type	Description
1	generateAccessToken(authorization?)	{ access_token, token_type, expires_in }	Generates a signed inbound callback token after Basic Authentication.
2	validateAccessToken(token)	void	Validates callback bearer token signature, subject, and expiration.
3	parseBasicAuth(header?)	BasicAuthCredentials	Parses Basic Authentication header.

4	validateInboundCredentials(username, password)	void	Validates inbound username and password.
---	--	------	--

Parameter:

- authorization: Basic Authentication header.
- token: Bearer token sent by callback client.
- username: Inbound username.
- password: Inbound password.

Exception:

- UnauthorizedException if authentication header, credentials, token signature, subject, or expiration is invalid.
- InternalServerErrorException if token expiration configuration is invalid.

Method:

- Uses HMAC SHA-256 to sign and verify inbound access tokens.

4.4.3.15 Class VietqrSandboxService

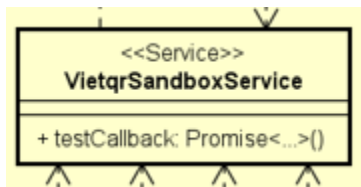


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	vietqrClient	VietqrClient	Injected by NestJS	Sends test callback requests to

				VietQR sandbox.
2	configService	VietqrConfigService	Inject ed by NestJS	Provides default sandbox values.
3	prisma	PrismaService	Inject ed by NestJS	Checks existing pending VietQR transaction.
4	stalePaymentTransactionCleanup	StalePaymentTransactionCleanup Service	Inject ed by NestJS	Expires stale pending transactions before testing.
5	tokenService	VietqrTokenService	Inject ed by NestJS	Provides VietQR access token with retry.

Table 3. Operation design

#	Name	Return type	Description
1	testCallback(dto)	Promise<any>	Sends a simulated VietQR callback request for testing.

Parameter:

- dto: Contains test callback amount, content, order ID, bank account, bank code, and transaction type.

Exception:

- BadRequestException if no matching pending VietQR payment exists.
- UnauthorizedException if VietQR authentication fails.
- BadGatewayException if sandbox callback request fails.

Method:

- Ensures a matching VietQR transaction is still pending.
- Builds callback request using DTO values and default configuration.
- Calls VietQR sandbox callback API.

4.4.3.16 Class *VietQrPaymentFlowService*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paymentService	PaymentService	Injected by Angular	Calls backend payment APIs.
2	paymentStatusApi	PaymentStatusApiService	Injected by Angular	Reads latest transaction status from backend.

Table 3. Operation design

#	Name	Return type	Description
1	createPayment(input)	Observable<VietQrPaymentSnapshot>	Creates a VietQR payment and maps result to page snapshot.

2	reuseExistingPayment(context)	Observable<VietQrPaymentSnapshot>	Reuses an existing VietQR transaction if still available.
3	pollStatus(orderId)	Observable<PaymentTransactionStatus>	Polls payment transaction status.
4	isSuccessfulStatus(status)	boolean	Checks whether transaction status is successful.

Parameter:

- input: Order and payment data required to request VietQR payment.
- context: Existing order/payment context.
- orderId: Public order ID.
- status: Payment transaction status object.

Exception:

- Errors are returned through the Observable error channel.

Method:

- Uses backend payment APIs to create QR payment and check whether callback has marked the transaction successful.

4.4.3.17 Class *VietQrPaymentPageFlowService*

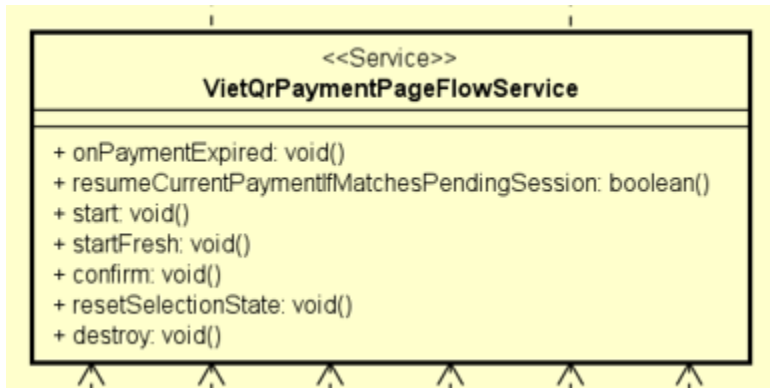


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paymentFlow	VietQrPaymentFlowService	Injected by Angular	Handles VietQR creation and status polling.
2	pendingPaymentSession	PendingPaymentSessionService	Injected by Angular	Provides pending payment session data.
3	stateStore	PaymentPageStateStore	Injected by Angular	Stores payment page UI state.

Table 3. Operation design

#	Name	Return type	Description
1	start(hasCheckoutDraft)	void	Starts VietQR payment creation flow.

2	confirm()	void	Checks whether VietQR payment has been completed.
---	-----------	------	---

Parameter:

- hasCheckoutDraft: Indicates whether checkout draft data exists.

Exception:

- Does not throw directly; errors are reflected in payment page state.

Method:

- Creates VietQR payment when session is ready.
- Updates QR code, loading state, status message, and error message in the state store.
- Confirms payment by polling backend transaction status.

4.4.3.18 Class *VietQrPaymentViewComponent*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	qrCode	string	"	Raw QR code data.
2	qrImageSrc	string	"	QR image source displayed to the customer.
3	qrContent	string	"	Payment transfer content shown to the customer.

Table 3. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

1	None	-	This component renders the VietQR payment UI.
---	------	---	---

Parameter:

- No method parameters.

Exception:

- No direct exceptions.

Method:

- Displays QR image and payment content for bank transfer.

4.4.3.19 Class *PaymentPageFacade*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	route	ActivatedRoute	Injected by Angular	Reads query parameters such as PayPal token or cancellation flag.
2	router	Router	Injected by Angular	Navigates between payment and result pages.
3	stateStore	PaymentPageStateStore	Injected by Angular	Central UI state store for payment page.
4	vietQrPageFlow	VietQrPaymentPageFlowService	Injected by Angular	Handles VietQR page flow.

5	paypalPageFlow	PaypalPaymentPageFlowService	Injected by Angular	Handles PayPal page flow.
---	----------------	------------------------------	---------------------	---------------------------

Table 3. Operation design

#	Name	Return type	Description
1	initialize()	void	Initializes payment page and handles query parameters.
2	confirmPayment()	void	Delegates confirmation to the selected payment method flow.
3	selectPaymentMethod(method)	void	Changes selected payment method and starts the corresponding flow.
4	cancelPayment()	void	Cancels or exits the payment flow.

Parameter:

- method: Selected payment method.

Exception:

- Does not throw directly; errors are handled by child flows and state store.

Method:

- Acts as the frontend coordinator between UI components and payment method-specific flows.

4.4.3.20 Class *PaypalService*

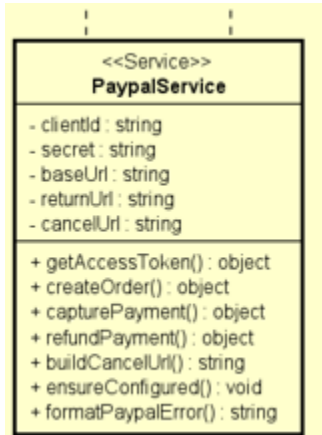


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	clientId	string undefined	process.env.PAYPAL_CLIENT_ID	The PayPal client ID used for API authentication.
2	secret	string undefined	process.env.PAYPAL_SECRET	The PayPal secret key used with the client ID to request an access token.
3	baseUrl	string undefined	process.env.PAYPAL_BASE_URL	The base URL of the PayPal API.
4	returnUrl	string	PAYPAL_RETURN_URL or http://localhost:4200/payment	The URL PayPal redirects to after the customer approves the payment.
5	cancelUrl	string	PAYPAL_CANCEL_URL or http://localhost:4200/payment	The URL PayPal redirects to when the customer cancels the payment.

Table 3. Operation design

#	Name	Return type	Description
1	getAccessToken()	Promise<string>	Requests an OAuth access token from PayPal.
2	createOrder(amount)	Promise<any>	Creates a PayPal checkout order with the given USD amount.
3	capturePayment(orderId)	Promise<any>	Captures an approved PayPal order.
4	refundPayment(captureId)	Promise<any>	Refunds a successful PayPal capture.
5	buildCancelUrl()	string	Builds the cancellation redirect URL with paypalCancel=1.
6	ensureConfigured()	void	Validates that required PayPal environment variables are configured.
7	formatPaypalError(err)	string	Extracts a readable error message from a PayPal or Axios error response.

Parameter:

- amount: No default value. The payment amount in USD. It must be greater than 0.
- orderId: No default value. The PayPal order ID to be captured.
- captureId: No default value. The PayPal capture ID to be refunded.
- err: No default value. The error object returned by PayPal or Axios.

Exception:

- UnauthorizedException if PayPal credentials are missing or the access token request fails.
- BadRequestException if the amount is invalid, order creation fails, capture fails, or refund fails.

Method:

- createOrder(amount) first calls getAccessToken(), then sends POST /v2/checkout/orders to PayPal with intent = CAPTURE.
- capturePayment(orderId) sends POST /v2/checkout/orders/{orderId}/capture.
- refundPayment(captureId) sends POST /v2/payments/captures/{captureId}/refund.

4.4.3.21 Class PaypalGatewayAdapter

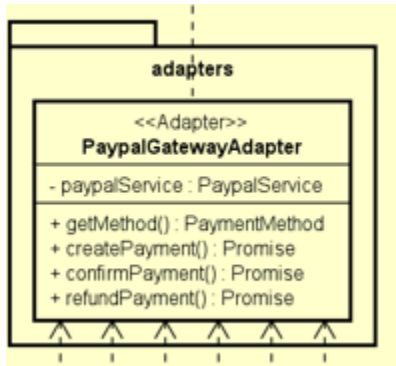


Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paypalService	PaypalService	Injected by NestJS	The service responsible for communicating with the PayPal API.

Table 3. Operation design

#	Name	Return type	Description
1	getMethod()	PaymentMethod	Returns the supported payment method, which is PAYPAL.

2	<code>createPayment(context)</code>	<code>Promise<PaymentGatewayResult></code>	Creates a PayPal order and returns the approval URL to the client.
3	<code>confirmPayment(transactionRef)</code>	<code>Promise<PaymentGatewayResult></code>	Captures the PayPal payment using the PayPal order ID.
4	<code>refundPayment(transactionRef)</code>	<code>Promise<PaymentGatewayResult></code>	Refunds the PayPal payment using the PayPal capture ID.

Parameter:

- `context`: No default value. Contains payment information such as amount, order ID, and invoice ID.
- `transactionRef`: No default value. For confirmation, this is the PayPal order ID. For refund, this is the PayPal capture ID.

Exception:

- `BadRequestException` if the PayPal order response does not contain an order ID or approval link.
- Exceptions from `PaypalService` are propagated if the PayPal API request fails.

Method:

- `createPayment(context)` converts the amount to USD, calls `paypalService.createOrder()`, finds the approval link, and returns `paymentUrl`, `providerData`, and `gatewayOrderId`.
- `confirmPayment(transactionRef)` calls PayPal capture and extracts the `captureId` from the response for future refund operations.

4.4.3.22 Class *PaymentGatewayTransactionRefResolver*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	None	-	-	This class does not maintain internal state. It only resolves the transaction reference used by the payment gateway.

Table 3. Operation design

#	Name	Return type	Description
1	resolve(transaction)	string	Resolves the transaction reference used when calling the payment gateway. For PayPal, it must return the real PayPal order ID.

Parameter:

- transaction.id: No default value. The internal transaction ID.
- transaction.paymentMethod: No default value. The payment method of the transaction.
- transaction.gatewayOrderId: No default value. The gateway order ID stored in the transaction.

Exception:

- BadRequestException if a PayPal transaction does not have gatewayOrderId.
- BadRequestException if the PayPal gatewayOrderId is still a local temporary ID instead of a real PayPal order ID.

Method:

- If the payment method is not PayPal, return gatewayOrderId or transaction.id.
- If the payment method is PayPal, validate that gatewayOrderId exists and is not the local temporary gateway order ID, then return it.

4.4.3.23 Class *PaymentGatewayOrderIdService*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paymentTransactionService	PaymentTransactionService	Injected by NestJS	The service used to update transaction data in the database.

Table 3. Operation design

#	Name	Return type	Description
1	ensureForCreatePayment(paymentMethod, transaction)	Promise<string>	Ensures that the transaction has a proper gateway order ID before creating a payment.

Parameter:

- paymentMethod: No default value. The selected payment method, such as PAYPAL or VIETQR.
- transaction: No default value. Contains id and gatewayOrderId.

Exception:

- May throw an exception if paymentTransactionService.updateGatewayOrderId() fails.

Method:

- For VietQR, it creates a local gateway order ID and stores it in the database.
- For PayPal, it returns the existing gatewayOrderId or a temporary local ID. The real PayPal order ID is updated later after PayPal creates the order successfully.

4.4.3.24 Class PaypalPaymentFlowService

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paymentService	PaymentService	Injected by Angular	The service used to call backend payment APIs.
2	paymentSessionStorage	PaymentSessionStorageService	Injected by Angular	The service used to save, load, and clear the PayPal session on the client side.

Table 3. Operation design

#	Name	Return type	Description
1	createOrReusePayment(pendingPaymentSession)	Observable<PlaceOrderPaymentResult>	Creates or reuses a PayPal payment for the current order.
2	capture(paymentSession)	Observable<PaymentResultDto>	Confirms the PayPal payment after the customer approves it.

3	saveSession(orderId, invoiceId)	void	Saves the PayPal session to client-side storage.
4	loadSession()	PaypalPaymentSession null	Loads the saved PayPal session.
5	clearSession()	void	Clears the saved PayPal session.

Parameter:

- pendingPaymentSession: No default value. Contains orderId, invoiceId, totalAmount, and customerEmail.
- paymentSession: No default value. Contains orderId and invoiceId.
- orderId: No default value. The order ID.
- invoiceId: No default value. The invoice ID.

Exception:

- This class does not throw exceptions directly. API errors are returned through the Observable error channel.

Method:

- createOrReusePayment() calls requestPayment() with paymentMethod = PAYPAL.
- capture() calls confirmTransaction() with transaction content PayPal capture.

4.4.3.25 Class *PaypalPaymentPageFlowService*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paypalPaymentSession	PaypalPaymentSession null	null	The current PayPal payment session on the payment page.
2	paymentExpiredHandler	() => void	undefined	Callback function used when the PayPal payment expires.
3	isHandlingExpiredPayment	boolean	false	Prevents duplicated handling of an expired PayPal payment.

Table 3. Operation design

#	Name	Return type	Description
1	onPaymentExpired(handler)	void	Registers a callback for expired PayPal payments.
2	start(hasCheckoutDraft)	void	Starts the PayPal payment creation flow.
3	handleReturn()	void	Handles the browser redirect from PayPal back to the application.
4	discardApprovalSession()	void	Clears the current PayPal approval session.
5	confirm()	void	Captures payment, navigates to result page, or redirects to PayPal depending on the current state.

Parameter:

- handler: No default value. Callback executed when the PayPal payment expires.
- hasCheckoutDraft: No default value. Indicates whether a checkout draft exists.

Exception:

- This class does not throw exceptions directly. Errors are handled by updating the page state through `stateStore.patch()`.

Method:

- `start()` sets the page to loading state, ensures a pending payment session exists, then calls `PaypalPaymentFlowService.createOrReusePayment()`.
- `handleReturn()` loads the saved PayPal session, marks the payment as approved, and starts capture.
- `confirm()` captures the payment if it is approved, navigates to `/order-result` if it is already confirmed, or redirects the user to the PayPal approval URL.

4.4.3.26 Class *PaypalPaymentViewComponent*

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	paypalRedirect Url	string	""	The PayPal approval URL used to redirect the customer.
2	paypalApproved	boolean	false	Indicates whether the customer has approved the payment on PayPal.
3	paypalConfirmed	boolean	false	Indicates whether the system has captured and confirmed the payment successfully.
4	isLoading	boolean	false	Indicates whether the PayPal checkout or confirmation process is loading.

Table 3. Operation design

#	Name	Return type	Description
1	None	-	This component does not contain business operations. It only receives input values and renders the PayPal payment UI.

Method:

- If the payment is not approved or confirmed, the component displays a message telling the customer they will be redirected to PayPal.
- If paypalRedirectUrl exists, it displays the Pay with PayPal button.
- If paypalApproved is true, it displays the payment finalization state.
- If paypalConfirmed is true, it displays the successful payment state.

4.4.3.27 Class “PlaceOrderController”

<<boundary>> PlaceOrderController	
- placeOrderService : PlaceOrderBeService	
+ checkStock(dto : PlaceOrderRequestDto) : Promise<StockCheckResultDto> + submitDeliveryInfo(dto : SubmitDeliveryInfoDto) : Promise<InvoicePreviewDto> + confirmOrder(dto : ConfirmOrderDto) : Promise<OrderSuccessDto> + createPayment(dto : PlaceOrderPaymentRequestDto) : Promise<PlaceOrderPaymentResultDto>	

Table. Attribute design of PlaceOrderController

#	Name	Data type	Default value	Description
1	placeOrderService	PlaceOrderBeService	(injected)	Application facade the controller delegates every request to.

Table. Operation design of PlaceOrderController

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	checkStock	Promise<StockCheckResultDto>	REST POST /place-order/check-stock – checks cart availability.
2	submitDeliveryInfo	Promise<InvoicePreviewDto>	POST /place-order/delivery-info – validates delivery info and returns priced invoice.
3	confirmOrder	Promise<OrderSuccessDto>	POST /place-order/confirm – confirms a paid order.
4	createPayment	Promise<PlaceOrderPaymentResultDto>	POST /place-order/payment – creates a pending order and requests payment.

Parameter:

- checkStock(placeOrderRequestDto: PlaceOrderRequestDto) – cart items to verify.
- submitDeliveryInfo(submitDeliveryInfoDto: SubmitDeliveryInfoDto) – items + delivery info.
- confirmOrder(confirmOrderDto: ConfirmOrderDto) – items + delivery + payment transaction.
- createPayment(placeOrderPaymentRequestDto: PlaceOrderPaymentRequestDto) – items + delivery + payment method.

Method

Thin REST adapter. Each handler simply forwards the validated DTO to PlaceOrderBeService and returns its result; it contains no business logic.

4.4.3.28 Class “PlaceOrderBeService”

<<control>> PlaceOrderBeService	
- repository : IOrderRepository - validation : OrderValidationService - stockChecker : StockCheckerService - invoiceCalculator : InvoiceCalculator - orderFactory : OrderFactory - eventPublisher : IOrderEventPublisher - paymentService : PaymentService [0..1]	
+ checkStock(dto : PlaceOrderRequestDto) : Promise<StockCheckResultDto> + processDeliveryInfo(dto : SubmitDeliveryInfoDto) : Promise<InvoicePreviewDto> + createPayment(dto : PlaceOrderPaymentRequestDto) : Promise<PlaceOrderPaymentResultDto> + confirmOrder(dto : ConfirmOrderDto) : Promise<OrderSuccessDto> - priceOrder(items : CartItemDto[], deliveryInfo : DeliveryInfoDto) : Promise<InvoicePreviewDto> - ensureSufficientStock(items : CartItemDto[], products : ProductSnapshot[]) : void - buildSuccessFromExistingTransaction(tx : PaymentTransactionSnapshot) : Promise<OrderSuccessDto>	

Table. Attribute design of PlaceOrderBeService

#	Name	Data type	Default value	Description
1	repository	IOrderRepository	(injected)	Persistence port (PrismaOrderRepository)
2	validation	OrderValidationService	(injected)	Validates cart items and delivery/payment info.
3	stockChecker	StockCheckerService	(injected)	Computes stock sufficiency.
4	invoiceCalculator	InvoiceCalculator	(injected)	Builds priced invoice (VAT + shipping).
5	orderFactory	OrderFactory	(injected)	Assembles the order-create input.

6	eventPublisher	IOrderEventPublisher	(injected)	Publishes the order-confirmed event (email).
7	paymentService	PaymentService	undefined	Optional payment gateway (VietQR/PayPal).

Table. Operation design of PlaceOrderBeService

#	Name	Return type	Description (purpose)
1	checkStock	Promise<StockCheckResultDto>	Normalize items, load products, run stock check.
2	processDeliveryInfo	Promise<InvoicePreviewDto>	Validate delivery info, then price the invoice.
3	createPayment	Promise<PlaceOrderPaymentResultDto>	Price order, persist PENDING_PAYMENT order, request payment.
4	confirmOrder	Promise<OrderSuccessDto>	Validate, idempotency check, persist confirmed order, email customer.
5	priceOrder	Promise<InvoicePreviewDto>	[private] Shared pipeline: normalize → load products →

			ensure stock → build invoice.
6	ensureSufficientStock	void	[private] Throw if any item is short of stock.
7	buildSuccessFromExistingTransaction	Promise<OrderSuccessDto>	[private] Idempotency: map an already-stored transaction to a success DTO.

Parameter:

- priceOrder(items: CartItemDto[], deliveryInfo: DeliveryInfoDto)
- ensureSufficientStock(items, products: ProductSnapshot[])

Exception:

- BadRequestException – if paymentService is not configured (createPayment).
- InvalidQuantityException – if requested quantity exceeds available stock.
- InvalidDeliveryInfoException – if delivery info is invalid.
- PaymentNotSuccessfulException – if payment info is missing on confirm.

Method

Pure orchestrator (Facade). It only coordinates collaborators – no DB access, no VAT/shipping math, no email logic of its own. priceOrder() is reused by processDeliveryInfo / createPayment / confirmOrder (DRY).

4.4.3.29 Class “OrderFactory”

<<factory>> OrderFactory	
+ buildCreateOrderInput(invoice : InvoicePreviewDto, deliveryInfo : DeliveryInfoDto, status : string) : CreateOrderInput	
- generateOrderCode() : string	

Table. Attribute design of OrderFactory

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of OrderFactory

#	Name	Return type	Description (purpose)
1	buildCreateOrderInput	CreateOrderInput	Assemble the persistence input for a new order from a priced invoice + delivery info.
2	generateOrderCode	string	[private] Produce a unique order code (PO- {timestamp} - {random}).

Parameter:

- buildCreateOrderInput(invoice: InvoicePreviewDto, deliveryInfo: DeliveryInfoDto, status: string)

Method

Trims user-entered fields, sets STANDARD delivery method, copies invoice totals, and maps invoice items to order lines. Removes order-building code that used to be duplicated in createPayment() and confirmOrder().

4.4.3.30 Class “OrderSuccessMapper”

<<factory>> OrderSuccessMapper	
+ fromDeliveryInfo(deliveryInfo : DeliveryInfoDto, totalAmount : number, tx : OrderSuccessTransaction) : OrderSuccessDto + fromPersistedOrder(order : PersistedOrderDetail, totalAmount : number, tx : OrderSuccessTransaction) : OrderSuccessDto	

Table. Attribute design of OrderSuccessMapper

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of OrderSuccessMapper

#	Name	Return type	Description (purpose)
1	fromDeliveryInfo	OrderSuccessDto	[static] Build the success DTO from raw form input (trims user fields).
2	fromPersistedOrder	OrderSuccessDto	[static] Build the success DTO from an already-stored order.

Parameter:

- fromDeliveryInfo(deliveryInfo: DeliveryInfoDto, totalAmount: number, transaction: OrderSuccessTransaction)
- fromPersistedOrder(order: PersistedOrderDetail, totalAmount: number, transaction: OrderSuccessTransaction)

Method

Single source of truth for assembling OrderSuccessDto. Two entry points because the customer fields come from two sources (raw form vs. stored order).

4.4.3.31 Class “InvoiceCalculator”

<<service>> InvoiceCalculator	
- shippingCalculator : IShippingFeeCalculator	
+ build(items : CartItemDto[], products : ProductSnapshot[], deliveryInfo : DeliveryInfoDto) : InvoicePreviewDto	
+ buildFromPersistedOrder(order : PersistedOrderDetail) : InvoicePreviewDto	
- toKg(weightGrams : number) : number	

Table. Attribute design of InvoiceCalculator

#	Name	Data type	Default value	Description
1	shippingCalculator	IShippingFeeCalculator	(injected)	Strategy used to compute the delivery fee.

Table. Operation design of InvoiceCalculator

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	build	InvoicePreviewDto	Compute line amounts, VAT, shipping fee and total for a fresh order.
2	buildFromPersistedOrder	InvoicePreviewDto	Rebuild the invoice preview from a stored order (for the confirmation email / idempotent flow).
3	toKg	number	[private] Convert grams to kilograms (guards against invalid weight).

Parameter:

- build(items: CartItemDto[], products: ProductSnapshot[], deliveryInfo: DeliveryInfoDto)
- buildFromPersistedOrder(order: PersistedOrderDetail)
- toKg(weightGrams: number)

Method

subtotalBeforeVat = sum(price*qty); vatAmount = subtotal * VAT_RATE (0.1); delegates shipping to the IShippingFeeCalculator strategy; totalAmount = subtotalAfterVat + deliveryFee.

4.4.3.32 Class “StockCheckerService”

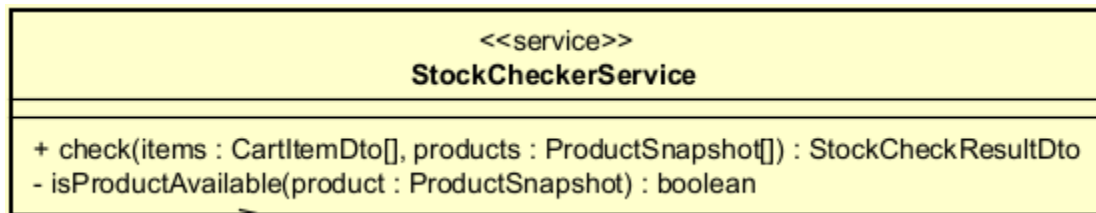


Table. Attribute design of StockCheckerService

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of StockCheckerService

#	Name	Return type	Description (purpose)
1	check	StockCheckResultDto	For each item, compute availability and collect items that lack stock.
2	isProductAvailable	boolean	[private] False when the product status is in the UNAVAILABLE set.

Parameter:

- check(items: CartItemDto[], products: ProductSnapshot[])
- isProductAvailable(product: ProductSnapshot)

Method

A product missing from the catalog or with an unavailable status (DEACTIVATED/DELETED/INACTIVE/UNAVAILABLE) counts as 0 available.
Returns sufficient=true only when no item is short.

4.4.3.33 Class “OrderValidationService”

<<service>> OrderValidationService	
- deliveryValidator : IDeliveryInfoValidator	
+ normalizeItems(items : CartItemDto[]) : CartItemDto[] + validateDeliveryInfo(deliveryInfo : DeliveryInfoDto) : void + validateDeliveryInfoForOrder(deliveryInfo : DeliveryInfoDto) : void + validatePaymentInfo(dto : ConfirmOrderDto) : void + getRequiredEmail(deliveryInfo : DeliveryInfoDto) : string - collectDeliveryErrors(deliveryInfo : DeliveryInfoDto, requireEmail : boolean) : string[] - assertNoErrors(errors : string[]) : void	

Table. Attribute design of OrderValidationService

#	Name	Data type	Default value	Description

1	deliveryValidator	IDeliveryInfoValidator	(injected)	Rule-based delivery-info validator (behind a DIP token).
---	-------------------	------------------------	------------	--

Table. Operation design of OrderValidationService

#	Name	Return type	Description (purpose)
1	normalizeItems	CartItemDto[]	Validate and merge cart items by productId.
2	validateDeliveryInfo	void	Validate delivery info WITHOUT requiring email.
3	validateDeliveryInfoForOrder	void	Validate delivery info AND require a non-empty email.
4	validatePaymentInfo	void	Require a non-empty transactionId and paymentMethod.
5	getRequiredEmail	string	Return the trimmed email or throw if missing.
6	collectDeliveryErrors	string[]	[private] Run the validator and optionally add the email-required error.
7	assertNoErrors	void	[private] Throw InvalidDeliveryInfoException when there are errors.

Parameter:

- normalizeItems(items: CartItemDto[])
- validateDeliveryInfo(deliveryInfo: DeliveryInfoDto)
- validatePaymentInfo(dto: ConfirmOrderDto)

Exception:

- BadRequestException – empty items / non-positive productId or quantity (normalizeItems).
- InvalidDeliveryInfoException – delivery info invalid or email missing.
- PaymentNotSuccessfulException – transactionId or paymentMethod missing.

Method

normalizeItems sums quantities of duplicate productIds. Delivery validation is delegated to IDeliveryInfoValidator; email requirement is enforced only for order-creating flows.

4.4.3.34 Class “OrderEventPublisher”

<<observer>> OrderEventPublisher	
- observers : INotificationObserver[1..*]	
+ publish(event : OrderConfirmedEvent) : Promise<void>	

Table. Attribute design of OrderEventPublisher

#	Name	Data type	Default value	Description
1	observers	INotificationObserver[]	(injected)	Registered notification channels.

Table. Operation design of OrderEventPublisher

#	Name	Return type	Description (purpose)
1	publish	Promise<void>	Notify every observer about an order-confirmed event.

Parameter:

- publish(event: OrderConfirmedEvent)

Method

Calls `observer.update()` for all observers in parallel and isolates failures (a failing observer is logged and does not break the others). Adding a channel (SMS/Zalo/Push) = register a new observer (OCP).

4.4.3.35 Class “BaseOrderState”

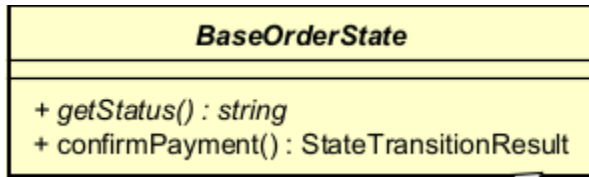


Table. Attribute design of BaseOrderState

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of BaseOrderState

#	Name	Return type	Description (purpose)
1	getStatus	string	[abstract] The status string this state represents.
2	confirmPayment	StateTransitionResult	Default behaviour: illegal – throws IllegalOrderTransitionError.

Exception:

- IllegalOrderTransitionError – confirming payment from a non-payable state.

Method

Template for concrete order states; subclasses override `confirmPayment()` to allow the transition.

4.4.3.36 Class “PendingPaymentState”

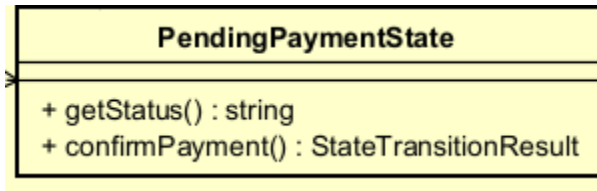


Table. Operation design of PendingPaymentState

#	Name	Return type	Description (purpose)
1	getStatus	string	Returns PENDING_PAYMENT.
2	confirmPayment	StateTransitionResult	Returns { alreadyInTarget: false } – transition to PENDING_PROCESSING is allowed.

State

Order created and awaiting payment.

4.4.3.37 Class "PendingProcessingState"

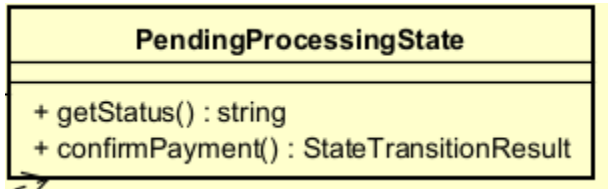


Table. Attribute design of PendingProcessingState

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of PendingProcessingState

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	getStatus	string	Returns PENDING_PROCESSING.
2	confirmPayment	StateTransitionResult	Returns { alreadyInTarget: true } - already paid, the transition is a no-op (idempotent).

State

Order already paid, awaiting product-manager processing.

4.4.3.38 Class "OrderStateFactory"

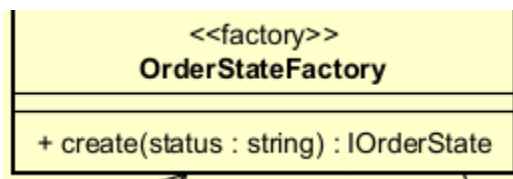


Table. Attribute design of OrderStateFactory

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of OrderStateFactory

#	Name	Return type	Description (purpose)
1	create	IOrderState	Map a status string to the matching state object.

Parameter:

- create(status: string)

Exception:

- IllegalOrderTransitionError - unknown / non-payable status.

Method

switch(status): PENDING_PAYMENT -> PendingPaymentState;
PENDING_PROCESSING -> PendingProcessingState; otherwise throw.

4.4.3.39 Class "DeliveryInfoValidator"

DeliveryInfoValidator	
- rules	: IDeliveryInfoRule[1..*]
+ validate	(info : DeliveryInfoValidationInput) : ValidationResult

Table. Attribute design of DeliveryInfoValidator

#	Name	Data type	Default value	Description
1	rules	IDeliveryInfoRule[]	DEFAULT_DELIVERY_INFO_RULES	Ordered set of validation rules.

Table. Operation design of DeliveryInfoValidator

#	Name	Return type	Description (purpose)
1	validate	ValidationResult	Run every rule and collect the non-null error messages.

Parameter:

- validate(info: DeliveryInfoValidationInput)

Method

Aggregates a collection of IDeliveryInfoRule. Order of rules = order errors are reported.
Returns { valid, errors }.

4.4.3.40 Class "ReceiverNameRule"

ReceiverNameRule
+ validate(info : DeliveryInfoValidationInput) : string

Table. Attribute design of ReceiverNameRule

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of ReceiverNameRule

#	Name	Return type	Description (purpose)
1	validate	string null	Receiver name is required and must contain letters only.

Parameter:

· validate(info: DeliveryInfoValidationInput)

Method

Returns an error message when the name is blank or contains non-letter characters, otherwise null.

4.4.3.41 Class "PhoneNumberRule"

PhoneNumberRule
+ validate(info : DeliveryInfoValidationInput) : string

Table. Attribute design of PhoneNumberRule

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of PhoneNumberRule

#	Name	Return type	Description (purpose)
1	validate	string null	Phone is required: exactly 10 digits, a single separator type (".", space or "-") allowed.

Parameter:

- validate(info: DeliveryInfoValidationInput)

Method

Rejects more than one separator type; strips separators and checks for exactly 10 digits.

4.4.3.42 Class "ProvinceRule"

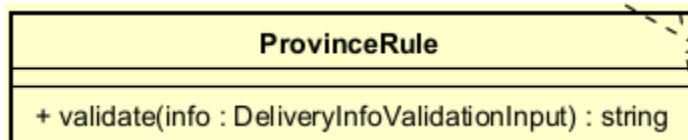


Table. Attribute design of ProvinceRule

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of ProvinceRule

#	Name	Return type	Description (purpose)
1	validate	string null	Province is required (must not be blank).

Parameter:

- validate(info: DeliveryInfoValidationInput)

Method

Returns the "Province is required." message when blank, otherwise null.

4.4.3.43 Class "AddressRule"

AddressRule
+ validate(info : DeliveryInfoValidationInput) : string

Table. Attribute design of AddressRule

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of AddressRule

#	Name	Return type	Description (purpose)
1	validate	string null	Street address is required (must not be blank).

Parameter:

- validate(info: DeliveryInfoValidationInput)

Method

Returns the "Delivery address is required." message when blank, otherwise null.

4.4.3.44 Class "ShippingInstructionsRule"

ShippingInstructionsRule
+ validate(info : DeliveryInfoValidationInput) : string

Table. Attribute design of ShippingInstructionsRule

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of ShippingInstructionsRule

#	Name	Return type	Description (purpose)
1	validate	string null	Optional; if present must not exceed 200 characters.

Parameter:

- validate(info: DeliveryInfoValidationInput)

Method

Null when absent; otherwise rejects values longer than 200 characters.

4.4.3.45 Class "PrismaOrderRepository"

<<repository>> PrismaOrderRepository	
- - prisma : PrismaService	
+ findProductsByIds(productIds : number[]) : Promise<ProductSnapshot[]>	
+ findPaymentTransactionByTransactionId(transactionId : string) : Promise<PaymentTransactionSnapshot>	
+ findOrderDetailByOrderId(orderId : string) : Promise<PersistedOrderDetail>	
+ createPendingPaymentOrder(input : CreateOrderInput) : Promise<CreatedOrderRef>	
+ createConfirmedOrder(input : CreateOrderInput, payment : ConfirmedPaymentInput) : Promise<void>	
+ applyPaidTransition(orderId : string) : Promise<PersistedOrderDetail>	
- insertOrderWithLinesAndInvoice(tx : int, input : CreateOrderInput) : void	
- tryDecrementStock(tx : int, productId : number, quantity : number) : Promise<boolean>	
- decrementStockForOrderCreation(tx : int, productId : number, quantity : number) : Promise<void>	
- decrementStockForPaidCallback(tx : int, productId : number, quantity : number) : Promise<void>	
- toProductSnapshot(product : int) : ProductSnapshot	
- toOrderDetail(order : int) : PersistedOrderDetail	

Table. Attribute design of PrismaOrderRepository

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	prisma	PrismaService	(injected)	Prisma client wrapper - the only DB access point.
---	--------	---------------	------------	---

Table. Operation design of PrismaOrderRepository

#	Name	Return type	Description (purpose)
1	findProductsByIds	Promise<ProductSnapshot[]>	Load products and normalise to ProductSnapshot.
2	findPaymentTransactionByTransactionId	Promise<PaymentTransactionSnapshot null>	Look up an existing payment transaction (idempotency).
3	findOrderDetailByOrderId	Promise<PersistedOrderDetail null>	Load full order detail (lines + invoice).
4	createPendingPaymentOrder	Promise<CreatedOrderRef>	Insert a PENDING_PAYMENT order (no stock decrement).
5	createConfirmedOrder	Promise<void>	Insert an already-paid order, decrement stock and record the payment - atomically.
6	applyPaidTransition	Promise<PersistedOrderDetail>	Move PENDING_PAYMENT -> PENDING_PROCESSING and decrement stock atomically.
7	insertOrderWithLinesAndInvoice	{ orderId, invoiceId }	[private] Insert order + lines + invoice within a transaction.

8	tryDecrementStock	Promise<boolean>	[private] Atomic conditional decrement (guards against overselling).
9	decrementStockForOrderCreation	Promise<void>	[private] Decrement during order create; shortage -> InvalidQuantityException.
10	decrementStockForPaidCallback	Promise<void>	[private] Decrement during paid callback; shortage -> BadRequestException.
11	toProductSnapshot	ProductSnapshot	[private] Map a Prisma product row to a normalised snapshot.
12	toOrderDetail	PersistedOrderDetail	[private] Map a Prisma order row (+relations) to a normalised read-model.

Parameter:

- findProductsByIds(productIds: number[])
- createConfirmedOrder(input: CreateOrderInput, payment: ConfirmedPaymentInput)
- applyPaidTransition(orderId: string)

Exception:

- InvalidQuantityException - insufficient stock during order creation (customer-facing).
- BadRequestException - insufficient stock during paid callback, or missing order context.

Method

The only class allowed to touch Prisma. Normalises Decimal/BigInt to plain number/string. applyPaidTransition uses updateMany guarded by status to atomically claim the PENDING_PAYMENT -> PENDING_PROCESSING transition, so duplicate/concurrent payment callbacks never decrement stock twice (idempotent under races).

4.4.3.46 Class "WeightBasedShippingStrategy"

<<strategy>> WeightBasedShippingStrategy	
- config : ShippingFeeConfig	
+ calculate(context : ShippingContext) : ShippingBreakdown - resolveZone(province : string) : ShippingZone - calculateRawShippingFee(zone : ShippingZone, totalWeightKg : number) : number - calculateDiscount(rawShippingFee : number, subtotalBeforeVat : number) : number - normalizeProvinceName(province : string) : string	

Table. Attribute design of WeightBasedShippingStrategy

#	Name	Data type	Default value	Description
1	config	ShippingFeeConfig	DEFAULT_SHIPPING_FEE_CONFIG	Injected pricing data (zones, fees, thresholds).

Table. Operation design of WeightBasedShippingStrategy

#	Name	Return type	Description (purpose)
1	calculate	ShippingBreakdown	Compute the final shipping fee for a package.
2	resolveZone	ShippingZone	[private] Match the province to a pricing zone (else default zone).

3	calculateRawShippingFee	number	[private] Base fee + extra 0.5kg blocks beyond the base weight.
4	calculateDiscount	number	[private] Free-ship discount when subtotal exceeds the threshold.
5	normalizeProvinceName	string	[private] Lowercase, strip diacritics/dots for province matching.

Parameter:

- calculate(context: ShippingContext)

Method

Ha Noi/HCM: 22,000 VND for first 3kg. Elsewhere: 30,000 VND for first 0.5kg. +2,500 VND per extra 0.5kg block (ceil). If items subtotal > 100,000 VND, discount up to 25,000 VND. finalShippingFee = max(0, rawFee - discount).

4.4.3.47 Class "EmailNotificationObserver"

<<observer>> EmailNotificationObserver	
- mailService : MailService	
+ update(event : OrderConfirmedEvent) : Promise<void>	

Table. Attribute design of EmailNotificationObserver

#	Name	Data type	Default value	Description
1	mailService	MailService	(injected)	Mail transport used to send the confirmation email.

Table. Operation design of EmailNotificationObserver

#	Name	Return type	Description (purpose)
1	update	Promise<void>	On an order-confirmed event, render and send the confirmation email.

Parameter:

- update(event: OrderConfirmedEvent)

Method

Skips when no recipient email. Builds a cancel-order link from CLIENT_URL + orderId, renders the order-confirmation template, and sends it via MailService.

4.4.3.48 Class "PlaceOrderPaymentAdapter"

<<adapter>> PlaceOrderPaymentAdapter	
- repository : IOrderRepository - orderStateFactory : OrderStateFactory - invoiceCalculator : InvoiceCalculator - eventPublisher : IOrderEventPublisher	
+ getPaymentContext(lookup : PaymentContextLookup) : Promise<PaymentContext> + markPaidAndPendingProcessing(context : PaidOrderContext) : Promise<void> - notifyOrderConfirmed(order : PersistedOrderDetail, context : PaidOrderContext) : Promise<void>	

Table. Attribute design of PlaceOrderPaymentAdapter

#	Name	Data type	Default value	Description
1	repository	IOrderRepository	(injected)	Order persistence port.
2	orderStateFactory	OrderStateFactory	(injected)	Resolves the order state for the transition.

3	invoiceCalculator	InvoiceCalculator	(injected)	Rebuilds the invoice for the email.
4	eventPublisher	IOrderEventPublisher	(injected)	Publishes the order-confirmed event.

Table. Operation design of PlaceOrderPaymentAdapter

#	Name	Return type	Description (purpose)
1	getPaymentContext	Promise<PaymentContext>	Return order payment context for the PaymentService.
2	markPaidAndPendingProcessing	Promise<void>	On successful payment: verify amount/invoice, apply the state transition, decrement stock, notify.
3	notifyOrderConfirmed	Promise<void>	[private] Build success DTO + invoice and publish the order-confirmed event.

Parameter:

- getPaymentContext(lookup: PaymentContextLookup)
- markPaidAndPendingProcessing(context: PaidOrderContext)

Exception:

- NotFoundException - order/invoice not found.
- BadRequestException - invoice does not belong to order, or amount mismatch.

Method

Server-side payment callback path. Uses the State pattern (OrderStateFactory.create(status).confirmPayment()) to decide if the transition is legal or a no-op before applying applyPaidTransition().

4.4.3.49 Class "InvalidQuantityException"

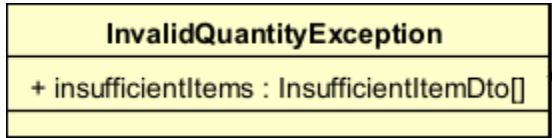


Table. Attribute design of InvalidQuantityException

#	Name	Data type	Default value	Description
1	insufficientItems	InsufficientItemDto[]	-	Items whose requested quantity exceeds the available stock.

Table. Operation design of InvalidQuantityException

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Thrown when stock is insufficient. Carries the per-product shortage so the client can show available quantities.

4.4.3.50 Class "InvalidDeliveryInfoException"

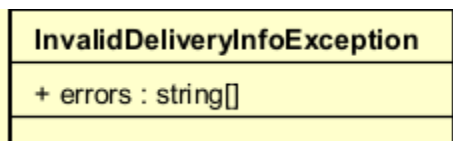


Table. Attribute design of InvalidDeliveryInfoException

#	Name	Data type	Default value	Description
1	errors	string[]	-	List of human-readable delivery-info validation messages.

Table. Operation design of InvalidDeliveryInfoException

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Thrown when one or more delivery-info rules fail.

4.4.3.51 Class "PaymentNotSuccessfulException"

PaymentNotSuccessfulException

Table. Attribute design of PaymentNotSuccessfulException

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of PaymentNotSuccessfulException

#	Name	Return type	Description (purpose)
1	constructor	-	Wraps a message into a BadRequestException.

Method

Thrown when the order payment is missing/unsuccessful (e.g., no transaction id or payment method on confirm).

4.4.3.52 Class "IllegalOrderTransitionError"

IllegalOrderTransitionError

Table. Attribute design of IllegalOrderTransitionError

#	Name	Data type	Default value	Description
-	(none)			This class declares no attributes.

Table. Operation design of IllegalOrderTransitionError

#	Name	Return type	Description (purpose)
1	constructor	-	Pure domain error (no HTTP/framework dependency).

Method

Raised when an order is asked to perform a status transition that is not allowed from its current state.

4.4.3.53 Class "CartItemDto"

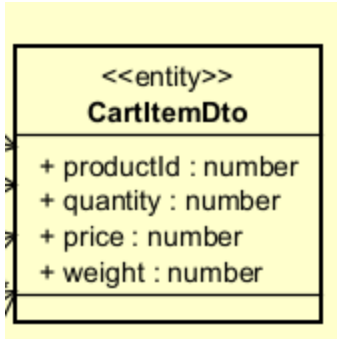


Table. Attribute design of CartItemDto

#	Name	Data type	Default value	Description
1	productId	number	-	Product identifier (positive integer).
2	quantity	number	-	Requested quantity (positive integer).
3	price	number	undefined	[ISP] Unused by the server (price always taken from DB).
4	weight	number	undefined	[ISP] Unused by the server (weight always taken from DB).

Table. Operation design of CartItemDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.54 Class "DeliveryInfoDto"

<<entity>> DeliveryInfoDto
+ receiverName : string + phoneNumber : string + province : string + streetAddress : string + shippingInstructions : string + email : string

Table. Attribute design of DeliveryInfoDto

#	Name	Data type	Default value	Description
1	receiverName	string	-	Recipient name (letters only).
2	phoneNumber	string	-	Recipient phone (10 digits).
3	province	string	-	Province/city.
4	streetAddress	string	-	Detailed street address.
5	shippingInstructions	string	undefined	Optional, up to 200 chars.
6	email	string	-	Customer email (required for order creation).

Table. Operation design of DeliveryInfoDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.55 Class "SubmitDeliveryInfoDto"



Table. Attribute design of SubmitDeliveryInfoDto

#	Name	Data type	Default value	Description
1	items	CartItemDto[]	-	Cart items.
2	deliveryInfo	DeliveryInfoDto	-	Delivery information.

Table. Operation design of SubmitDeliveryInfoDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.56 Class "PlaceOrderRequestDto"

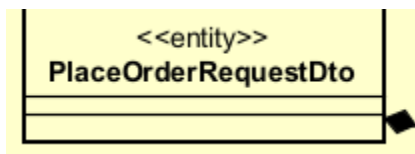


Table. Attribute design of PlaceOrderRequestDto

#	Name	Data type	Default value	Description
1	items	CartItemDto[]	-	Cart items to check stock for.

Table. Operation design of PlaceOrderRequestDto

#	Name	Return type	Description (purpose)

-	(none)		Data class - no operations.
---	--------	--	-----------------------------

Method

Data Transfer Object - no operations.

4.4.3.57 Class "PlaceOrderPaymentRequestDto"

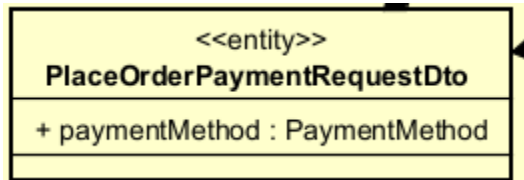


Table. Attribute design of PlaceOrderPaymentRequestDto

#	Name	Data type	Default value	Description
1	items	CartItemDto[]	-	Cart items.
2	deliveryInfo	DeliveryInfoDto	-	Delivery information.
3	paymentMethod	PaymentMethod	PaymentMethod.VIETQR	Chosen payment method (default QR).

Table. Operation design of PlaceOrderPaymentRequestDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.58 Class "ConfirmOrderDto"

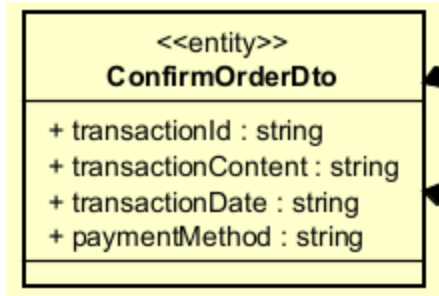


Table. Attribute design of ConfirmOrderDto

#	Name	Data type	Default value	Description
1	items	CartItemDto[]	-	Cart items.
2	deliveryInfo	DeliveryInfoDto	-	Delivery information.
3	transactionId	string	-	Payment transaction id.
4	transactionContent	string	undefined	Optional transaction content.
5	transactionDate	string	undefined	Optional ISO transaction date.
6	paymentMethod	string	-	Payment method used.

Table. Operation design of ConfirmOrderDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.59 Class "InvoiceItemDto"

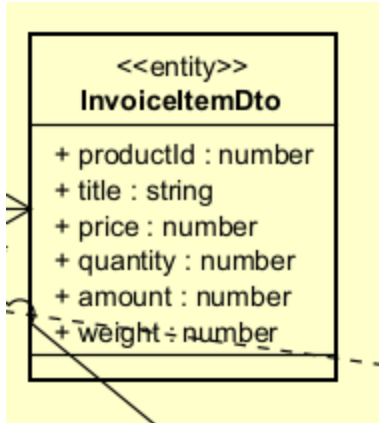


Table. Attribute design of InvoiceItemDto

#	Name	Data type	Default value	Description
1	productId	number	-	Product id.
2	title	string	undefined	Product title.
3	price	number	-	Unit price.
4	quantity	number	-	Quantity.
5	amount	number	-	price * quantity.
6	weight	number	undefined	Unit weight (kg).

Table. Operation design of InvoiceItemDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.60 Class "InvoicePreviewDto"

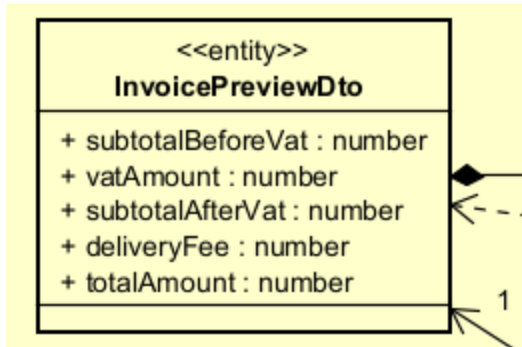


Table. Attribute design of InvoicePreviewDto

#	Name	Data type	Default value	Description
1	items	InvoiceItemDto[]	-	Priced line items.
2	subtotalBeforeVat	number	-	Items subtotal before VAT.
3	vatAmount	number	-	VAT amount.
4	subtotalAfterVat	number	-	Subtotal including VAT.
5	deliveryFee	number	-	Final delivery fee.
6	totalAmount	number	-	Subtotal incl. VAT + delivery fee.

Table. Operation design of InvoicePreviewDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.61 Class "OrderSuccessDto"

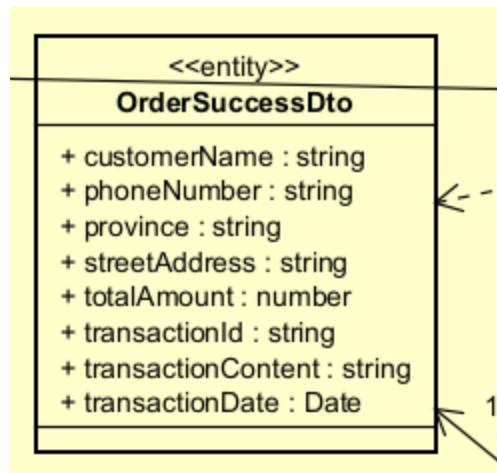


Table. Attribute design of OrderSuccessDto

#	Name	Data type	Default value	Description
1	customerName	string	-	Customer name.
2	phoneNumber	string	-	Customer phone.
3	province	string	-	Province.
4	streetAddress	string	-	Street address.
5	totalAmount	number	-	Total paid.
6	transactionId	string	-	Transaction id.
7	transactionContent	string	-	Transaction content.
8	transactionDate	Date	-	Transaction date.

Table. Operation design of OrderSuccessDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.62 Class "InsufficientItemDto"

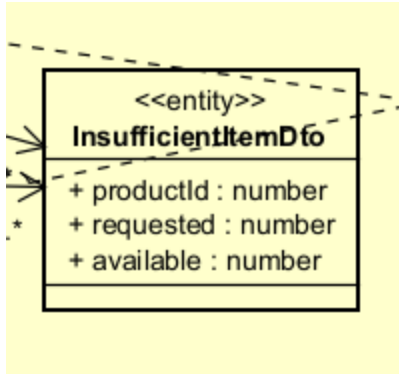


Table. Attribute design of InsufficientItemDto

#	Name	Data type	Default value	Description
1	productId	number	-	Product id that is short.
2	requested	number	-	Requested quantity.
3	available	number	-	Available quantity.

Table. Operation design of InsufficientItemDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.63 Class "StockCheckResultDto"

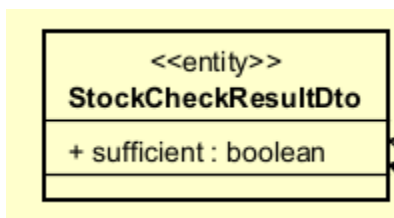


Table. Attribute design of StockCheckResultDto

#	Name	Data type	Default value	Description
1	sufficient	boolean	-	True if every item has enough stock.
2	insufficientItems	InsufficientItemDto[]	-	Items lacking stock.

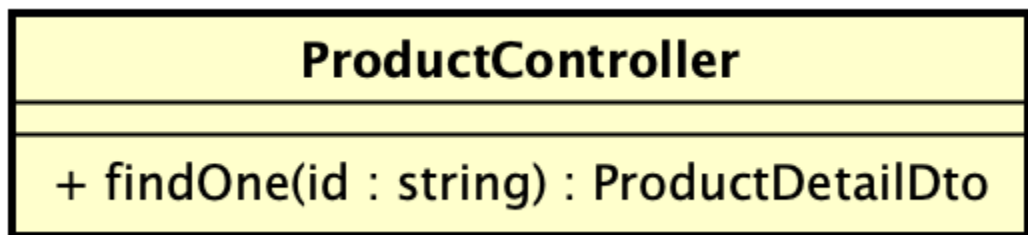
Table. Operation design of StockCheckResultDto

#	Name	Return type	Description (purpose)
-	(none)		Data class - no operations.

Method

Data Transfer Object - no operations.

4.4.3.64 Class ProductController



REST controller (NestJS) that exposes the product read API. For UC235 it exposes GET /products/:id and delegates to GetProductDetailUseCase. Kept thin: no Prisma queries, DTO mapping or UI logic.

Table 2. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

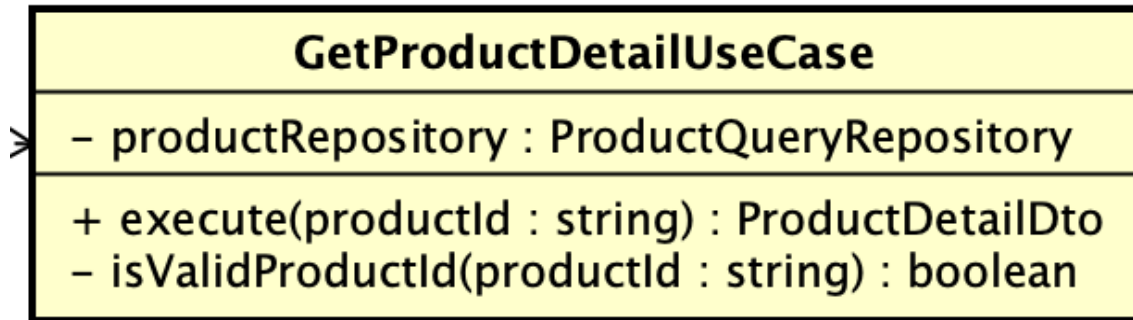
1	productService	ProductService	-	Product command operations (not used by UC235).
2	getProductsListUseCase	GetProductsListUseCase	-	Catalog listing use case.
3	getProductDetailUseCase	GetProductDetailUseCase	-	Main dependency for UC235: retrieves a product detail.
4	getProductLogsUseCase	GetProductLogsUseCase	-	Product log query (manager).
5	clouinaryService	ClouinaryService	-	Image upload (manager).

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	findOne(id: string)	Promise<ProductDetailDto>	Handles GET /products/:id; delegates to getProductDetailUseCase.execute(id) and returns the product detail.
2	findAll(...)	Promise<...>	Handles GET /products catalog listing (filtering, sorting, pagination).

Parameter: id – product identifier taken from the URL path (@Param('id')).

4.4.3.65 Class *GetProductDetailUseCase*



Application use case that orchestrates the business flow for retrieving one product detail: validate id → fetch via repository → map to DTO. Depends on the **ProductQueryRepository** abstraction (DIP).

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	productRepository	ProductQueryRepository	-	Injected read-side repository abstraction (token PRODUCT_QUERY_REPOSITORY).

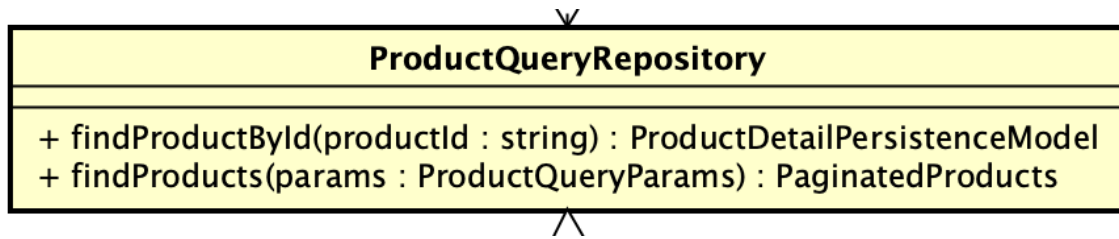
Table 3. Operation design

#	Name	Return type	Description (purpose)
1	execute(productId: string)	Promise<ProductDetailDto>	Validates the id, loads the product, and returns the mapped ProductDetailDto .
2	isValidProductId(productId: string)	boolean	Private. Returns true if the id matches ^[1-9]\d*\$.

Exception: **BadRequestException** if the product id is invalid.

Exception: **NotFoundException** if no product with the given id exists.

4.4.3.66 Class *ProductQueryRepository*



Domain abstraction that exposes only read operations needed for product viewing; PrismaProductRepository is the Prisma-based implementation. Command operations (create/update/delete) are intentionally kept in separate abstractions (ISP).

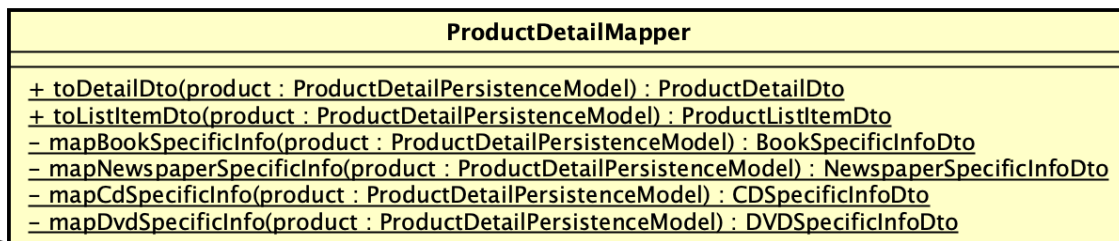
Table 2. Attribute design

#	Name	Data type	Default value	Description
1	(no fields – interface)	-	-	Implementation holds the Prisma client / data source.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	findProductById(productId: string)	Promise<ProductDetailPersistenceModel null>	Returns the persistence record for the id, or null when not found (used by UC235).
2	findProducts(params?: ProductQueryParams)	Promise<PaginatedProducts>	Returns a filtered/sorted/paginated catalog page.

4.4.3.67 Class *ProductDetailMapper*



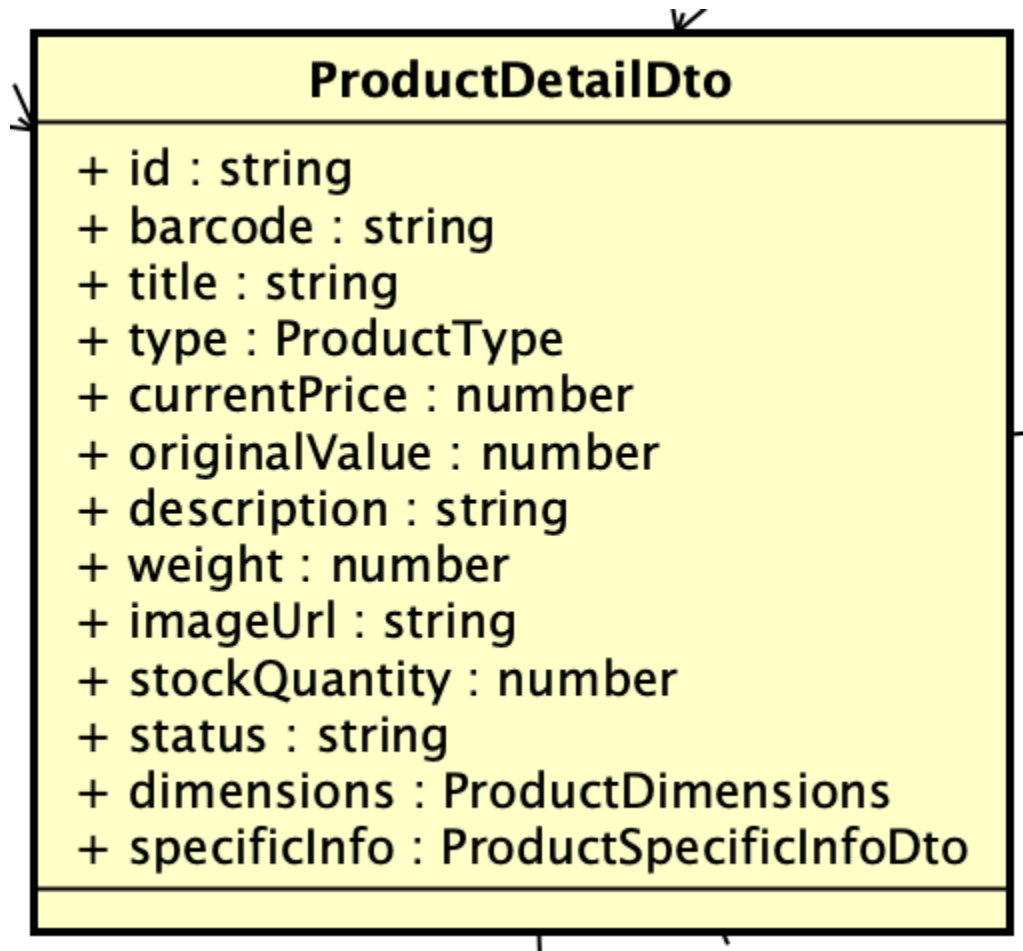
Static mapper that transforms a persistence record into response DTOs. Type-specific mapping is dispatched by a Strategy map (Record<ProductType, mapperFn>) so each product type is handled by its own method.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	toDetailDto(product)	ProductDetailDto	Maps common product fields and attaches the type-specific specificInfo section.
2	toListItemDto(product)	ProductListItemDto	Maps a record into a lightweight catalog list item.
3	mapSpecificInfo(type, product)	ProductSpecificInfoDto	Strategy dispatch: selects the mapper for the product type.
4	mapBookSpecificInfo / mapNewspaperSpecificInfo / mapCdSpecificInfo / mapDvdSpecificInfo	*SpecificInfoDto	Build the type-specific section for each product type.

Helpers: normalizeType, normalizeStatus, parseDimensions, toNumber, toStringValue, toDateString, toStringList (private conversion utilities).

4.4.3.68 Class ProductDetailDto



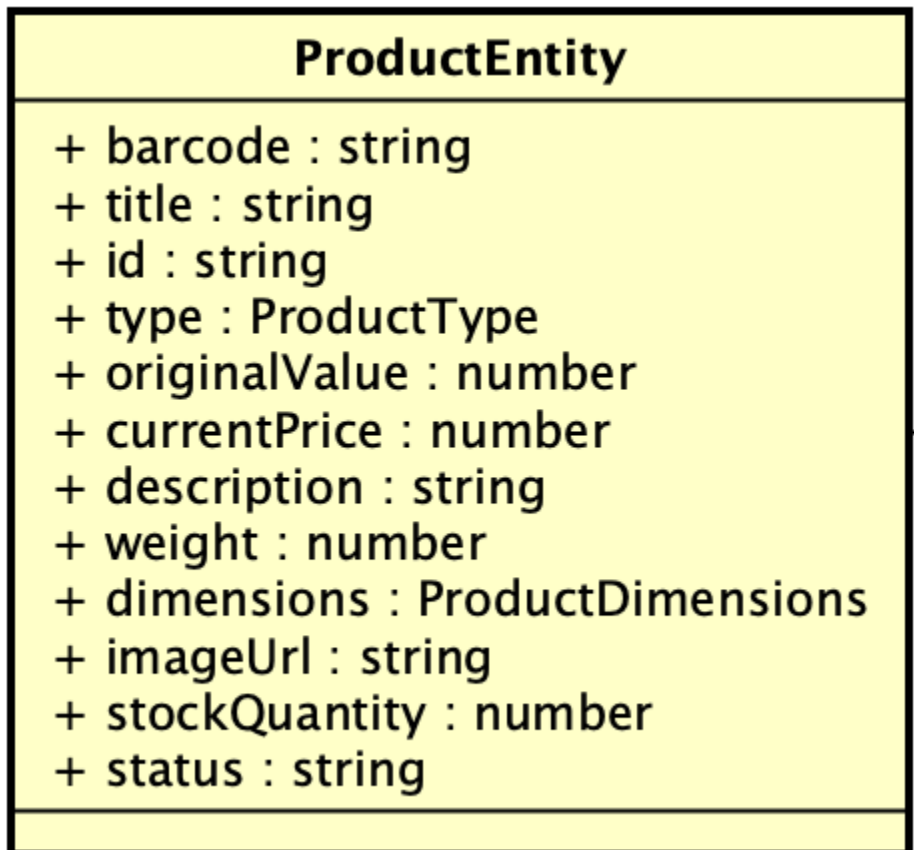
Response contract returned to the frontend for the product detail screen. Common fields are stable; the specificInfo section is extended per product type (OCP).

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	id	string	-	Product identifier.
2	barcode	string	-	Product barcode.
3	title	string	-	Product title.
4	type	ProductTypeValue	-	BOOK NEWSPAPER CD DVD.
5	currentPrice	number	-	Current selling price.
6	originalValue	number	-	Original value of the product.

7	description	string	-	Product description.
8	weight	number	-	Weight (for shipping calculation).
9	dimensions	ProductDimensions	-	height x width x length.
10	imageUrl	string?	undefined	Optional product image URL.
11	stockQuantity	number	-	Available stock quantity.
12	status	string	-	Availability status (e.g. AVAILABLE, OUT_OF_STOCK).
13	specificInfo	ProductSpecificInfoDto	-	Type-specific section (Book/Newspaper/CD/DVD).

4.4.3.69 Class *ProductEntity*



Domain entity holding the common product fields used by read use cases. Type-specific fields are kept out of this common contract and live in the specificInfo DTOs.

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	id	string	-	Identifier.
2	barcode	string	-	Barcode.
3	title	string	-	Title.
4	type	ProductTypeValue	-	Product type.
5	currentPrice	number	-	Current price.
6	originalValue	number	-	Original value.
7	description	string	-	Description.
8	weight	number	-	Weight.
9	dimensions	ProductDimensions	-	Dimensions.
10	imageUrl	string?	undefined	Optional image.
11	stockQuantity	number	-	Stock.
12	status	string	-	Status.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	constructor(partial: Partial<ProductEntity>)	ProductEntity	Builds an entity from a partial object (Object.assign).

4.4.3.70 Class *ProductDetailPageComponent*

ProductDetailPageComponent	
+ product : ProductDetail + isLoading : boolean + errorMessage : string + notFound : boolean + cartCount : number	-
+ ngOnInit() : void + goToProducts() : void + goToCatalog() : void + goToCart() : void + searchProducts(query : string) : void - handleLoadError(error : unknown) : void	-

Angular page (smart) component for the product detail screen. Coordinates route-parameter reading, loading state and error handling, and delegates rendering to child components.

Table 2. Attribute design

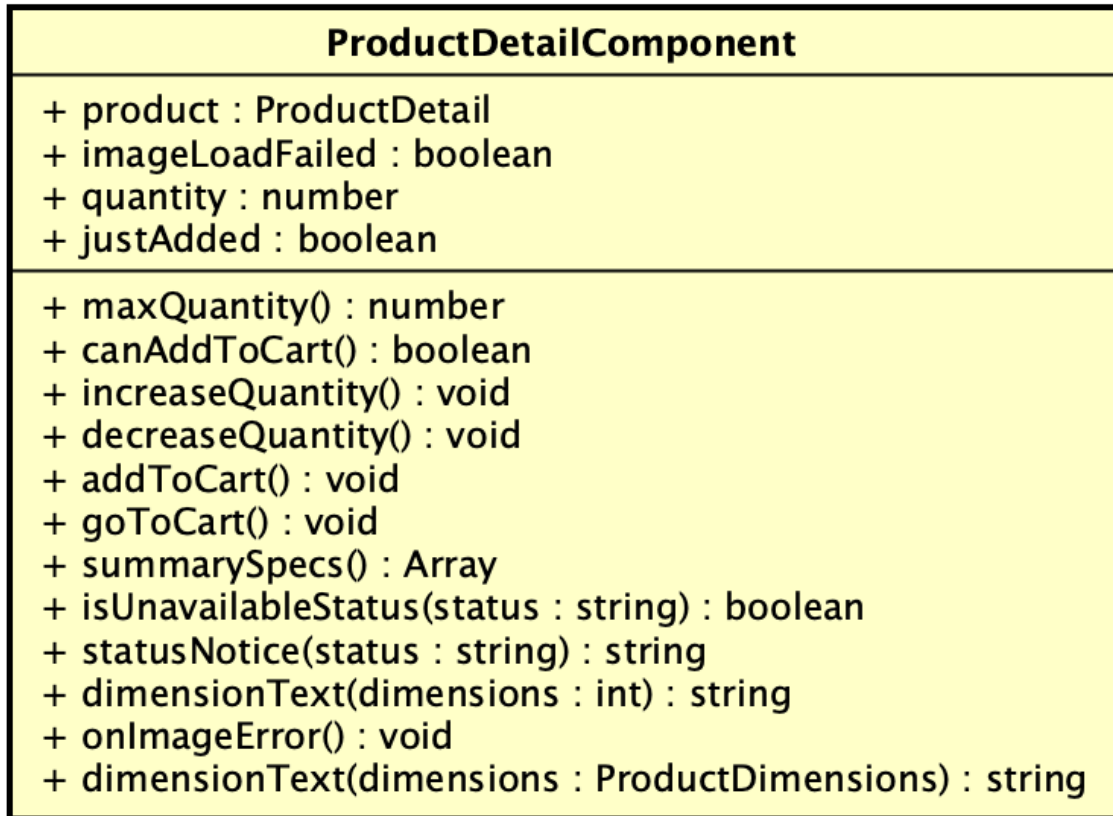
#	Name	Data type	Default value	Description
1	product	Signal<ProductDetail null>	null	Loaded product detail.
2	isLoading	Signal<boolean>	true	Loading flag.
3	errorMessage	Signal<string>	"	Error message for non-404 failures.
4	notFound	Signal<boolean>	false	True when the product id returns 404.

5	route	ActivatedRoute	-	Reads the :id route parameter.
6	productApi	ProductApiService	-	Fetches the product detail.
7	cartCount	Signal<number>	-	Cart item count for the header.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	ngOnInit()	void	Subscribes to route params, fetches the product via productApi.getProductDetail(id), updates state.
2	handleLoadError(error)	void	Private. Sets notFound on HTTP 404, otherwise sets an error message.
3	goToProducts() / goToCatalog()	void	Navigate back to the catalog.
4	goToCart()	void	Navigate to the cart.
5	searchProducts(query: string)	void	Navigate to the catalog with a search query.

4.4.3.71 Class *ProductDetailComponent*



Presentational component that renders the common product detail fields and owns the local add-to-cart quantity selector. Cart persistence is delegated to CartStoreService; type-specific fields are delegated to ProductSpecificInfoComponent.

Table 2. Attribute design

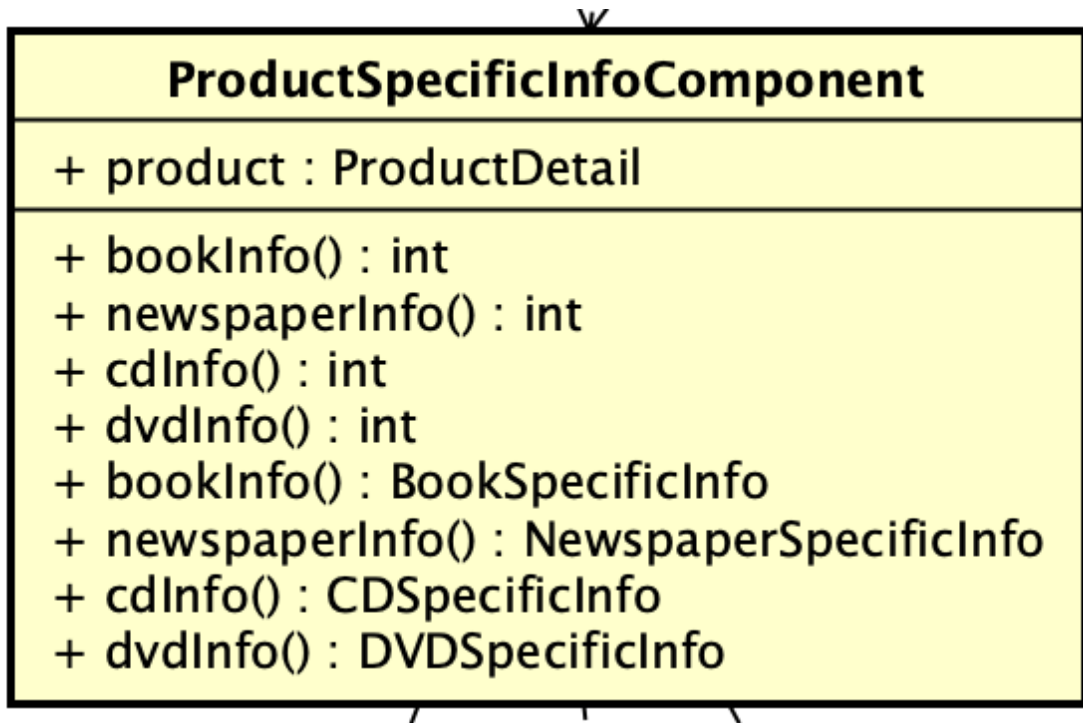
#	Name	Data type	Default value	Description
1	product	ProductDetail (@Input, required)	-	Product to display.
2	quantity	Signal<number>	1	Selected add-to-cart quantity.
3	justAdded	Signal<boolean>	false	Shows the 'added to cart' notice.

4	imageLoadFailed	boolean	false	True when the product image fails to load.
5	cartStore	CartStoreService	-	Cart state (add item).

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	maxQuantity	number (getter)	Maximum selectable quantity from stock.
2	canAddToCart	boolean (getter)	True when the product is available and in stock.
3	increaseQuantity() / decreaseQuantity()	void	Adjust the selected quantity within bounds.
4	addToCart()	void	Adds the product to the cart via CartStoreService.
5	summarySpecs()	Array<{label,value}>	Returns the short type-specific spec summary.
6	isUnavailableStatus(status) / statusNotice(status)	boolean / string	Availability checks and user-facing notice.
7	dimensionText(dimensions)	string	Formats dimensions as h x w x l.
8	onImageError()	void	Marks the image as failed to load.

4.4.3.72 Class *ProductSpecificInfoComponent*



Small dispatcher (presentational) component that renders the correct type-specific information by delegating to Book/Newspaper/CD/DVD specific-info components.

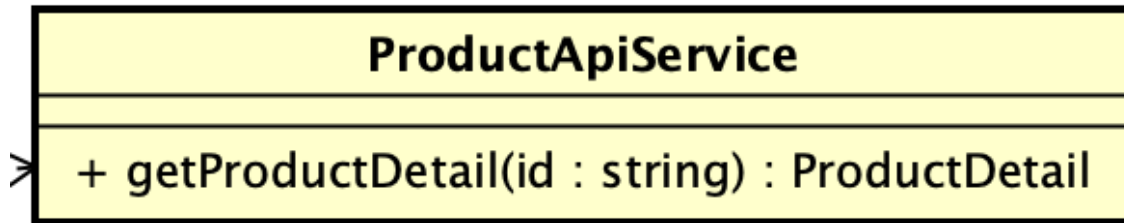
Table 2. Attribute design

#	Name	Data type	Default value	Description
1	product	ProductDetail (@Input, required)	-	Product whose specificInfo is rendered.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	bookInfo / newspaperInfo / cdInfo / dvdInfo	*SpecificInfo (getters)	Cast specificInfo to the matching type and pass it to the corresponding child component.

4.4.3.73 Class *ProductApiService*



Angular service that centralizes the product read HTTP calls so components do not duplicate API URLs.

Table 2. Attribute design

#	Name	Data type	Default value	Description
1	apiUrl	string	`\${AIMS_API_BASE_URL}/products`	Base products endpoint.
2	http	HttpClient	-	Angular HTTP client.

Table 3. Operation design

#	Name	Return type	Description (purpose)
1	getProductDetail(productId: string)	Observable<ProductDetail>	Calls GET /products/:id and normalizes the response into the ProductDetail model (used by UC235).
2	getProducts()	Observable<ProductListItem[]>	Loads the product list.
3	getProductsPaged(query)	Observable<ProductPage>	Loads a paginated/filtered/sorted catalog page.
4	normalizeProductDetail(response)	ProductDetail	Private. Maps the raw API payload into the

			typed ProductDetail model.
--	--	--	----------------------------

4.4.3.74 Class "PlaceOrderPaymentResultDto"

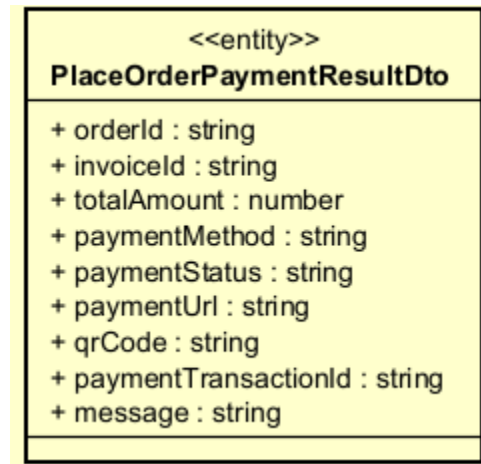


Table. Attribute design of PlaceOrderPaymentResultDto

#	Name	Data type	Default value	Description
1	orderId	string	-	Created order id.
2	invoiceId	string	-	Invoice id.
3	totalAmount	number	-	Amount to pay.
4	paymentMethod	string	-	Payment method.
5	paymentStatus	string	-	Payment status.
6	paymentUrl	string	-	Redirect/payment URL.
7	qrCode	string	-	QR code payload.
8	paymentTransactionId	string	-	Gateway transaction id.
9	message	string	-	Result message.

Table. Operation design of PlaceOrderPaymentResultDto

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

-	(none)		Data class - no operations.
---	--------	--	-----------------------------

Method

Data Transfer Object - no operations.

4.4.3.75 Class "ProductController"

<<boundary>> ProductController			
+ create(dto : CreateProductDto, user : any) : Promise<ProductDetail> + findAllForManager() : Promise<any[]> + update(id : string, dto : UpdateProductDto, user : any) : Promise<any> + uploadImage(id : string, file : any, user : any) : Promise<ImageUploadResult> + deleteProduct(id : string, req : any) : Promise<DeleteResult> + deleteBulk(ids : string[], req : any) : Promise<DeleteResult[]> + getLogs(id : string) : Promise<any[]> + getAllLogs() : Promise<any[]>			

Table 1. Attributes of ProductController

#	Name	Data type	Default value	Description
1	productService	ProductService		Service handling CUD operations for products
2	getProductsList UseCase	GetProductsListUse Case		Use case for querying the product list
3	getProductDetailUseCase	GetProductDetailUse Case		Use case for querying product detail

4	getProductLogsUseCase	GetProductLogsUseCase		Use case for querying logs of a single product
5	getAllProductLogsUseCase	GetAllProductLogsUseCase		Use case for querying all product logs
6	cloudinaryService	CloudinaryService		Service for uploading images to Cloudinary

Table 2. **Operations of ProductController**

#	Name	Return type	Description
1	create(dto, user)	Promise<any>	Creates a new product. Requires role Product Manager.
2	findAll(...)	Promise<any>	Queries the product list with filter/sort/pagination (public).
3	findAllForManager()	Promise<any[]>	Retrieves all products without filtering, for Product Managers.
4	findOne(id)	Promise<any>	Queries a single product by ID (public).
5	update(id, dto, user)	Promise<any>	Updates product information. Requires role Product Manager.
6	uploadImage(id, file, user)	Promise<{imageUrl}>	Uploads a product image to Cloudinary and updates the URL in the DB.
7	deleteProduct(id, req)	Promise<DeleteResult>	Soft-deletes or hard-deletes a product based on business conditions.
8	deleteBulk(ids, req)	Promise<DeleteResult[]>	Deletes multiple products at once. Maximum 10 products.

9	getLogs(id)	Promise<any[]>	Retrieves the operation history of a single product.
10	getAllLogs()	Promise<any[]>	Retrieves the full product operation history.

Method:

create(dto, user)

- Parameters:
 - dto: CreateProductDto — Product creation data (required)
 - user: any — Current user info extracted from JWT token
- Exception: Throws 403 Forbidden if the user does not have the Product Manager role
- Flow: Receive DTO → Call productService.create(dto, user.userId) → Return the created product

uploadImage(id, file, user)

- Parameters:
 - id: string — ID of the product
 - file: any — Image file from multipart/form-data
 - user: any — Current user
- Exception: Throws BadRequestException if file is null/undefined
- Flow: Validate file → Upload to Cloudinary → Get secure_url → Call productService.updateImageUrl() → Return { imageUrl }

deleteBulk(ids, req)

- Parameters:
 - ids: string[] — Array of product IDs to delete (max 10)
 - req: any — Request object containing user info
- Exception: BadRequestException if ids is empty or exceeds 10 items

4.4.3.76 Class "ProductService"

<<control>> ProductService	
- repository : ProductRepository - productLogService : IProductLogService - dailyQuotaService : IDailyQuotaService - specifications : IProductSpecification[] - hardDeleteSpec : HardDeleteSpecification	
+ create(dto : CreateProductDto, userId : string) : Promise<any> + findAll() : Promise<any[]> + findOne(id : string) : Promise<any> + update(id : string, dto : UpdateProductDto, userId : string, action? : string) : Promise<any> + deleteProduct(id : string, userId : string) : Promise<DeleteResult> + deleteBulk(ids : string[], userId : string) : Promise<DeleteResult[]> + updateImageUrl(id : string, imageUrl : string, userId : string) : Promise<any> - validateProductSpecifications(productData : any) : void	

Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	repository	ProductRepository	—	Repository executing DB operations for Product
2	productLogService	IProductLogService	—	Service for writing audit logs, injected via token
3	dailyQuotaService	IDailyQuotaService	—	Service for enforcing daily deletion quotas
4	specifications	IProductSpecification[]	[]	List of business rule specifications
5	hardDeleteSpec	ISpecification<any>	—	Specification determining if a product may be hard-deleted

Table 2. **Operations**

#	Name	Return type	Description
1	create(dto, userId)	Promise<any>	Validates specifications → creates product → logs CREATE
2	findAll()	Promise<any[]>	Retrieves all products from the repository
3	findOne(id)	Promise<any>	Finds a product by ID; throws an error if not found
4	update(id, dto, userId, action?)	Promise<any>	Validates specifications → updates product → logs UPDATE
5	deleteProduct(id, userId)	Promise<{status}>	Checks quota → hard/soft delete → logs action → increments quota
6	deleteBulk(ids, userId)	Promise<DeleteResult[]>	Checks bulk quota → deletes each item → increments bulk quota
7	updateImageUrl(id, imageUrl, userId)	Promise<any>	Updates the image URL and logs UPDATE_IMAGE
8	validateProductSpecifications(productData)	void	Iterates all specifications; throws an error if any is violated

4.4.3.77 Class "ProductRepository"

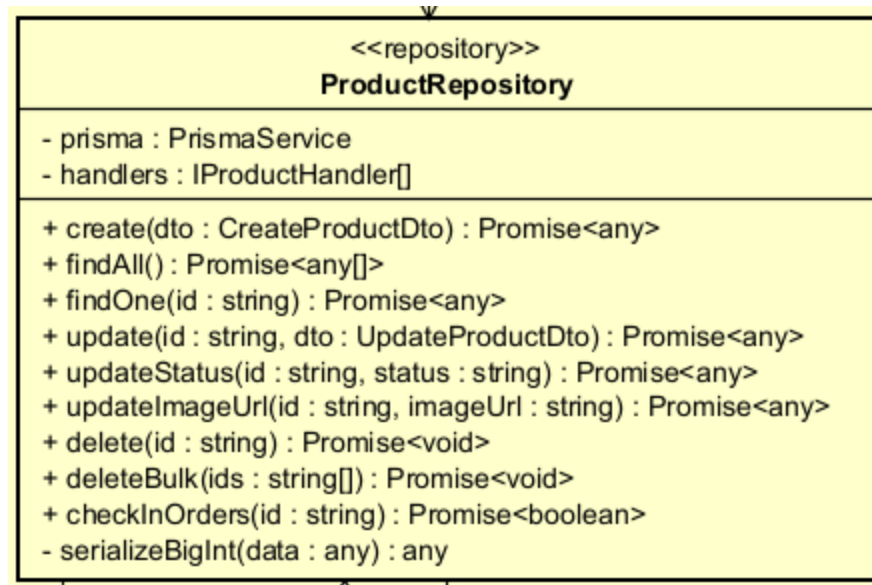


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	prisma	PrismaService	—	ORM client for executing DB queries
2	handlers	IProductHandler[]	—	List of handlers for each product type

Table 2. **Operations**

#	Name	Return type	Description
1	create(dto)	Promise<bigint>	Creates the base product record and delegates to the matching handler within a transaction
2	findAll()	Promise<any>	Retrieves all products with nested includes
3	findOne(id)	`Promise	null>`

4	update(id, dto)	Promise<any>	Updates the base product and delegates to the matching handler within a transaction
5	updateStatus(id, status)	Promise<any>	Updates the status field of a product
6	updateImageUrl(id, imageUrl)	Promise<any>	Updates the imageUrl field of a product
7	delete(id)	Promise<void>	Deletes a product after checking for order constraints
8	deleteBulk(ids)	Promise<void>	Deletes multiple products within a single transaction
9	checkInOrders(id)	Promise<boolean>	Checks whether a product exists in any orders
10	serializeBigInt(data)	any	Converts BigInt values to strings for JSON serialization

4.4.3.78 Class "CloudinaryService"

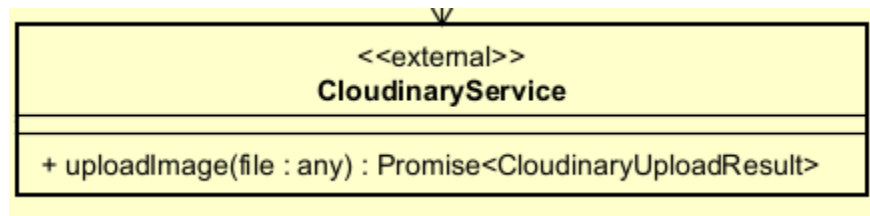


Table 2. **Operation**

#	Name	Return type	Description
1	uploadImage(file)	Promise<Cloudinary UploadResult>	Uploads an image file to Cloudinary and returns the result containing secure_url

4.4.3.79 Class "BaseCreateProductDto"

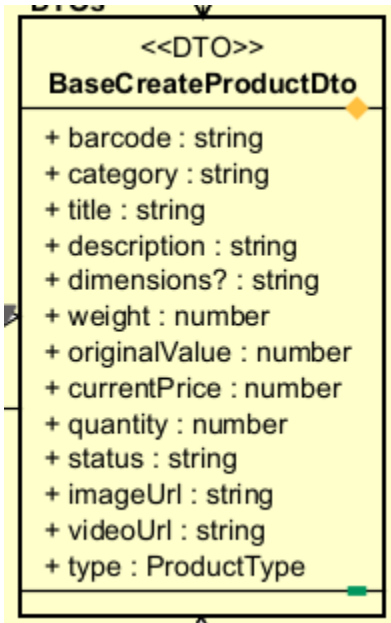


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	barcode	string	—	Product barcode; required
2	category	string	—	Product category; required
3	title	string	—	Product name; required
4	description	string	—	Product description; required
5	dimensions?	string	—	Product dimensions (HxWxL); optional
6	weight	number	—	Weight in kg; ≥ 0 ; required

7	originalValue	number	—	Original product value; ≥ 0 ; required
8	currentPrice	number	—	Current selling price; ≥ 0 ; required
9	quantity	number	—	Stock quantity; ≥ 0 ; required
10	status	string	—	Product status (ACTIVE/INACTIVE); required
11	imageUrl	string	—	Product image URL; required
12	videoUrl	string	—	Product video URL; required
13	type	ProductType	—	Product type (BOOK/CD/DVD/NEWSPAPER); required

4.4.3.80 Class "CreateBookDto"

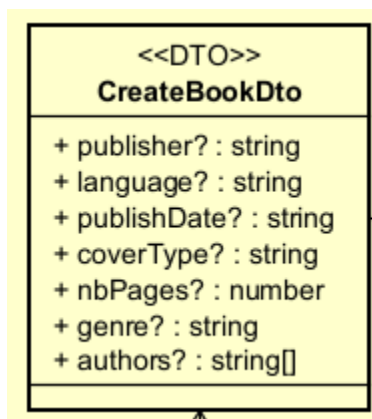


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	publisher?	string	—	Publisher name
2	language?	string	—	Language of the book
3	publishDate?	string (ISO date)	—	Publication date
4	coverType?	string	—	Cover type (hardcover/paperback)
5	nbPages?	number	—	Number of pages
6	genre?	string	—	Book genre
7	authors?	string[]	—	List of author names

4.4.3.81 Class "CreateCdDto"

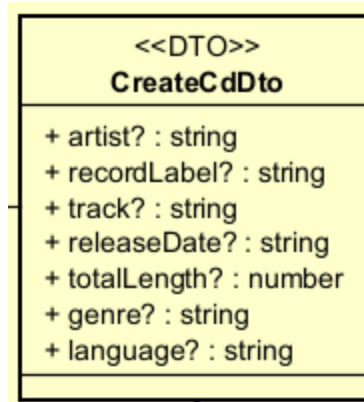


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	artist?	string	—	Artist name
2	recordLabel?	string	—	Record label
3	track?	string	—	Track listing
4	releaseDate?	string (ISO date)	—	Release date
5	totalLength?	number	—	Total duration in seconds
6	genre?	string	—	Music genre
7	language?	string	—	Language

4.4.3.82 Class "CreateDvdBookDto"

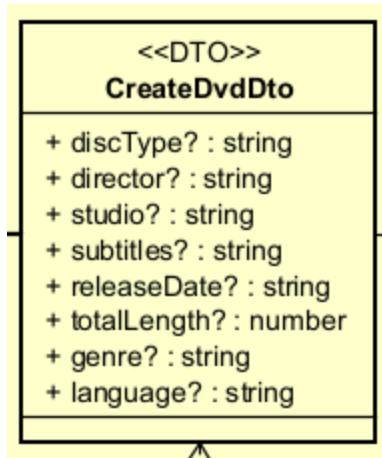


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	discType?	string	—	Disc type (Blu-ray/DVD)
2	director?	string	—	Director name
3	studio?	string	—	Production studio
4	subtitles?	string	—	Available subtitles
5	releaseDate?	string (ISO date)	—	Release date
6	totalLength?	number	—	Film runtime in minutes
7	genre?	string	—	Film genre
8	language?	string	—	Language

4.4.3.83 Class "CreateNewspaperDto"

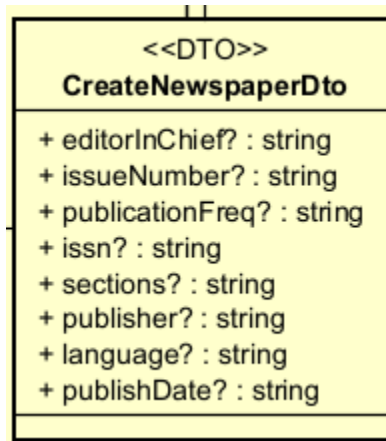


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	editorInChief?	string	—	Editor-in-chief name
2	issueNumber?	string	—	Issue number
3	publicationFreq?	string	—	Publication frequency
4	issn?	string	—	ISSN code
5	sections?	string	—	Sections included in the newspaper
6	publisher?	string	—	Publisher name
7	language?	string	—	Language
8	publishDate?	string (ISO date)	—	Publication date

4.4.3.84 Class "CloudinaryUploadResult"

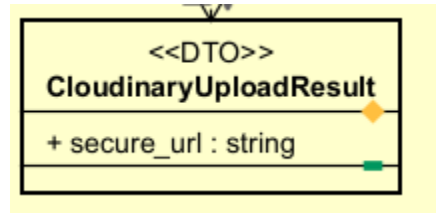


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	secure_url	string	—	HTTPS URL of the uploaded image on Cloudinary

4.4.3.85 Interface "IProductHandler<T, U>"

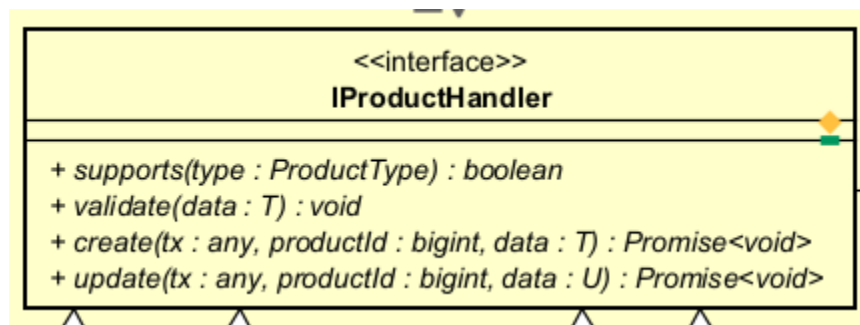


Table 1. **Operations**

#	Name	Return type	Description
1	supports(type)	boolean	Returns whether this handler is responsible for the given product type

2	validate(data)	void	Validates type-specific data before persistence
3	create(tx, productId, data)	Promise<void>	Creates type-specific DB records within a Prisma transaction
4	update(tx, productId, data)	Promise<void>	Updates type-specific DB records within a Prisma transaction

Parameter Details:

supports(type: ProductType): boolean

- type: The product type to check

create(tx: any, productId: bigint, data: T): Promise<void>

- tx: Prisma transaction client
- productId: ID of the base Product record just created
- data: T: Type-specific DTO (e.g., CreateBookDto, CreateCdDto)

update(tx: any, productId: bigint, data: U): Promise<void>

- tx: Prisma transaction client
- productId: ID of the Product to update
- data: U: Type-specific update DTO (currently typed as any)

4.4.3.86 Interface "ISpecification<T>"

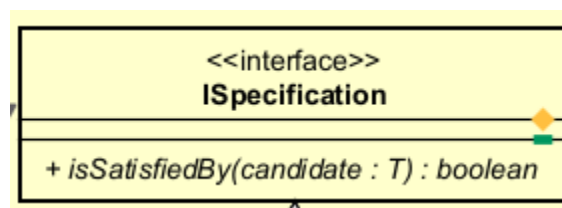


Table 1. **Operations**

#	Name	Return type	Description
1	isSatisfiedBy(candidate)	boolean	Checks whether candidate satisfies the specification

4.4.3.87 Interface "IProductSpecification"

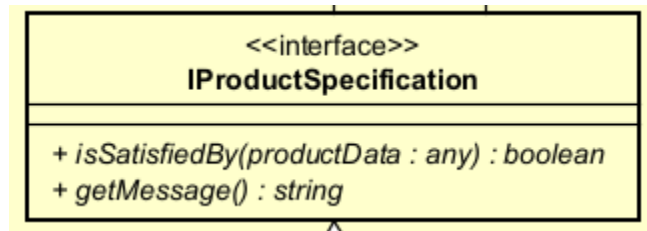


Table 1. **Operations**

#	Name	Return type	Description
1	isSatisfiedBy(candidate)	boolean	Inherited from ISpecification
2	getMessage()	string	Returns the error message when the specification is not satisfied

4.4.3.88 Class "HardDeleteSpecification"

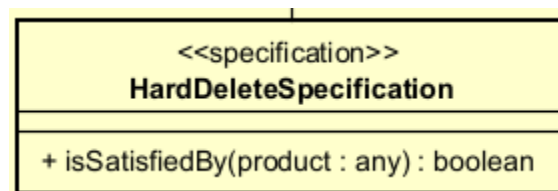


Table 1. **Operations**

#	Name	Return type	Description
1	isSatisfiedBy(product)	boolean	Returns true if product.quantity == 0

Method Details:

isSatisfiedBy(product)

- Parameters:

product: any — The product object to evaluate (must contain the quantity field)

- **Logic:** return product.quantity === 0
- **Business Rule:** If quantity > 0, the product is DEACTIVATED instead of DELETED

4.4.3.89 Class "PriceRatioSpecification"

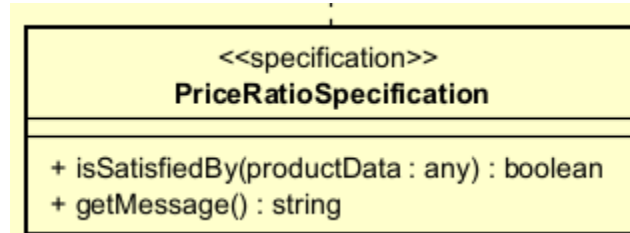


Table 1. **Operations**

#	Name	Return type	Description
1	isSatisfiedBy(productData)	boolean	Checks that currentPrice is within [30%, 150%] of originalValue
2	getMessage()	string	Returns a validation error message

Method Details:

isSatisfiedBy(productData)

- **Parameters:**
 - o productData: any — Must contain currentPrice and originalValue
- **Logic:**

```

currentPrice = Number(productData.currentPrice)
originalValue = Number(productData.originalValue)
IF either is NaN return true (skip; let DTO validation handle it)
IF originalValue === 0 return currentPrice === 0
ELSE return 0.3 * originalValue ≤ currentPrice ≤ 1.5 * originalValue
  
```
- **Business Rule :** The selling price must be between 30% and 150% of the original value. If the original value is 0, the selling price must also be 0.

getMessage()

- **Return:** "The current price must be between 30% and 150% of the original value. (Or must be 0 if the original value is 0)."

4.4.3.90 Interface "IDailyQuotaService"

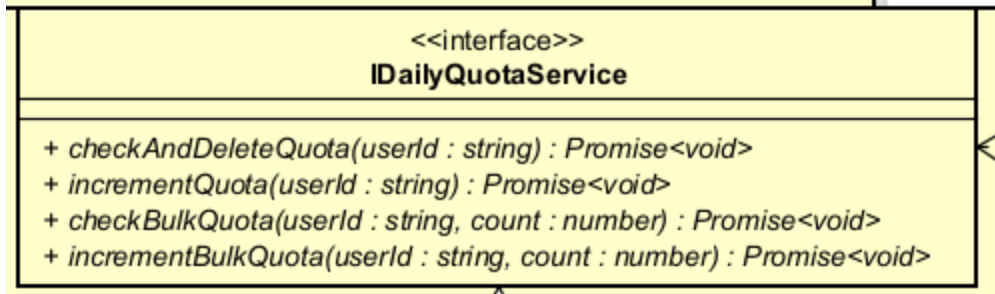


Table 1. **Operations**

#	Name	Return type	Description
1	checkAndDeleteQuota(userId)	Promise<void>	Checks remaining quota for a single deletion. Throws HTTP 429 if the limit is reached.
2	incrementQuota(userId)	Promise<void>	Increments the user's deletion counter by 1
3	checkBulkQuota(userId, count)	Promise<void>	Checks remaining quota for a bulk deletion of count items. Throws HTTP 429 if insufficient.
4	incrementBulkQuota(userId, count)	Promise<void>	Increments the user's deletion counter by count

4.4.3.91 Class "DailyQuotaService"

<<control>> DailyQuotaService	
- MAX_DAILY_DELETES : number - quotaRepository : IDailyQuotaRepository	
+ checkAndDeleteQuota(userId : string) : Promise<void> + incrementQuota(userId : string) : Promise<void> + checkBulkQuota(userId : string, count : number) : Promise<void> + incrementBulkQuota(userId : string, count : number) : Promise<void> - calculateCurrentDeletes(quotaData : UserQuotaData) : number	

Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	MAX_DAILY_DELETES	number (static const)	20	Maximum number of deletions allowed per user per day
2	quotaRepository	IDailyQuotaRepository	—	Repository for reading and writing quota data

Table 2. **Operations**

#	Name	Return type	Description
1	checkAndDeleteQuota(userId)	Promise<void>	Fetches current quota and enforces the daily limit
2	incrementQuota(userId)	Promise<void>	Persists dailyDeletes + 1 to the DB
3	checkBulkQuota(userId, count)	Promise<void>	Checks that currentDeletes + count $\leq$ MAX
4	incrementBulkQuota(userId, count)	Promise<void>	Persists dailyDeletes + count to the DB

5	calculateCurrentDeletes(quotaData)	number	Returns 0 if lastDeleteDate is not today (UTC); resets the counter for a new day
---	------------------------------------	--------	--

Method Details:

calculateCurrentDeletes(quotaData)

- Parameters:

quotaData: UserQuotaData — Quota data retrieved from DB

- Logic:

IF isTodayUTC(quotaData.lastDeleteDate) → return quotaData.dailyDeletes

ELSE return 0 (new day — reset counter)

checkAndDeleteQuota(userId)

- Parameters: userId: string

- Exception:

BadRequestException if the user does not exist

HttpException (429) if currentDeletes >= MAX_DAILY_DELETES

4.4.3.92 Class "UserQuotaData"

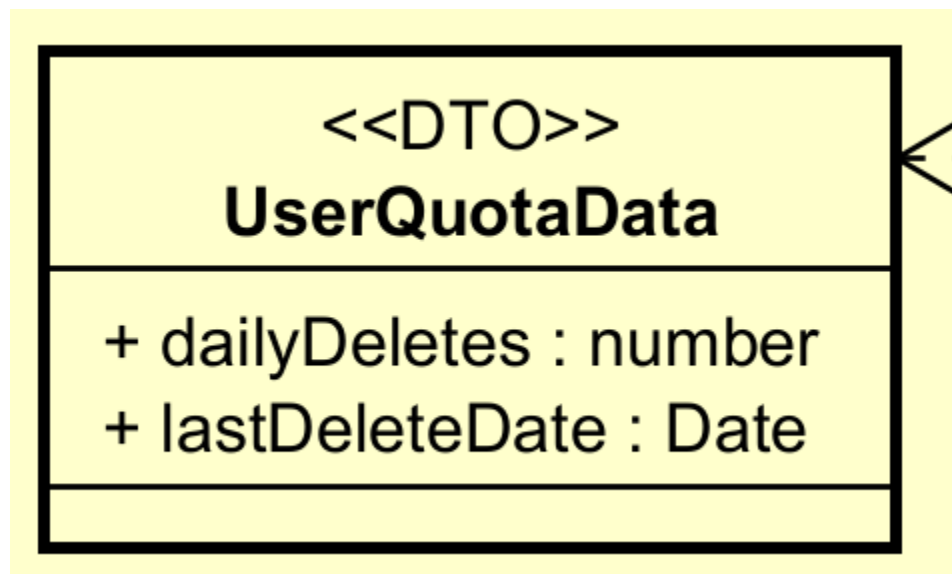


Table 1. Attributes

#	Name	Data type	Default value	Description
1	dailyDeletes	number	—	Number of products deleted by the user today
2	lastDeleteDate	Date \	null	—

4.4.3.93 Interface "IDailyQuotaRepository"

<<interface>> IDailyQuotaRepository	
+ findUserQuota(userId : string) : Promise<UserQuotaData> + updateUserQuota(userId : string, dailyDeletes : number, lastDeleteDate : Date) : Promise<void>	

Table 1. **Operations**

#	Name	Return type	Description
1	findUserQuota(userId)	Promise	null>
2	updateUserQuota(userId, dailyDeletes, lastDeleteDate)	Promise<void>	Overwrites the user's dailyDeletes and lastDeleteDate

Parameter Details for updateUserQuota:

- userId: string — Target user ID
- dailyDeletes: number — New deletion count to persist
- lastDeleteDate: Date — UTC midnight of today

4.4.3.94 Class "DailyQuotaRepository"

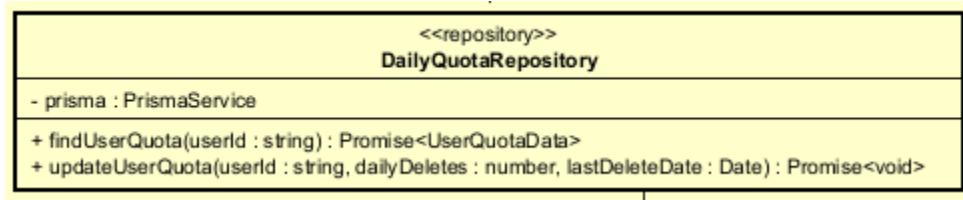


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	prisma	PrismaService	—	Prisma ORM client

Table 2. **Operations**

#	Name	Return type	Description
1	findUserQuota(userId)	Promise	null>
2	updateUserQuota(userId, dailyDeletes, lastDeleteDate)	Promise<void>	Updates the User table with the new quota values

4.4.3.95 Interface "IProductLogService"

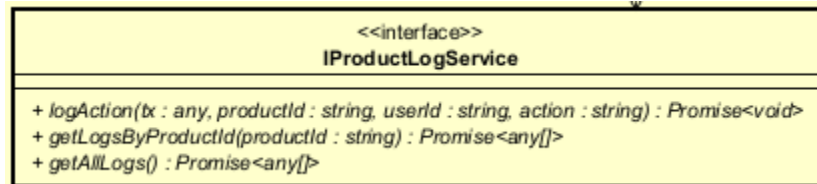


Table 1. **Operations**

#	Name	Return type	Description
1	logAction(tx, productId, userId, action)	Promise<void>	Records an audit log entry for the given action
2	getLogsByProductId(productId)	Promise<any[]>	Retrieves all log entries for a specific product
3	getAllLogs()	Promise<any[]>	Retrieves all product audit log entries in the system

Parameter Details for logAction:

- tx: Prisma TransactionClient (optional) — used when logging must occur within an existing transaction
- productId: string — ID of the target product
- userId: string — ID of the user performing the action
- action: string — Action name: 'CREATE', 'UPDATE', 'DELETE', 'DEACTIVATE', 'UPDATE_IMAGE'

4.4.3.96 Class "ProductLogService"

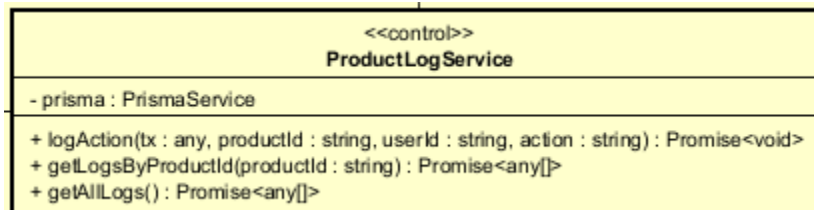


Table 1. **Attributes**

#	Name	Data type	Default value	Description
1	prisma	PrismaService	—	Prisma ORM client

Table 2. **Operations**

#	Name	Return type	Description
1	logAction(tx, productId, userId, action)	Promise<void>	Creates a ProductLog record in the DB
2	getLogsByProductId(productId)	Promise<any[]>	Queries logs by productId; includes the createdBy user object
3	getAllLogs()	Promise<any[]>	Queries all logs; includes product and user info; ordered by createdAt DESC

Method Details:

logAction(tx, productId, userId, action)

- **Parameters:**

tx: Prisma transaction client or undefined (falls back to this.prisma)

productId: string, userId: string, action: string

- **Special Logic:** If userId is falsy/empty → skip silently; no log is written (prevents errors when no authenticated user is present)
- **Exception:** Throws a Prisma error if productId or userId do not exist in the DB

getLogsByProductId(productId)

- **Parameters:** productId: string
- **Return:** Array of log objects including createdBy: { id, userName, email }, ordered by createdAt DESC
- **Note:** BigInt fields are converted to strings via JSON.parse(JSON.stringify(..., BigInt → string))

getAllLogs()

- **Return:** Array of all log objects including product: { id, title, barcode } and createdBy: { id, userName, email }, ordered by createdAt DESC
- **Note:** Same BigInt serialization as getLogsByProductId

5.Design Considerations

<Describe issues which need to be addressed or resolved before attempting to devise a complete design solution. Remember that, you have to refactor your source code to strictly follow the final design>

5.1 Goals and Guidelines

5.1.1 Design goals

- **High Performance:** The system prioritizes swift responses to user requests, aiming for interactions within 2 seconds under normal load and 5 seconds during peak traffic. This is crucial for a smooth e-commerce experience and user satisfaction, as slow responses can lead to abandonment.
- **Scalability:** AIMS is designed to support up to 1,000 concurrent users seamlessly. This goal is vital for accommodating future growth and ensuring system stability as the user base expands. Achieved through modular architecture, efficient resource management, and load balancing
- **Reliability and Availability:** The system is engineered to remain operational for at least 300 hours continuously and recover within 1 hour following any downtime. This minimizes service interruptions and maintains user trust, particularly important for an e-commerce platform where transactions are frequent.
- **Security:** Strong security practices are integrated throughout the design, including encrypted data storage, secure user authentication (e.g., JWT), role-based access control, and input validation. This is paramount to protect sensitive user data, payment information, and maintain compliance.
- **User-Friendliness (Intuitive UI/UX):** The user interface is designed to be simple, intuitive, and accessible for non-technical users. This focus ensures a positive user experience, reduces the learning curve, and encourages engagement with the platform.
- **Maintainability:** The system is designed with clear separation of concerns (e.g., Client-Server architecture) and adherence to design principles (e.g., SOLID) to simplify future updates, bug fixes, and feature additions, reducing long-term development costs and efforts.
- **Extensibility:** The architecture is modular to support the addition of new product types, payment methods, and external integrations without significant overhauls. This ensures the system can adapt to evolving business needs and technological Advancements.

5.1.2 Guidelines

- **Maintainability:** The system should be organized into clear modules so that each feature, such as product management, ordering, payment, authentication, and administration, can be developed and maintained independently. This reduces complexity and makes future updates easier.
- **Separation of Concerns:** The frontend, backend, database access, and external service integrations should have clearly separated responsibilities. This helps developers understand the system structure and reduces the risk of unintended side effects when modifying a feature.
- **Consistency:** Coding style, API structure, naming conventions, and user interface behavior should remain consistent across the whole application. This improves readability, simplifies collaboration, and makes the system easier to extend.
- **Usability:** The application should provide simple and predictable workflows for users, especially when browsing products, managing the cart, entering delivery information, placing orders, and completing payments. Clear feedback should be provided after each important action.
- **Reliability:** Order placement and payment processing must be handled carefully to prevent duplicated orders, incorrect payment status, or inconsistent transaction data. The system should prioritize correctness in all business-critical operations.
- **Security:** The system should validate user input, protect sensitive data, enforce authentication and authorization, and store secret configuration values securely. This helps prevent common security risks and protects user information.
- **Scalability:** The design should allow new features, payment methods, product types, and management functions to be added with minimal changes to existing modules.
- **Responsiveness:** The frontend should support different screen sizes and provide a stable user experience on both desktop and mobile devices.
- **Error Handling:** The system should handle errors gracefully and provide meaningful feedback to users while keeping technical details available for debugging by developers.
- **Testing and Quality Assurance:** Important business functions, especially product browsing, order placement, payment, authentication, and administration, should be tested to reduce regression risks and improve system stability.

5.2 Architectural Strategies

5.2.1 Monorepo and Technology Stack

The whole system lives in a single Nx **monorepo** containing three projects: apps/api (backend), apps/client (frontend), and apps/api-e2e (end-to-end tests). The backend is built with **NestJS 11** on **TypeScript**, the frontend with **Angular 21**, and the build pipeline uses **Webpack** + **SWC** with shared **ESLint/Prettier** config. *Reasoning*: a monorepo lets the two apps share types, tooling, and CI while Nx's task caching keeps builds fast; the trade-off (a larger repository) is acceptable for a single coordinated team.

5.2.2 Layered + Hexagonal (Ports & Adapters) Architecture

The backend's core principle is **Dependency Inversion**: business modules (e.g. place-order, payment) are split into domain/ (entities, services, **ports** = interfaces, states, factories, validators), application/ (use-case orchestration / Facade), and infrastructure/ (adapters, repositories). High-level logic depends only on **port interfaces wired through NestJS DI tokens** (ORDER_REPOSITORY, SHIPPING_FEE_CALCULATOR, PLACE_ORDER_PAYMENT_PORT, ...), never on concrete frameworks. *Reasoning / trade-off*: this is what gives the system **loose coupling, high cohesion, testability, and OCP extensibility**; the cost is more indirection (more small classes), which we judged worthwhile for maintainability.

5.2.3 RESTful API and DTO-based Contracts

Communication between the Angular client and the NestJS backend is via a **stateless RESTful API**. Each NestJS controller exposes resource endpoints (e.g. POST /place-order/check-stock, /delivery-info, /payment, /confirm) and accepts/returns **DTOs validated declaratively with class-validator + class-transformer**. *Reasoning*: DTOs decouple the public API contract from internal domain/persistence models and centralise input validation at the boundary.

5.2.4 Frontend – Angular Single-Page Application

The UI (apps/client) is an **Angular 21 component-based SPA** (standalone components, SCSS styling). It holds no business logic — it renders state and calls the REST API. *Reasoning*: a component model maximises reuse and a clear input/output (request/response) interaction model with the backend.

5.2.5 Persistence and Data Management

Data is stored in **PostgreSQL**, accessed through **Prisma 7 using the driver-adapter pattern** (@prisma/adapters-pg over a pg connection Pool). A single PrismaService wraps the generated client, and the **Repository pattern** ensures *only* repository adapters touch

Prisma — the domain receives normalised read-models (plain number/string), so Prisma Decimal/BigInt types never leak inward. Multi-row writes (order + lines + invoice + stock + payment transaction) run inside a single **\$transaction** for atomicity. *Reasoning / trade-off*: the driver-adapter + repository indirection keeps the domain database-agnostic and swappable, at the price of an extra mapping layer.

5.2.6 Security and Authentication

Authentication is **stateless JWT** via Passport (@nestjs/passport, passport-jwt, jwt.strategy.ts); passwords are hashed with **bcrypt**; an **OTP service** supports sensitive flows. Cookies are parsed via cookie-parser, and secrets/keys are kept out of code in environment variables. *Reasoning*: JWT enables scalable, server-side-stateless sessions consistent with the REST design.

5.2.7 Payment Gateway Integration (pluggable)

Payments are integrated through an abstraction layer: a **payment-gateway.port.ts** (**IPaymentGateway**) with a **payment-gateway.factory.ts** that selects the concrete gateway — **VietQR** (QR generated with qrcode, default method) or **PayPal** (REST via axios) — implemented as separate vietqr/ and paypal/ adapter modules. The place-order and payment modules integrate only through ports (PlaceOrderPaymentPort), avoiding a circular module dependency. *Reasoning*: this is a **Strategy/Adapter** design so a new gateway is a new adapter, not a change to existing code (the problem statement’s “other gateways may be supported in the future”).

5.2.8 Notifications and Communication

Order notifications use the **Observer pattern**: a confirmed order publishes an event to registered observers. Today an **EmailNotificationObserver** sends the invoice via the mail module (**nodemailer / Resend**, HTML templates); SMS/Zalo/Push can be added as new observers without touching the order flow. *Reasoning*: decouples “an order was paid” from “how the customer is notified” (OCP).

5.2.9 Error Detection and Recovery

Errors are separated by layer: **pure domain errors** (DomainError, IllegalOrderTransitionError) carry no HTTP/framework dependency, while application/infrastructure raise NestJS HTTP exceptions (InvalidQuantityException, InvalidDeliveryInfoException, PaymentNotSuccessfulException). Recovery mechanisms include **idempotent payment confirmation** (look up an existing transaction before creating an order), a **State-pattern guard** on illegal transitions, and a **stale-payment-transaction cleanup service** that releases orphaned

PENDING_PAYMENT orders. *Reasoning*: keeps the domain testable and makes the payment flow resilient to retries/abandonment.

5.2.10 Concurrency and Synchronization

The system is **stateless at the service layer** (no shared mutable in-memory state) and pushes concurrency control to **atomic PostgreSQL operations**. The PENDING_PAYMENT → PENDING_PROCESSING transition is claimed exactly once with a status-guarded updateMany, and stock is decremented with a **conditional update (quantity >= requested)** so concurrent orders and duplicate payment callbacks can never oversell or double-decrement. *Reasoning / trade-off*: relying on the database for synchronisation keeps services horizontally scalable (matching the “1,000 concurrent customers” requirement) instead of using in-process locks.

5.2.11 File and Asset Management

Product images are uploaded with **Multer** and stored in **Cloudinary** (external object storage) rather than in the database or app server. *Reasoning*: offloads large binary storage/CDN delivery to a specialised service and keeps the relational DB lean.

5.2.12 Configuration and Environment Management

All environment-specific values — DATABASE_URL/DIRECT_URL, CLIENT_URL, payment-gateway keys, mail keys — are externalised via **@nestjs/config + dotenv**, never hard-coded. Pricing data (e.g. DEFAULT_SHIPPING_FEE_CONFIG) is also injected, not embedded. *Reasoning*: supports clean deployment across dev/staging/production and per-environment overrides.

5.2.13 Testing and Quality Assurance

Quality is enforced with **Jest** unit tests (apps/api/jest.config) and a separate **e2e** project (apps/api-e2e), using **@nestjs/testing** for DI-based mocking. *Reasoning*: the Ports & Adapters design lets domain logic be tested with in-memory fakes (e.g. new OrderFactory()) without a real database or HTTP server.

5.2.14 Extensibility and Future Plans (OCP)

The architecture is designed for the AIMS “may be supported in the future” requirements: a new **shipping rule**, **notification channel**, **payment gateway**, or **product type** is added by implementing the relevant port and changing **one DI binding** — existing code is untouched. *Reasoning*: this Open–Closed extensibility was the primary design goal that justified the Hexagonal layering chosen in 5.2.2.

5.3 Design and Program Evaluation

5.3.1 Cohesion & SRP

Explain in detail for each class in the table.

Table 4. Cohesion & SRP of AIMS

#	Class Name	PIC	Cohesion	SRP	Solution
1	PaymentPage	TuNHT, BinhNT	Functional cohesion. It only acts as the payment page boundary: receives user actions, displays payment state, and delegates workflow actions to PaymentPageFacade.	Yes. It does not call backend business logic, database code, or provider APIs directly.	No change needed. Keep UI display and event forwarding only.
2	PaymentPageFacade	TuNHT, BinhNT	Sequential/functional cohesion. It coordinates the payment page workflow: initialize page, select payment method, confirm order, cancel order, and route to result/cart pages.	Yes. It coordinates client-side flow but does not create gateway payments or mutate backend state directly.	If more payment methods are added, keep using separate flow services instead of adding provider-specific logic here.
3	PaymentServiceClient	TuNHT, BinhNT	Functional cohesion. It is an Angular HTTP client for payment endpoints such as requestPayment,	Yes. It only serializes DTOs and calls REST endpoints; it	requestCustomerRefund() may stay here because refund is part of the

			confirmTransaction, and cancelTransaction.	contains no business rules.	payment API boundary.
4	PaymentController	TuNHT, BinhNT	Functional cohesion. All methods expose payment endpoints: request, confirm, cancel, refund, and transaction lookup.	Yes. It only receives HTTP requests and delegates to PaymentService. It does not query Prisma or call gateway providers directly.	If refund grows into a larger domain, split RefundController; otherwise no change needed.
5	PaymentService	TuNHT, BinhNT	Sequential cohesion. It coordinates ordered steps in the payment lifecycle: validate amount, create/reuse transaction, cancel other pending transactions, select gateway, update provider data, confirm, cancel, and refund.	Mostly yes. It is an application service, so orchestration is its accepted responsibility. The risk is that it can become too large.	The design already extracts transaction mutation, completion, gateway selection, and reference resolution to separate classes. If refund logic grows, create a dedicated RefundService.

6	PaymentTransactionService	TuNHT, BinhNT	Functional cohesion. It only handles payment transaction state changes: create/reuse pending, cancel other pending transactions, mark failed, and mark success.	Yes. It does not know provider APIs or UI details.	No change needed.
7	PaymentGatewayFactory	TuNHT, BinhNT	Functional cohesion. It only resolves payment gateways by method and capability: creation, confirmation, and refund.	Yes. It does not execute payments; it only returns the proper adapter.	No change needed. This is the main OCP extension point.
8	PaymentCompletionService	TuNHT, BinhNT	Functional cohesion. It only completes a successful payment: marks the transaction successful, cancels other pending transactions, and marks the order as pending processing.	Yes. It separates post-success completion from PaymentService.	No change needed.
9	PaymentGatewayTransactionRefResolver	TuNHT, BinhNT	Functional cohesion. It only resolves the transaction reference that should be sent to the gateway.	Yes. It validates and returns a reference; it does not perform capture/refund.	No change needed.

10	PaymentSessionStorageService	TuNHT, BinhNT	Functional cohesion. It only stores, loads, and clears client-side payment session data.	Yes. It does not call the API or apply business rules.	No change needed.
11	PaypalPaymentPageFlowService	TuNHT	Functional cohesion. Manages only the PayPal page-level flow: start checkout, handle return, capture payment, show errors.	Yes. Does not know PayPal HTTP endpoints or DB schema.	No change needed. Extract a presenter if UI-state handling grows.
12	PaypalPaymentFlowService	TuNHT	Functional cohesion. Groups PayPal client operations: create, capture, save/load/clear session.	Yes. Bridges page flow, API client, and session storage only.	No change needed.
13	PaypalGatewayAdapter	TuNHT	Functional cohesion. Adapts PayPal provider operations to the generic payment gateway interfaces.	Yes. Does not handle Angular HTTP requests or mutate Prisma records directly.	No change needed.
14	PaypalService	TuNHT	Functional cohesion. Communicates solely with the PayPal HTTP API: auth, create, capture, refund.	Yes. Does not know AIMS order state, payment transactions, or UI sessions.	Split into separate auth/order/refund clients if the API scope grows.

15	VietQRPaymentPageFlowService	BinhNT	Sequential/functional cohesion. It manages the VietQR page flow: create/resume QR payment, update QR image, confirm current payment, handle expired/failed status, and complete successful payment.	Mostly yes. It coordinates UI state and VietQR page-level workflow, but it also contains QR image update and error handling logic.	Extract QR image update or UI error-message handling into smaller helpers if the page flow keeps growing
16	VietQRPaymentFlowService	BinhNT	Functional cohesion. It owns VietQR client-side payment operations: request payment, store snapshot, poll latest transaction status, resume/clear snapshot, and trigger sandbox callback	Mostly yes. It focuses on VietQR flow logic, but polling and sandbox callback triggering are both handled here.	Move sandbox callback triggering into a dedicated VietQrSandboxCallbackFlowService if confirmation logic becomes more complex
17	PaymentStatusApiService	BinhNT	Functional cohesion. It wraps payment status endpoints used by the client, including latest transaction lookup and VietQR sandbox callback request	Mostly yes. It only performs HTTP calls, but it mixes general payment status lookup with VietQR-specific sandbox callback API	Split latest transaction lookup and VietQR sandbox callback API into separate services if more status or sandbox endpoints are added

18	QrImageService	BinhNT	Functional cohesion. It only converts VietQR QR source data into an image URL for display.	Yes. It does not know payment state, order state, or backend APIs.	No change needed.
19	VietqrGatewayAdapter	BinhNT	Functional cohesion. It adapts VietQR QR creation to the generic payment gateway interface	Yes. It builds VietQR payment output from VietqrService and does not mutate Prisma records directly.	No change needed. This is a clean adapter between generic payment flow and VietQR.
20	VietqrController	BinhNT	Functional cohesion. It exposes VietQR sandbox-related endpoints: transaction sync callback and test callback	Mostly yes. It delegates business logic, but it groups callback sync and sandbox test callback in the same controller.	If sandbox endpoints expand, create a VietqrSandboxController and keep this controller focused on callback sync
21	VietqrService	BinhNT	Functional cohesion. It handles VietQR QR generation: validates amount, builds VietQR request, gets token, calls the VietQR client, and maps QR response data.	Mostly yes. It is focused on QR generation, but it also maps provider response into internal QR data.	Extract response mapping into a VietqrQrResponseMapper if VietQR response handling becomes more detailed.

22	Vietqr SandboxService	BinhNT	Sequential cohesion. It validates that a pending VietQR transaction exists, fills sandbox callback defaults, gets an access token, and requests the VietQR test callback.	Mostly yes. It is focused on sandbox callback simulation, but includes pending transaction validation.	Extract pending VietQR transaction validation into a helper/service if other sandbox actions reuse the same validation.
23	Vietqr CallbackService	BinhNT	Sequential cohesion. It processes VietQR callback steps: validate transaction type, detect duplicates, find matching VietQR transaction, check amount, and complete successful payment	Mostly yes. Callback processing is its responsibility, but duplicate lookup and transaction matching make the service relatively dense.	Extract duplicate-callback lookup and transaction matching into small resolver methods/classes if callback rules grow.
24	Vietqr InboundController	BinhNT	Functional cohesion. It exposes only the VietQR inbound token generation endpoint	Yes. It only receives the Authorization header and delegates token generation to VietqrInboundService	No change needed.

25	Vietqr InboundService	BinhNT	Functional cohesion. It handles VietQR inbound authentication: parse Basic Auth, validate credentials, generate callback token, and validate callback bearer token.	Mostly yes. It stays inside inbound auth, but token signing, token validation, and Basic Auth parsing are all in one class.	Extract token signing/verification into a small token service if more inbound auth logic is added.
26	Vietqr Token Service	BinhNT	Functional cohesion. It manages VietQR outbound access tokens: fetch, cache, refresh, clear, and retry after auth failure.	Yes. It does not generate QR codes or process callbacks directly	No change needed.
27	Vietqr HttpClient	BinhNT	Functional cohesion. It communicates only with VietQR HTTP endpoints: token generation, QR generation, and sandbox test callback	Yes for current sandbox size. It contains provider HTTP details only, with no AIMS business rules.	Keep as is. Split into smaller clients only if the VietQR API surface becomes much larger
28	Vietqr QrRequest Builder	BinhNT	Functional cohesion. It only builds VietQR QR generation requests from payment input and VietQR config.	Yes. It does not call HTTP APIs or update payment state.	No change needed.

29	Vietqr Callba ckAut hGuar d	BinhNT	Functional cohesion. It only validates VietQR callback bearer-token authentication before allowing callback sync.	Yes. It does not process payment data or call provider APIs.	No change needed.
30	Vietqr Callba ckRes ponse Mapp er	BinhNT	Functional cohesion. It only maps VietQR callback results/errors into response DTOs.	Yes. It has no business workflow or persistence responsibility.	No change needed.
31	Vietqr Confi gServi ce	BinhNT	Functional cohesion. It centralizes VietQR sandbox configuration: base URL, credentials, QR config, callback defaults, and inbound auth env values.	Mostly yes. It is a configuration boundary, but it contains several groups of VietQR config.	No change needed.
32	Place Order Contr oller	Khanh NT	Functional – every handler only exposes an endpoint and delegates to the service	Yes – one reason to change: the HTTP boundary of place-order	None
33	Place Order BeSer vice	Khanh NT	Functional/Communicational – all methods coordinate one ordering workflow and share the same collaborators	Yes – it only orchestrates; no DB access / VAT math / e-mail of its own	None

34	Order Validation Service	Khanh NT	Communicational (near Logical) – groups three different kinds of validation	No – more than one reason to change: cart rules vs. payment rules	Extract normalizeItems → CartItemValidator and validatePaymentInfo → PaymentInfoValidator; delivery validation is already delegated to DeliveryInfoValidator
35	Stock Checker Service	Khanh NT	Functional	Yes – one reason: stock-checking rules	None
36	Invoice Calculator	Khanh NT	Functional	Yes – one reason: invoice/VAT pricing (shipping is delegated)	None
37	Product Controller	Nghia T	Functional: only routes HTTP requests for the product API	Yes: single responsibility = expose product read endpoints.	No change needed (keep thin).

38	GetProductDetailUseCase	NghiaTT	Functional: orchestrates the single flow of retrieving a product detail.	Yes: one business operation.	None.
39	ProductDetailMapper	NghiaTT	Functional/ Communicational: all methods transform one record into DTOs.	Yes: only persistence → DTO mapping.	None.
40	PrismaProductRepository	NghiaTT	Functional: implements product read queries.	Yes: read-side persistence only.	Keep command ops in a separate repository (done).
41	ProductDetailPageComponent	NghiaTT	Functional (mostly): reads route id, manages load state, renders.	Partial: also handles error + navigation.	Delegate rendering to child components (done).
42	ProductDetailComponent	NghiaTT	Communicational: renders detail fields and owns add-to-cart quantity.	Partial: display + add-to-cart.	Cart state delegated to CartStoreService.

43	ProductApiService	NghiaTT	Functional: centralizes product read HTTP calls.	Yes.	None.
44	ProductController	HieuBT	Functional cohesion. All methods serve one purpose: expose HTTP endpoints for the product CUD API and delegate to the service layer. No business logic, no Prisma calls, no gateway interaction exists here.	Yes. Its single reason to change is when the product HTTP API surface changes (new endpoints, route structure, or auth guards). It does not encode rules about deletion limits, price ratios, or DB schema.	No change needed. Keep the controller thin: receive request → delegate → return response. If the product surface grows significantly (e.g., a dedicated inventory API), split into ProductCudController and ProductQueryController.
45	ProductService	HieuBT	Sequential cohesion. Methods execute ordered steps through the product lifecycle: validate business specifications → persist via repository → record audit log → enforce/increment deletion quota. The output of one step feeds the next.	Mostly yes. As an application-layer orchestrator, coordinating multiple domain services is its accepted responsibility. The risk is that it can become too large if more lifecycle events are added.	The design already extracts quota enforcement, audit logging, and business rule validation into separate injected services. If the deletion flow grows more complex (e.g., order-aware rollback logic), extract a dedicated ProductDeletionService.

46	Product Repository	HieuBT	Functional cohesion. All methods perform a single concern: read or write the Product aggregate to the database via Prisma. Type-specific persistence is delegated to IProductHandler, keeping this class focused on base-product CRUD only.	Mostly yes. Its single reason to change is when the base product DB schema changes. The serializeBigInt utility method is a minor SRP deviation — it is a serialization concern, not a persistence concern. The checkInOrders method crosses domain boundaries (it queries the Order domain).	Extract serializeBigInt into a shared BigIntSerializer utility. Consider moving checkInOrders to an OrderRepository or a cross-domain query service to respect domain boundaries.
47	BookHandler	HieuBT	Functional cohesion. All logic in this class pertains exclusively to persisting Book-specific product data: creating PrintableProduct, Book, and Author/BookAuthor records, and updating them in isolation.	Yes. Its single reason to change is when the Book product's DB schema or Author upsert rules change. It does not know about CD, DVD, or Newspaper structure.	No change needed. If Author management grows complex (e.g., deduplication rules, author profiles), extract an AuthorRepository to isolate that logic from the handler.
48	CdHandler	HieuBT	Functional cohesion. All logic pertains exclusively to DiscProduct and CD record persistence.	Yes. Single reason to change: CD-specific DB schema changes.	No change needed.
49	DvdHandler	HieuBT	Functional cohesion. All logic pertains exclusively to DiscProduct and DVD record persistence.	Yes. Single reason to change: DVD-specific DB schema changes.	No change needed.

50	NewspaperHandler	HieuBT	Functional cohesion. All logic pertains exclusively to PrintableProduct and Newspaper record persistence.	Yes. Single reason to change: Newspaper-specific DB schema changes.	No change needed.
51	CreateProductDto	HieuBT	Communicational cohesion. All fields operate on the same data structure: the common attributes of any product being created. Fields are grouped because they all describe the same entity.	Yes. Single reason to change: when the set of common product fields changes (e.g., adding a sku field that all product types share).	No change needed. This is the correct base for type-specific DTOs to extend.
52	CreateBookDto / CreateCdDto / CreateDvdDto / CreateNewspaperDto	HieuBT	Communicational cohesion. Each DTO groups only the fields that belong to its specific product type.	Yes. Each DTO has one reason to change: when its specific product type's attributes change. They do not contain fields from other product types.	No change needed. This is the correct result of splitting the old God DTO.
53	UpdateProductDto	HieuBT	Logical cohesion. Fields from all four product types (Book, CD, DVD, Newspaper) are grouped together into a single class for no reason other than they all serve "update" operations. This is a God DTO.	No. It has four reasons to change: one for each product type. Adding a new field to the Book domain forces a change to this class, even though CD, DVD, and Newspaper are unaffected. This is a known technical debt.	Refactor needed. Split into UpdateBookDto, UpdateCdDto, UpdateDvdDto, and UpdateNewspaperDto following the same structure as the Create DTOs. The ProductService and ProductRepository should accept a union type or use the Strategy Pattern to

					delegate to the correct handler with the right DTO.
54	HardDeleteSpecification	HieuBT	Functional cohesion. The entire class exists to evaluate one business rule: can this product be hard-deleted? The single method <code>isSatisfiedBy</code> checks <code>quantity === 0</code> .	Yes. Single reason to change: when the hard-delete eligibility rule changes (e.g., "quantity must be 0 AND no pending returns").	No change needed. The Specification Pattern correctly isolates this rule.
55	PriceRatioSpecification	HieuBT	Functional cohesion. The entire class exists to evaluate one business rule: is the current price within the allowed ratio of the original value (30%–150%)?	Yes. Single reason to change: when the AIMS price ratio policy changes.	No change needed.
56	DailyQuotaService	HieuBT	Functional cohesion. All methods relate to a single concern: enforcing daily deletion quotas. Methods <code>checkAndDeleteQuota</code> , <code>checkBulkQuota</code> , <code>incrementQuota</code> , and <code>incrementBulkQuota</code> all operate on the same quota state. The private <code>calculateCurrentDeletes</code> is a pure helper for the same concern.	Yes. Single reason to change: when the quota business rules change (e.g., limit changes from 20 to 30, or a weekly quota is introduced). It does not know about Prisma, HTTP, or product domain details.	No change needed. This class was already refactored from a version that mixed business logic with direct Prisma access (see SOLID section below).
57	DailyQuotaRepository	HieuBT	Functional cohesion. All methods perform a single concern: read and write user quota fields (<code>dailyDeletes</code> , <code>lastDeleteDate</code>) in the database. It queries only the fields it needs, not the full User record.	Yes. Single reason to change: when the storage mechanism for quota data changes (e.g., migrating from PostgreSQL to Redis).	No change needed. This is a clean data access layer with no business logic.

58	UserQuotaData	HieuBT	Communicational cohesion. Both fields (dailyDeletes, lastDeleteDate) operate together to represent a user's quota state. They are inseparable in the context of quota calculation.	Yes. Single reason to change: when the data structure needed for quota calculation changes. It intentionally exposes only quota fields, not the entire User entity (ISP compliance).	No change needed.
59	ProductLogService	HieuBT	Functional cohesion. All methods relate to a single concern: audit logging of product operations. logAction writes a log; getLogsByProductId and getAllLogs read logs.	Mostly yes. Its primary responsibility is audit logging. However, mixing write (logAction) and read (getLogsByProductId, getAllLogs) operations means it has two reasons to change: when the log write format changes, and when the log query shape changes.	If the audit log querying requirements grow significantly (e.g., filtering, aggregation, export), extract a ProductLogQueryService for read operations and keep ProductLogService for write-only.
60	IProductHandler<T,U>	HieuBT	Functional cohesion. The interface defines exactly the operations needed to handle one product type's full lifecycle: check support, validate, create, and update.	Yes. Single reason to change: when the contract for what a product type handler must support changes (e.g., adding a delete method to the contract).	No change needed. Generic parameters T and U ensure type-safe DTO binding per implementation.
61	IProductLogService / IDailyQuotaService / IDaily	HieuBT	Functional cohesion. Each interface defines a single focused contract: log product events, manage quotas, access quota data, check a specification, and check a	Yes. Each interface has one reason to change: when its specific contract evolves. They are deliberately narrow	No change needed. These are the abstraction layer that allows the application core to be

	QuotaRepository / ISpecification / IProductSpecification		specification with a message, respectively.	to support DIP and ISP compliance.	independent of infrastructure and concrete implementations
62	WeightBasedShippingStrategy	Khanh NT	Functional – calculate + helpers (resolveZone, calculateRawShippingFee, calculateDiscount) all compute the shipping fee	Yes – one reason: the shipping-fee algorithm	None
63	OrderFactory	Khanh NT	Functional – buildCreateOrderInput, generateOrderCode both assemble the order-create input	Yes – one reason: how CreateOrderInput is assembled	None
64	OrderSuccessMapper	Khanh NT	Functional – fromDeliveryInfo, fromPersistedOrder both build OrderSuccessDto	Yes – one reason: the result-DTO shape	None
65	OrderEventPublisher	Khanh NT	Functional – publish only broadcasts the event to observers	Yes – one reason: the event-publishing mechanism	None
66	EmailNotificationObserver	Khanh NT	Functional – update only renders & sends the confirmation e-mail	Yes – one reason: the e-mail channel	None
67	OrderStateFactory	Khanh NT	Functional – create only maps a status to its state object	Yes – one reason: the set of valid states	None

68	BaseOrderState / PendingPaymentState / PendingProcessingState	Khanh NT	Functional – each class encapsulates the confirmPayment behaviour of exactly one state	Yes – each class = exactly one state	None
69	DeliveryInfoValidator	Khanh NT	Functional – the validator runs a set of rules; each rule (ReceiverNameRule, PhoneNumberRule, ...) validates exactly one field	Yes – each rule = one field; the validator = one job: run the rules	None
70	Prisma OrderRepository	Khanh NT	Functional – every method serves "order persistence" behind the IOrderRepository port (find*, create*, applyPaidTransition, stock decrement)	Yes – one reason: the persistence schema / Prisma	None
71	PlaceOrderPaymentAdapter	Khanh NT	Functional – getPaymentContext, markPaidAndPendingProcessing, notifyOrderConfirmed all bridge place-order ↔ payment	Yes – one reason: the payment-port contract	None

5.3.2 Coupling & Other SOLID

Explain in detail for each problem in the table. Each problem, you may provide a class diagram before and after the improvement.

Table 5. Coupling & other SOLID of AIMS

#	Problem	PIC	Location	Solution

1	Control coupling risk through paymentMethod. A weaker design would put if method == ... logic throughout the service layer.	TuNHT, BinhNT	PaymentService, gateway selection.	Gateway selection is centralized in PaymentGatewayFactory. Provider-specific behavior is moved into adapters. This reduces control coupling and improves OCP.
2	Stamp coupling risk if full persistence objects are passed into gateways. Gateways should not receive full Prisma models.	TuNHT, BinhNT	PaymentService to gateway adapters.	The current design passes focused DTOs such as PaymentGatewayContext and transaction references. This is data coupling, which is acceptable.
3	External coupling with PayPal API.	TuNHT	PaypalService	Unavoidable but isolated inside PaypalService. Other AIMS classes do not know the endpoints or payload details.
4	Content coupling risk if PayPal HTTP logic is placed in PaymentService.	TuNHT	PaymentService / PaypalGatewayAdapter	Solved by PaypalGatewayAdapter. PaymentService communicates strictly through generic gateway interfaces.
5	Temporal coupling caused by browser redirect.	TuNHT	PaypalPaymentPageFlow Service	Session persistence is isolated. Missing/expired

				sessions are handled as explicit error paths.
6	Stamp coupling risk with PayPal provider data.	TuNHT	PaypalGatewayAdapter, PaypalService	Converted to Data coupling: Only primitive data (amount, PayPal order id, capture id) is passed instead of full entity objects.
7	OCP risk when PayPal refund behavior differs from other providers.	TuNHT	Refund flow	PayPal implements Payment RefundGateway; providers without automatic refund omit it and fall back to manual handling.
8	External coupling with VietQR sandbox API. AIMS must call VietQR sandbox endpoints for token generation, QR generation, and test callback.	BinhNT	VietqrHttpClient, VietqrService, VietqrSandboxService	This external coupling is isolated inside VietqrHttpClient. Other classes do not know VietQR URLs, headers, or raw HTTP payload details. VietqrService and VietqrSandboxService communicate through VietqrClient and DTOs only.
9	Content coupling risk if VietQR request payload construction is placed directly in PaymentService or	BinhNT	VietqrGatewayAdapter, VietqrService, VietqrQrRequestBuilde	VietQR payload construction is extracted to VietqrQrRequestBuilder. PaymentService only sends generic payment data, while

	VietqrGatewayAdapter.			VietqrGatewayAdapter converts it to VietQR payment intent. This improves SRP and reduces content coupling.
10	Temporal coupling in VietQR sandbox payment flow. The client must create QR payment, poll transaction status, trigger sandbox callback, then check status again.	BinhNT	VietQrPaymentFlowService, VietQrPaymentPageFlowService	The ordered workflow is centralized in VietQrPaymentFlowService and VietQrPaymentPageFlowService instead of being spread across components. Snapshot storage and polling make the sequence explicit and easier to recover after refresh.
11	coupling risk through VietQR transaction status. UI behavior changes based on PENDING, SUCCESS, FAILED, and REFUND_REQUIRED.	BinhNT	VietQrPaymentPageFlowService, VietQrPaymentFlowService	Status-based behavior is kept inside VietQR flow services. The payment component does not contain large conditional logic for every transaction status. This keeps the UI boundary thinner and improves SRP.
12	Stamp coupling risk if full PaymentTransaction records are passed to VietQR callback response logic.	BinhNT	VietqrCallbackService, VietqrCallbackResponseMapper	The callback flow uses TransactionSyncDto and TransactionStatusDto instead of exposing full Prisma entities. VietqrCallbackResponseMapper maps only the fields

				needed by the callback response.
13	Content coupling risk if VietQR callback processing directly updates all order/payment fields.	BinhNT	VietqrCallbackService, PaymentCompletionService	VietqrCallbackService only validates and matches the callback, then delegates successful completion to PaymentCompletionService. This keeps callback-specific logic separate from general payment completion logic.
14	Data coupling with environment configuration. VietQR sandbox depends on many config values such as bank account, bank code, credentials, callback defaults, and token secret	BinhNT	VietqrConfigService	Configuration access is centralized in VietqrConfigService. Other VietQR classes request named config groups instead of reading process.env directly everywhere. If config grows, split it into outbound QR, inbound auth, and sandbox callback config groups.
15	Authentication coupling between VietQR callback guard and inbound token validation.	BinhNT	VietqrCallbackAuthGuard, VietqrInboundService	VietqrCallbackAuthGuard only extracts the bearer token and delegates validation to VietqrInboundService. This avoids duplicating token validation rules in

				the controller or callback service.
16	OCP risk when adding more sandbox provider actions. Adding new VietQR sandbox operations directly into PaymentStatusApiService can make the service too broad.	BinhNT	PaymentStatusApiService	Current design is acceptable for the existing sandbox flow. If more sandbox actions are added, split VietQR sandbox callback calls into a dedicated VietQrSandboxApiService while keeping PaymentStatusApiService for transaction lookup only
17	ISP risk because PaymentStatusApiService exposes both general payment status lookup and VietQR-specific sandbox callback request.	BinhNT	PaymentStatusApiService	The current interface is still small, but clients that only need status lookup can technically access VietQR sandbox callback too. A cleaner design would separate PaymentTransactionStatus Api and VietQrSandboxCallbackApi if the client API grows.
18	DIP risk if services depend directly on concrete VietQR HTTP implementation.	BinhNT	VietqrService, VietqrSandboxService, VietqrHttpClient	The code uses the VietqrClient interface/token instead of directly depending on VietqrHttpClient. This follows DIP and makes the

				sandbox client replaceable in tests.
19	Control coupling risk in gateway behavior if PaymentService manually checks VietQR-specific QR behavior.	BinhNT	PaymentService, VietqrGatewayAdapter	VietQR-specific QR creation is handled inside VietqrGatewayAdapter. PaymentService uses generic gateway interfaces, so adding or changing VietQR behavior does not require spreading VietQR conditions through PaymentService.
20	Temporal coupling with token caching and retry. VietQR calls must use a valid access token and retry after auth failure.	BinhNT	VietqrTokenService, VietqrService, VietqrSandboxService	Token lifecycle is isolated in VietqrTokenService. VietqrService and VietqrSandboxService call withAccessTokenRetry(), so they do not need to duplicate token refresh logic
21	Control coupling (resolved → improved)	NghiaTT	ProductDetailMapper previously selected type-specific mapping with if/else on product type	Refactored to a Strategy map Record → data coupling + OCP.
22	DIP	NghiaTT	GetProductDetailUseCase ↔ persistence.	Depends on the ProductQueryRepository abstraction (tokeninjected), not on Prisma directly.

23	ISP	NghiaTT	ProductQueryRepository.	Exposes only read ops (findProducts, findProductById); command ops kept in separate abstractions.
24	OCP	NghiaTT	Adding a new product type.	Add a new specificInfo mapper + child component; existing dispatch logic is not modified.
25	Data coupling (good)	NghiaTT	controller → use case → repository → mapper.	Layers pass simple ids/DTOs, no shared global state.
26	Control coupling & OCP risk if product type specific logic (Book, CD, DVD) is hard-coded using if/switch throughout the service layer.	HieuBT	ProductService, ProductRepository	Product type handling is centralized using the Strategy Pattern via IProductHandler. The repository dynamically selects the handler. This reduces control coupling and improves OCP.
27	Content coupling & DIP risk if business logic accesses the database directly. Previously, DailyQuotaService contained raw Prisma queries, mixing rules with data access.	HieuBT	DailyQuotaService	Extracted DB access to IDailyQuotaRepository and injected it. DailyQuotaService now depends on abstractions (DIP) and passes focused DTOs (Data Coupling).

28	DIP risk & Content coupling if services depend directly on concrete rule classes. ProductService previously injected the concrete HardDeleteSpecification class.	HieuBT	ProductService, HardDeleteSpecification	Injected the specification via the HARD_DELETE_SPECIFICATION DI token typed as ISpecification<any>. The service now depends on an abstraction.
29	Common coupling & SRP risk due to a God DTO. UpdateProductDto groups fields from all four product types, forcing handlers to see irrelevant fields.	HieuBT	UpdateProductDto, Handlers	Pending Refactor: Split the God DTO into UpdateBookDto, UpdateCdDto, etc., to achieve Data Coupling and satisfy ISP/SRP for handlers.
30	Content coupling at infrastructure boundary. Handlers depend directly on the Prisma transaction internals and exact nested schema.	HieuBT	BookHandler, CdHandler, DvdHandler, NewspaperHandler	Accepted trade-off. Handlers operate specifically at the infrastructure boundary to bridge DTOs and the ORM. This prevents over-engineering.
31	Cross-domain coupling. Product persistence layer directly queries the Order domain to check if a product can be deleted.	HieuBT	ProductRepository.checkInOrders()	Pending Refactor: Move the order checking logic to an OrderRepository or a dedicated cross-domain query service to respect domain boundaries.
32	External coupling with image hosting API. ProductController needs to upload	HieuBT	ProductController, CloudinaryService	Isolated in CloudinaryService. The controller only passes the file object and receives a

	images but should not know the HTTP/API details.			URL result (Data coupling), ensuring SRP.
33	OCP & SRP risk for product validation rules. Business rules (e.g., price ratio) might change frequently, risking bloated validation methods.	HieuBT	ProductService.validateProductSpecifications()	Implemented Specification Pattern. ProductService injects IProductSpecification[] and loops through them. Adding rules requires no changes to the service.
34	ISP risk if quota tracking exposes the entire User entity to the business layer.	HieuBT	IDailyQuotaRepository, DailyQuotaService	Created UserQuotaData DTO containing only dailyDeletes and lastDeleteDate, ensuring the business logic only receives what it needs.
35	ISP violation	KhanhNT	CartItemDto — consumed by PlaceOrderController, OrderValidationService.normalizeItems, StockCheckerService.check. Fields price and weight are never read by the serve	Remove price and weight; keep only productId + quantity (the cart's real intent). If a future use case (e.g., a client-side price preview) needs them, define a separate DTO instead of widening the shared one.
36	Stamp coupling	KhanhNT	CartItemDto is a composite object passed to OrderValidationService.normalizeItems(items) and StockCheckerService.check(items, ...), but only	Pass only the elementary fields actually required, or trim CartItemDto.. This drops the coupling to Data coupling.

			productId/quantity are used → unused fields ride along.	
37	SRP violation	KhanhNT	OrderValidationService — validates three different inputs: cart items (normalizeItems), delivery info (validateDeliveryInfo*), and payment info (validatePaymentInfo). More than one reason to change.	Split into CartItemValidator (normalizeItems) and PaymentInfoValidator (validatePaymentInfo); keep delivery validation delegated to DeliveryInfoValidator
38	DIP	KhanhNT	PlaceOrderBeService (high-level Facade) → injects the concrete PaymentService from the payment module, not an abstraction.	Define a payment port interface (e.g. IPaymentGateway with requestPayment(...)) and inject it via a DI token, so the facade depends on the abstraction (consistent with how IOrderRepository/IShippingFeeCalculator are already done).
39	Control coupling	KhanhNT	OrderValidationService.collectDeliveryErrors(deliveryInfo, requireEmail: boolean) — a boolean flag selects behaviour (whether to add the “email required” error).	Remove the flag: split into two private helpers (collectDeliveryErrorsBasic / ...ForOrder), or compose the email rule into the rule list.

				Eliminates the flag parameter.
--	--	--	--	--------------------------------

5.4 Design Patterns

5.4.1 Payment

5.4.1.1 Strategy Pattern

Why it is used

Different payment methods have different algorithms for payment creation, confirmation, and refund.

- PayPal creates a checkout order, redirects the customer, then captures the approved order.
- VietQR generates a QR code and is confirmed by callback.
- Some methods support automatic refund, while others require manual refund.

If all of these behaviors were implemented directly inside PaymentService with many if/else or switch(paymentMethod) branches, PaymentService would become tightly coupled to every provider and violate OCP when a new payment method is added.

Concrete strategies implement the capabilities they support:

- PaypalGatewayAdapter: create, confirm/capture, refund
- VietqrGatewayAdapter: create QR payment; confirmation is handled by callback

PaymentService uses the selected strategy without knowing provider details.

Benefit

Adding a new method such as Momo or ZaloPay mainly requires adding a new gateway adapter. The generic payment lifecycle remains stable.

5.4.1.2 Factory Pattern

Why it is used

The system must select the correct gateway at runtime based on payment method and required operation: create, confirm, or refund.

Benefit

Gateway selection is centralized. This reduces control coupling and supports OCP because new providers can be registered without rewriting payment orchestration logic.

5.4.1.3 Adapter Pattern

Why it is used

External providers expose different APIs and data formats. The generic payment module should not depend directly on PayPal or VietQR request/response structures.

Benefit

Provider-specific details are isolated. If PayPal or VietQR changes its API, only the adapter/provider service needs to change, not PaymentService or PaymentController.

5.4.2 View Product Detail

5.4.2.1 Strategy Pattern

Problem

The product detail must show different fields for Book, Newspaper, CD and DVD. A naive design uses a long if/else (or switch) on the product type in the mapper and in the UI – this creates control coupling and violates OCP because every new product type forces edits to existing code.

Solution

Each product type is handled by its own strategy. On the backend, ProductDetailMapper.mapSpecificInfo uses a Strategy map Record ProductSpecificInfoDto> that selects the matching mapBookSpecificInfo / mapNewspaperSpecificInfo / mapCdSpecificInfo / mapDvdSpecificInfo. On the frontend, ProductSpecificInfoComponent dispatches to a dedicated Book/Newspaper/CD/DVD specific-info component.

Benefit

Adding a new product type means adding one new strategy (mapper + component) without modifying the existing dispatch – satisfying OCP and reducing coupling to data coupling.

5.4.2.2 Repository Pattern

The application layer depends on the `ProductQueryRepository` abstraction (read operations only) instead of `Prisma` directly. `PrismaProductRepository` is the concrete implementation injected via a DI token. This isolates persistence, supports DIP, and lets the data source be replaced without touching the use case.

5.4.1.3 DTO+Mapper Pattern

Persistence records are never exposed to the controller or the frontend. `ProductDetailMapper` converts them into a stable `ProductDetailDto` read contract, decoupling the API/UI from the database shape.

5.4.3 CUD Product

5.4.3.1 Strategy Pattern

Why it is used: Different product types have different database structures and logic for creation and updating.

- Book creates a `PrintableProduct` and Book record, and must handle complex Author upsert and linking logic.
- CD creates a `DiscProduct` and CD record with tracklist details.
- Newspaper creates a `PrintableProduct` and Newspaper record.

If all of these behaviors were implemented directly inside `ProductRepository` with many if/else or switch(`productType`) branches, `ProductRepository` would become tightly coupled to every specific product type and violate OCP when a new product type is added.

Concrete strategies implement the persistence capabilities they support:

- `BookHandler`: create, update for Book database records.
- `CdHandler`, `DvdHandler`, `NewspaperHandler`: create, update for their respective records.

`ProductRepository` dynamically selects and uses the correct strategy at runtime based on the `ProductType` enum, without knowing type-specific schema details.

Benefit: Adding a new product type such as Vinyl or EBook mainly requires adding a new `IProductHandler` adapter. The generic product creation and update lifecycle in `ProductRepository` remains stable and untouched.

5.4.3.2 Specification Pattern

Why it is used: The system has complex and evolving business rules that must be evaluated before creating, updating, or deleting a product.

- Deletion rule: A product can only be hard-deleted if its stock quantity is 0; otherwise, it must be soft-deleted (deactivated).
- Pricing rule: A product's selling price must remain within 30% to 150% of its original value.

If all of these business validations were written directly inside ProductService as procedural if/else checks, the service would become a bloated God Object, tightly coupling application orchestration with volatile business rules.

Concrete specifications implement the evaluation logic:

- HardDeleteSpecification: checks if quantity == 0.
- PriceRatioSpecification: calculates and checks if currentPrice is valid against originalValue.

ProductService relies on injected specifications without knowing their internal logic (e.g., iterating through an array of validations or checking the hard-delete rule).

Benefit: Business rules are isolated and decentralized. If AIMS changes its pricing policy or adds a new validation rule (e.g., weight limits or barcode formats), developers only need to create a new Specification class and register it. ProductService orchestration logic remains completely unchanged, significantly improving OCP and SRP.

5.4.4 Place Order

5.4.4.1 Strategy - shipping-fee calculation

Intent: Define a family of interchangeable algorithms, encapsulate each one, and make them selectable at runtime.

Motivation (problem): The delivery fee depends on weight, zone, and free-ship rules, and the problem statement explicitly says new rules may be supported in the future (e.g., volumetric weight). If the fee formula were hard-coded inside the invoice logic, every rule change would force us to edit the ordering code — violating OCP and mixing two responsibilities.

Solution:

- Strategy interface: IShippingFeeCalculator with calculate(context: ShippingContext): ShippingBreakdown.
- Concrete strategy: WeightBasedShippingStrategy (22,000 VND first 3 kg for Hanoi/HCM, 30,000 VND first 0.5 kg elsewhere, +2,500 VND per extra 0.5 kg, free-ship discount).
- Context: InvoiceCalculator, which holds the strategy and only calls calculate(...) - it never knows the formula.

Implementation: The concrete strategy is bound to the SHIPPING_FEE_CALCULATOR token in PlaceOrderDomainModule via useClass. The pricing data (zones/fees) is injected separately through ShippingFeeConfig. To switch to a volumetric-weight strategy we add a new class implementing IShippingFeeCalculator and change one useClass line.

Pros: New shipping algorithm = new class, no edit to existing code (OCP).

5.4.4.2 Observer — order-confirmation notifications

Intent: Define a one-to-many dependency so that when one object (the subject) changes state, all dependents (observers) are notified automatically, while the subject stays decoupled from them.

Motivation (problem): When an order is successfully paid, the customer must receive the invoice by e-mail; the statement adds that SMS or push notifications may also be supported in the future. Calling the mail service directly from the ordering flow would hard-wire “order paid” to one specific channel and force edits whenever a channel is added.

Solution:

- Subject interface: IOrderEventPublisher with publish(event: OrderConfirmedEvent).
- Concrete subject: OrderEventPublisher, holding a list of INotificationObserver.
- Observer interface: INotificationObserver with update(event: OrderConfirmedEvent).
- Concrete observer: EmailNotificationObserver (renders the template and calls MailService).
- Event (DTO): OrderConfirmedEvent (recipient, order, invoice).

Implementation: Observers are registered through the NOTIFICATION_OBSERVERS provider; publish() iterates them in parallel and isolates failures (one failing channel is logged and does not break the others). Adding SMS = implement INotificationObserver and append it to the provider list.

Pros: The publisher knows nothing about concrete channels (loose coupling); new channels obey OCP; no fixed notification order is assumed.

5.4.4.3 State - order payment lifecycle

Intent: Allow an object to alter its behaviour when its internal state changes; the object appears to change its class, replacing if/else/switch over a status field.

Motivation (problem): A paid-callback may arrive once, twice, or out of order. Deciding “is it legal to mark this order paid?” with conditionals on a status string

(PENDING_PAYMENT, PENDING_PROCESSING, ...) becomes tightly coupled and error-prone as states grow.

Solution:

- State interface: `IOrderState` with `confirmPayment(): StateTransitionResult`.
- Abstract base: `BaseOrderState` — default `confirmPayment()` throws `IllegalOrderTransitionError`.
- Concrete states: `PendingPaymentState` (transition allowed $\rightarrow$ { `alreadyInTarget`: false }) and `PendingProcessingState` (already paid $\rightarrow$ { `alreadyInTarget`: true }, an idempotent no-op).

Implementation: In `PlaceOrderPaymentAdapter.markPaidAndPendingProcessing()`, the code does `orderStateFactory.create(order.status).confirmPayment()`. Each state encapsulates its own rule, so a duplicate payment callback safely becomes a no-op instead of decrementing stock twice.

Pros: No conditional logic on status; polymorphic, robust, and trivially extensible (add a state class); prevents inconsistent transitions.

5.4.4.4 Factory (Simple Factory)

Intent: Encapsulate the logic that decides which object to create / how to assemble it, hiding construction details from the caller.

Motivation: Mapping a status string to the right `IOrderState` should live in one place, not be duplicated; building the persistence input for an order (trim fields, map lines, generate code) used to be copy-pasted across `createPayment()` and `confirmOrder()`.

Solution:

- `OrderStateFactory.create(status)` $\rightarrow$ returns the correct `IOrderState` (the clearest Simple Factory; throws for unknown states).
- `OrderFactory.buildCreateOrderInput(invoice, deliveryInfo, status)` $\rightarrow$ assembles `CreateOrderInput` and privately generates the order code.
- `OrderSuccessMapper` $\rightarrow$ static factory that builds `OrderSuccessDto` from either raw form input or a persisted order.

Implementation: Each factory exposes a single build/create method and keeps the construction rules (generateOrderCode, field trimming, line mapping) private.

Pros: Removes duplicated construction code (DRY); centralises creation rules; callers depend only on the produced abstraction (e.g., `IOrderState`), not on concrete classes.

5.4.4.5 Adapter - integrating with payment and persistence

Intent: Convert the interface of a class into another interface that clients expect, so that otherwise-incompatible classes can work together.

Motivation: Two boundaries must be bridged without coupling: (1) the Payment module needs order data through its own contract, and (2) the domain must talk to Prisma without depending on Prisma's Decimal/BigInt types.

Solution:

- PlaceOrderPaymentAdapter implements PlaceOrderPaymentPort (the interface the Payment module expects). It adapts the place-order domain (repository, state, invoice, publisher) to that contract via `getPaymentContext()` / `markPaidAndPendingProcessing()`.
- PrismaOrderRepository implements IOrderRepository — it wraps PrismaService (the adaptee) and normalises DB types into plain read-models (ProductSnapshot, PersistedOrderDetail, ...). This is simultaneously the Repository pattern.

Implementation: Both adapters are bound behind DI tokens (`PLACE_ORDER_PAYMENT_PORT`, `ORDER_REPOSITORY`), so neither side sees the other's concrete class.

Pros: Payment and place-order evolve independently; the persistence technology can be swapped by writing a new adapter; the domain stays free of framework/DB types (DIP).

5.4.4.6 Facade

Intent: Provide a unified, simplified interface to a set of interfaces in a subsystem.

Motivation: Placing an order involves validation, stock checking, invoice/VAT/shipping calculation, persistence, payment, and notification. The REST controller should not orchestrate all of these collaborators directly.

Solution: PlaceOrderBeService is the facade: the controller calls four high-level operations (`checkStock`, `processDeliveryInfo`, `createPayment`, `confirmOrder`), and the facade coordinates OrderValidationService, StockCheckerService, InvoiceCalculator, OrderFactory, IOrderRepository, and IOrderEventPublisher. It contains no DB access, VAT/shipping math, or e-mail logic of its own.

Pros: The controller depends on one simple interface; the workflow lives in one cohesive place; subsystem changes don't ripple to the boundary (low coupling, high cohesion).